

CINEVERSE-LARAVEL

Documentación técnica

Tecnologías

Vue 3 · Laravel 12 · Inertia 2 · TypeScript · PHP · MySQL 8 · TMDB API

Documento técnico del proyecto · Laravel 12 + Vue 3 + Inertia.js 2

Autor: Gastón Rizzo

Índice

1. Descripción general de CineVerse	3
2. Tecnologías, entorno y conceptos fundamentales	4
3. Inertia: cómo se conectan Laravel y Vue en CineVerse	9
4. TMDB: qué es, cómo se obtiene el acceso y cómo lo utiliza CineVerse	14
5. Arquitectura general de CineVerse Laravel	22
6. Estructura del proyecto	23
7. Archivos de configuración y variables de entorno	25
8. Base de datos MySQL, migraciones y Eloquent	28
9. Datos de prueba y seeders	30
10. Cómo recorre CineVerse una acción del usuario	31
11. TypeScript en CineVerse	32
12. Cómo está organizado el frontend con Vue	35
13. Cómo se guardan y reutilizan datos en el frontend	38
14. Navegación sin recargar la página: SPA y Vue Router	41
15. Idiomas: vue-i18n y cambio de idioma	43
16. Comunicación entre Vue, TMDB y Laravel	47
17. Explorador de películas y sistema de búsqueda	50
18. Cómo se guardan los filtros y se reflejan en la URL	56
19. Cómo se cargan más películas	59
20. Página de inicio	61
21. Detalle de una película	63
22. Trailers, películas similares e imágenes	65
23. Actores y filmografía	67
24. Favoritos	68
25. Cómo está organizado el backend Laravel	71
26. Autenticación y cuenta: Breeze, sesiones y recuperación	75
27. Reseñas y perfil	81
28. Validación, seguridad, CSRF y RateLimiter	87
29. Estados de la interfaz y manejo de errores	90
30. Logs: registro y diagnóstico de errores	94
31. Instalación local de CineVerse Laravel	98
32. Cómo ejecutar CineVerse Laravel en desarrollo	102
33. Cómo publicar CineVerse Laravel en un servidor	104
34. Pruebas y comprobaciones técnicas	105

1. Descripción general de CineVerse

CineVerse es una aplicación web dedicada a dar información sobre películas y actores. Desde el navegador se pueden recorrer tendencias y estrenos, buscar títulos, combinar filtros, abrir el detalle de una película, consultar reparto y filmografía, reproducir trailers, guardar favoritos y cambiar la interfaz entre español e inglés. Cuando el visitante crea una cuenta y verifica su correo, también puede publicar reseñas, modificarlas, eliminarlas y consultar su actividad desde el perfil.

La versión documentada aquí es la integración Laravel del proyecto. El frontend continúa estando construido con Vue 3 y TypeScript, pero ya no vive como una aplicación separada de un backend PHP puro. Vue está incluido dentro del mismo repositorio Laravel, bajo `resources/js/cineverse`, y la entrada del navegador se realiza a través de Laravel, Inertia y Vite.

Dos clases de información

La información cinematográfica no se copia a MySQL. Películas, personas, posters, créditos y videos se consultan en TMDB cuando la interfaz los necesita. MySQL conserva los datos propios de CineVerse: usuarios, reseñas, sesiones, tokens de recuperación y tablas de infraestructura de Laravel.

1.1 Qué cambia respecto de la versión PHP pura

El cambio importante no está en la apariencia de CineVerse, sino en la arquitectura del servidor. El antiguo recorrido `public/index.php` → Router propio → Middleware → Controller → Service → Model → PDO deja de ser la base del backend. Laravel aporta su router, middleware, controladores, Form Requests, autenticación, Eloquent, migraciones, sesiones y manejo de excepciones. Esto permite reemplazar varias soluciones manuales por mecanismos estándar del framework sin perder el frontend original.

También cambia la forma de instalar el proyecto. En lugar de importar un archivo SQL que crea todo el escenario, el esquema se reconstruye mediante migraciones. Los datos de prueba se cargan con seeders a partir de archivos NDJSON. El repositorio no necesita incluir `vendor`, `node_modules`, `.env` ni `public/build` porque esos elementos se reconstruyen localmente.

1.2 Qué se mantuvo deliberadamente

Vue Router sigue siendo el router interno de la interfaz. Esto es una decisión importante: Inertia no sustituyó la navegación SPA de CineVerse. Laravel entrega una única página Inertia llamada CineVerse y esa página monta CineVerseApp; a partir de allí, Vue Router decide si corresponde `HomeView`, `MoviesView`, `MovieDetailView`, `LoginView`, `ProfileView` u otra vista.

También se conserva el acceso directo de Vue a TMDB. Laravel no funciona como proxy del catálogo cinematográfico. La separación permite que las consultas de películas y personas continúen utilizando TanStack Vue Query y los servicios del frontend, mientras Laravel se concentra en identidad, seguridad y datos propios.

1.3 Cómo leer el documento

Cada mecanismo se explica desde una situación concreta antes de entrar en nombres técnicos. Cuando Carlos busca una película, abre una pestaña, inicia sesión o publica una reseña, el documento sigue la acción desde la View o el Component que recibe el gesto hasta el Service, la ruta HTTP, Laravel o TMDB, la base de datos cuando corresponde y, finalmente, la vuelta de los datos a la interfaz.

Cuando se muestra código fuente real, el bloque se precede por “Ubicación en el proyecto” e identifica el archivo, el tipo de pieza y, cuando existe, la clase, método o función responsable. Los diagramas con flechas se rotulan como recorridos explicativos: resumen el flujo entre piezas, pero no se presentan como si fueran líneas literales de un archivo. Esta distinción permite estudiar el proyecto sin confundir código real con un esquema pedagógico.

Regla de lectura: primero se explica qué hace la persona, luego dónde ocurre, qué datos entran, qué código interviene, qué respuesta vuelve y qué termina viendo el usuario.

Laravel es el framework PHP que organiza el servidor de esta versión. No reemplaza el frontend Vue ni el catálogo TMDB: reemplaza principalmente la infraestructura backend que antes CineVerse resolvía con un Router, sesiones, validadores, Models PDO, Response y logging propios.

1.4 Conceptos que se repiten durante todo el documento

Antes de seguir archivos concretos conviene fijar qué significa cada tipo de pieza dentro de CineVerse Laravel. Los nombres no son etiquetas teóricas: permiten leer un recorrido completo y saber qué responsabilidad tiene cada archivo.

Concepto	Qué significa en CineVerse Laravel	Ejemplo real
View	Pantalla completa asociada normalmente a una ruta interna de Vue Router.	MoviesView.vue, LoginView.vue, ProfileView.vue
Component	Parte visual reutilizable de una pantalla. Puede recibir datos del componente padre (props) y emitir eventos para avisarle que ocurrió una acción.	MovieCard.vue, MovieHero.vue, MovieReviews.vue
Composable	Función de Vue que reúne lógica reactiva reutilizable y puede coordinar consultas, filtros, estado y services sin convertirse en una pantalla.	useMoviesQuery.ts, useMovieDetail.ts, usePersonDetail.ts
Service del frontend	Módulo TypeScript que conoce cómo hablar con una API o con Laravel. No dibuja la interfaz.	movies.service.ts, person.service.ts, auth.service.ts, reviews.service.ts
Store	Estado compartido entre distintas partes del frontend.	useAuthStore, useFavoritesStore
API	Interfaz HTTP que permite pedir o enviar datos a otro sistema.	TMDB API y endpoints JSON de Laravel
Endpoint	Dirección concreta de una API que representa una operación.	/search/movie, POST /login, POST /api/reviews
JSON	Formato de texto estructurado utilizado para intercambiar datos.	Respuesta paginada de TMDB o respuesta JSON de ReviewController
Query parameter	Dato que viaja después de ? dentro de una URL.	page=1&language=es

Regla de lectura. Una View no debería confundirse con un Service: la View coordina lo que ve la persona; el Service conoce el destino HTTP. Un Composable conecta ambos mundos sin convertirse en una pantalla.

2. Tecnologías, entorno y conceptos fundamentales

CineVerse combina tecnologías que se ejecutan en el servidor con tecnologías que se ejecutan en el navegador. Cada una cumple una tarea concreta: Laravel organiza la capa de servidor; Inertia enlaza esa capa con Vue; Vue Router conserva la navegación interna; MySQL guarda datos propios; TMDB proporciona el catálogo; Vite prepara y sirve los archivos del frontend —los llamados assets—; Tailwind CSS aporta la base global importada por app.css; y varias librerías del frontend resuelven carruseles, iconos, controles de filtros, arrastrar y soltar y persistencia de estado.

Tecnología	Versión/constraint del proyecto	Responsabilidad principal
PHP	composer.json: ^8.2	Ejecuta Laravel y Composer.

Tecnología	Versión/constraint del proyecto	Responsabilidad principal
Laravel	framework ^12.0; lock 12.62.0	Rutas, sesiones, validación, controladores, Eloquent, migraciones y excepciones.
Inertia Laravel	^2.0	Entrega Pages/CineVerse.vue desde Laravel.
Vue 3	^3.4.0	Componentes, reactividad y vistas de la interfaz.
Vue Router	^5.1.0	Rutas internas /es, /movies, /profile, etc.
TypeScript	^5.6.3	Tipos estáticos del frontend y build con vue-tsc.
Vite	^7.0.7	Servidor de archivos del frontend (assets) durante desarrollo y construcción del paquete de producción.
Pinia	^3.0.4	Estado global de autenticación y favoritos.
TanStack Vue Query	^5.101.2	Consultas, caché, paginación y estados de datos remotos.
vue-i18n	^11.4.6	Interfaz ES/EN.
Axios	^1.11.0	HTTP hacia TMDB y Laravel.
Laravel Breeze	^2.4, require-dev	Estructura inicial de código para autenticación (scaffolding), adaptada por CineVerse.
Sanctum	^4.0	Instalado, aunque las rutas principales usan sesión web.
Ziggy	^2.0	Disponibiliza rutas Laravel en Vue.
MySQL	configurable por .env	Usuarios, reseñas, sesiones, caché, jobs y tokens.
TMDB API v3	VITE_TMDB_BASE_URL	Películas, personas, créditos, videos e imágenes.
Tailwind CSS	4.2.1; @tailwindcss/vite 4.2.1	Base/reset global importado por resources/css/app.css; la interfaz CineVerse utiliza principalmente CSS propio. tailwind.config.js registra además @tailwindcss/forms.
pinia-plugin-persistedstate	^4.7.1	Persiste el store de favoritos y otros estados configurados en localStorage.
Swiper	^14.0.5	Carruseles de la página de inicio.
lucide-vue-next	^1.0.0	Iconos reutilizables en componentes Vue.
vue-multiselect	^3.5.0	Controles de selección múltiple del explorador.
vue-slider-component	^4.1.0-beta.7	Sliders de año y puntuación en los filtros.
vuedraggable	^4.1.0	Arrastrar y reordenar paneles personalizables del explorador.

2.1 Qué es un framework

Un framework es una base de trabajo que ya trae una estructura y herramientas para resolver tareas repetidas de una aplicación. En un proyecto PHP sin framework, el equipo puede programar manualmente el enrutamiento, las sesiones, la validación, el acceso a la base de datos, el manejo de errores y muchas otras piezas. Laravel ofrece mecanismos estándar para esas tareas y CineVerse los utiliza como columna vertebral del servidor.

Idea clave: Laravel no reemplaza a Vue ni a TMDB. En CineVerse reemplaza principalmente la infraestructura backend que antes se resolvía con clases PHP propias.

2.2 Qué es Laravel y qué hace dentro de CineVerse

Laravel es un framework web para PHP. Recibe solicitudes HTTP, compara la URL y el método con las rutas registradas, ejecuta middleware, entrega la operación al Controller correspondiente, valida datos, administra autenticación y sesión, utiliza Eloquent para trabajar con MySQL y finalmente construye una respuesta HTTP. La versión instalada en este proyecto es Laravel 12.62.0, resuelta por composer.lock a partir de laravel/framework ^12.0.

```
Navegador / Vue
-> Laravel recibe la request
-> Router encuentra una ruta
-> middleware
-> Request / Form Request
-> Controller
-> Auth / Eloquent / Notification / otros servicios del framework
-> MySQL o respuesta directa
-> Response HTTP
-> Vue
```

La misma aplicación puede atravesar sólo una parte de ese recorrido. Por ejemplo, la carga de una película desde TMDB no pasa por Eloquent ni MySQL; en cambio, crear una reseña sí llega a Laravel, valida sesión y datos, y termina escribiendo en la tabla reviews.

2.3 Qué cambió respecto del backend PHP puro

En la versión anterior de CineVerse existían un Router propio, clases Request y Response, manejo manual de sesiones y CSRF, Services de negocio, Models con PDO y un AppLogger propio. La versión Laravel conserva las mismas necesidades funcionales, pero muchas de esas responsabilidades pasan a mecanismos del framework.

Backend PHP anterior	CineVerse Laravel
Router propio	Router de Laravel
Session.php	sesión del guard web
AuthService / AuthController	Auth + Controllers + LoginRequest
Validator / AuthValidator	validation + Form Requests
Models con PDO	Eloquent Models / Query Builder
Response::json()	response()->json()
AppLogger + Monolog manual	Log facade + canal Laravel + Monolog
SQL de estructura	migrations
script de datos de prueba	seeders

La comparación no significa que Laravel elimine la lógica del proyecto. Las reglas específicas de CineVerse siguen existiendo: una sola reseña por usuario y película, correo obligatorio antes del login, límite de favoritos local, idioma de los emails y forma de responder al frontend. Lo que cambia es la infraestructura utilizada para aplicar esas reglas.

2.4 Artisan: la consola de Laravel

Artisan es la interfaz de línea de comandos incluida con Laravel. El archivo artisan está en la raíz del proyecto. Cuando se ejecuta `php artisan <comando>`, PHP arranca Laravel en modo consola y el framework ejecuta la tarea solicitada.

```
php artisan serve          -> servidor de desarrollo
php artisan key:generate   -> genera APP_KEY
php artisan migrate        -> ejecuta migraciones pendientes
php artisan db:seed        -> ejecuta DatabaseSeeder
php artisan migrate:fresh --seed -> reconstruye deliberadamente el esquema y carga datos de prueba
php artisan test           -> ejecuta la suite PHPUnit de Laravel
```

Artisan no es un servidor separado ni una librería adicional. Es una puerta de entrada al mismo proyecto Laravel para tareas de consola. Por eso comandos como migrate conocen config/database.php, .env y las clases de database/migrations.

2.5 PHP, Composer, autoload y vendor

composer.json exige PHP ^8.2. Esto significa que el proyecto necesita PHP 8.2 o una versión posterior compatible dentro de la misma política semántica de Composer. La captura del entorno utilizado para esta documentación muestra PHP 8.2.12. Composer lee composer.json/composer.lock, descarga las librerías dentro de vendor y ejecuta los scripts de descubrimiento de paquetes de Laravel.

vendor no se guarda en Git. Es una consecuencia importante al instalar desde un repositorio limpio: tener composer.json no significa que Laravel ya pueda ejecutarse; primero hay que reconstruir vendor con composer install.

2.6 Node.js, npm y Vite

package.json define npm run dev como vite y npm run build como vue-tsc && vite build. El proyecto no declara un campo engines con una versión concreta de Node.js; por eso este documento no inventa una versión mínima. npm install reconstruye node_modules a partir de package-lock.json y deja disponible el ejecutable local de Vite que npm run dev necesita.

Vite recibe dos entradas desde vite.config.js: resources/css/app.css y resources/js/app.ts. También conoce los alias @/features, @/core, @/layouts, @/shared y otros, que apuntan a la estructura integrada del frontend bajo resources/js/cineverse.

2.7 Inertia: el puente entre Laravel y Vue

Inertia es la capa que conecta las rutas web de Laravel con la aplicación Vue. En CineVerse no reemplaza a Vue Router: Laravel e Inertia entregan la aplicación y, una vez montada, Vue Router decide qué pantalla interna debe mostrarse.

La conexión completa se desarrolla en profundidad en la sección 3, donde se sigue el recorrido desde routes/web.php hasta MovieDetailView.vue con el código fuente real del proyecto.

2.8 Blade: dónde se utiliza en esta versión

Blade es el sistema de plantillas de Laravel. La interfaz principal de CineVerse no está construida con Blade: está construida con Vue. Blade aparece en funciones concretas del servidor.

- resources/views/app.blade.php actúa como documento HTML raíz de Inertia y carga los assets de Vite.
- resources/views/emails/*.blade.php contiene las plantillas HTML de verificación y recuperación de contraseña.
- resources/views/errors/database.blade.php muestra una página HTML controlada cuando MySQL no está disponible durante una navegación web.

Esta distinción evita una confusión común: una aplicación Laravel puede utilizar Blade y Vue al mismo tiempo, pero no necesariamente para las mismas pantallas.

2.9 Eloquent, migraciones y seeders: mapa rápido

Eloquent es el ORM de Laravel. Un ORM permite trabajar con filas de una base relacional mediante modelos y consultas construidas desde PHP. CineVerse tiene modelos como User y Review. Eloquent no reemplaza a MySQL: genera y ejecuta consultas sobre MySQL y transforma los resultados en objetos que Laravel puede utilizar.

Esquema conceptual basado en consultas Eloquent utilizadas por los Controllers de CineVerse; no corresponde a un bloque literal copiado de un único archivo.

```
Review::query()
-> where("user_id", $userId)
-> orderByDesc("created_at")
-> get()
-> SQL ejecutado sobre MySQL
-> colección de modelos Review
```

Una migración describe cambios de estructura de la base, mientras un seeder carga datos de prueba. database/migrations contiene la historia reproducible del esquema y database/seeds contiene el escenario de 5.000 usuarios y 24.322 reseñas.

2.10 Middleware, Request, Form Request y Controller

Estas piezas aparecen en casi todos los recorridos del backend y conviene diferenciarlas desde el principio.

- **Middleware:** se ejecuta alrededor de una ruta y puede permitir, transformar o cortar la request antes de que llegue al Controller. `auth`, `guest`, `signed` y `throttle` son ejemplos reales del proyecto.
- **Request:** representa la solicitud HTTP actual; permite leer JSON, query parameters, archivos, usuario autenticado, sesión y cabeceras.
- **Form Request:** es una clase especializada de Laravel que encapsula preparación, autorización y validación. `LoginRequest` además ejecuta la autenticación y `RateLimiter`.
- **Controller:** coordina la acción final de una ruta. Recibe el Request o Form Request, utiliza modelos/facades y devuelve una respuesta.

```
POST /login
-> middleware guest
-> LoginRequest prepara y valida
-> AuthenticatedSessionController::store()
-> session()->regenerate()
-> HTTP 204
```

2.11 Facades y servicios del framework que aparecen en CineVerse

Laravel ofrece accesos sencillos a servicios del framework mediante clases conocidas como facades. En CineVerse aparecen porque permiten utilizar funciones ya configuradas —autenticación, hashing, límites de intentos, recuperación de contraseña, logs y URLs firmadas— sin construir esos mecanismos desde cero.

- **Auth:** abre o cierra la sesión mediante el guard web, que es el mecanismo de Laravel que relaciona una solicitud con el usuario autenticado en la sesión.
- **Hash:** crea y comprueba hashes de contraseñas. Un hash es una representación criptográfica que permite verificar una contraseña sin guardar su texto original.
- **RateLimiter:** cuenta intentos y decide cuándo bloquear temporalmente un login.
- **Password:** utiliza el Password Broker, el mecanismo de Laravel que coordina la creación y validación de tokens para restablecer una contraseña.
- **Log:** escribe registros mediante los canales definidos en `config/logging.php`.
- **URL:** genera la URL temporal firmada utilizada por la verificación de correo.

Estas facades no son bases de datos ni procesos separados. Son formas de acceder a servicios que Laravel ya tiene preparados; su comportamiento se apoya en la configuración de `config/*.php` y en los providers del framework.

2.12 Breeze: estructura inicial de autenticación (scaffolding), no motor separado

Laravel Breeze aparece en `require-dev`. Su papel fue aportar una estructura inicial de código para autenticación —lo que en desarrollo se denomina *scaffolding*—: controladores como `AuthenticatedSessionController` o `RegisteredUserController`, `LoginRequest`, rutas `auth` y pruebas `Feature/Auth`. CineVerse tomó esa base y la adaptó. Breeze no permanece ejecutándose como una capa independiente: en tiempo de ejecución el login lo realizan `Auth`, `Hash`, `RateLimiter`, la sesión del guard web y `App\Models\User`.

CineVerse además cambió comportamiento respecto del Breeze inicial. El registro crea la cuenta y envía un email de verificación, pero no inicia sesión automáticamente. `LoginRequest` exige correo verificado antes de `Auth::login`. Las pantallas visibles son componentes propios de CineVerse, no las páginas generadas originalmente por Breeze.

2.13 Sanctum está instalado, pero no define el login principal

`laravel/sanctum` está presente en `composer.json` y existe `config/sanctum.php`, pero las rutas funcionales de cuenta y reseñas utilizan `middleware web y auth`. `config/auth.php` define el guard web con `driver session`: el guard

determina cómo Laravel identifica al usuario de una solicitud y el driver session indica que esa identidad se conserva mediante la sesión. Por eso describir CineVerse como una aplicación autenticada mediante tokens Sanctum sería incorrecto para esta versión: el flujo principal utiliza sesión y cookie de Laravel.

3. Inertia: cómo se conectan Laravel y Vue en CineVerse

Inertia es la pieza que permite que Laravel entregue una aplicación construida con Vue sin convertir cada pantalla en una plantilla Blade independiente. En CineVerse funciona como un puente de entrada: Laravel recibe la navegación del navegador, decide entregar la página Inertia CineVerse y, a partir de esa página, se monta la aplicación Vue existente.

El término puente no significa que exista una única clase que haga todo el trabajo. La conexión se construye entre varias piezas del proyecto: `routes/web.php` decide la página Inertia; `HandleInertiaRequests` configura la vista raíz y los datos compartidos; `app.blade.php` prepara el documento HTML; `app.ts` inicia Inertia y Vue; `Pages/CineVerse.vue` adapta la página que Inertia espera a la aplicación real; y `cineverse/App.vue` entrega finalmente la vista central a Vue Router.

3.1 Qué problema resuelve Inertia en este proyecto

CineVerse tiene un backend Laravel y una interfaz Vue dentro del mismo repositorio. El navegador necesita entrar por Laravel porque Laravel controla las rutas web, la sesión, los middleware y la respuesta HTTP inicial. Sin embargo, la interfaz principal no está escrita con Blade, sino con componentes Vue. Inertia actúa como puente entre Laravel y Vue y permite integrar ambas partes dentro de la misma aplicación.

En este proyecto Inertia no reemplaza a Vue Router. Su función principal termina después de entregar y montar la aplicación CineVerse. Una vez que Vue está funcionando en el navegador, Vue Router interpreta rutas internas como `/es/movies/550`, `/es/profile` o `/en/favorites` y decide qué View debe aparecer dentro de RouterView.

```
Navegador
-> Laravel
-> Inertia
-> página Vue CineVerse
-> cineverse/App.vue
-> Vue Router
-> View concreta
```

3.2 Qué significa exactamente `Inertia::render("CineVerse")`

La conexión comienza en `routes/web.php`. La ruta raíz redirige a `/es`. La ruta principal `/locale` y el catch-all `/locale/{any}` entregan la misma página lógica de Inertia: CineVerse. Las rutas visibles de autenticación `/locale/{authPage}` también la entregan para invitados, pero redirigen al inicio si ya existe una sesión. El nombre CineVerse no es todavía `MovieDetailView`, `HomeView` ni `ProfileView`; es el nombre de la página que Inertia debe resolver en el frontend.

Ubicación en el proyecto: `routes/web.php` · **Rutas web Laravel** · rutas `/locale` y catch-all

Lo utiliza: navegador / Inertia. Responsabilidad: entregar la página Inertia CineVerse.

```
Route::get('/{locale}', function (string $locale) {
    return Inertia::render('CineVerse', [
        'locale' => $locale,
    ]);
})->whereIn('locale', ['es', 'en']);

Route::get('/{locale}/{any}', function (string $locale) {
    return Inertia::render('CineVerse', [
        'locale' => $locale,
    ]);
})->whereIn('locale', ['es', 'en'])
->where('any', '.*');
```

Si el navegador solicita `/es/movies/550`, Laravel no busca una página Inertia llamada `MovieDetail`. La ruta `catch-all` reconoce `locale="es"` y `any="movies/550"`, pero responde igualmente con `Inertia::render('CineVerse')`. La URL profunda se conserva; después Vue Router la utiliza para seleccionar `MovieDetailView`.

3.3 HandleInertiaRequests: la clase Laravel que configura la integración

`app/Http/Middleware/HandleInertiaRequests.php` extiende `Inertia\Middleware`. Esta es la clase específica de Laravel donde el proyecto configura aspectos globales de Inertia. No monta Vue por sí sola, pero define qué vista Blade sirve de documento raíz y qué props globales se comparten con las páginas Inertia.

Ubicación en el proyecto: `app/Http/Middleware/HandleInertiaRequests.php` · **Middleware Inertia** · `HandleInertiaRequests::share()`

Lo utiliza: grupo `web`. *Responsabilidad:* definir `rootView` y compartir `auth.user`.

```
class HandleInertiaRequests extends Middleware
{
    protected $rootView = 'app';

    public function share(Request $request): array
    {
        return [
            ...parent::share($request),
            'auth' => [
                'user' => $request->user(),
            ],
        ];
    }
}
```

`protected $rootView = 'app'` hace que la vista raíz sea `resources/views/app.blade.php`. El método `share()` agrega `auth.user` a los datos globales de Inertia. Ese dato compartido no sustituye el `GET /user` que `useAuthStore` utiliza después para reconstruir la sesión y obtener `csrf_token`; son dos mecanismos diferentes que conviven en el proyecto.

3.4 app.blade.php: la carcasa HTML que Laravel devuelve

La vista `resources/views/app.blade.php` es el documento HTML inicial. No construye Inicio, Películas o Perfil. Su trabajo es preparar el contenedor donde Inertia montará Vue y cargar los assets necesarios mediante Vite.

Ubicación en el proyecto: `resources/views/app.blade.php` · **Vista Blade raíz** · `app.blade.php`

Lo utiliza: `HandleInertiaRequests::$rootView`. *Responsabilidad:* cargar Vite e insertar el punto de montaje de Inertia.

```
<title inertia>{{ config('app.name', 'Laravel') }}</title>

@routes
@vite(['resources/js/app.ts', "resources/js/Pages/{$page['component']}.vue"])
@inertiaHead

<body>
    @inertia
</body>
```

`@vite` carga `resources/js/app.ts` y el componente de la página Inertia actual. La expresión `$page['component']` contiene el nombre lógico enviado por Inertia; cuando Laravel hizo `Inertia::render('CineVerse')`, ese componente es `CineVerse`. `@inertia` genera el punto de montaje y la información inicial de la página que el cliente de Inertia necesita para iniciar la aplicación.

3.5 app.ts: donde el nombre enviado por Laravel se convierte en un componente Vue

`resources/js/app.ts` es el entrypoint del frontend integrado. `createInertiaApp()` recibe la página enviada por Laravel, resuelve su componente Vue y crea la única instancia principal de Vue utilizada por `CineVerse`.

Ubicación en el proyecto: resources/js/app.ts · **Entrypoint Vue/Inertia** · createInertiaApp()

Lo utiliza: resources/views/app.blade.php /Inertia. *Responsabilidad:* resolver Pages/CineVerse.vue e instalar Pinia, Router, Vue Query, i18n y Ziggy.

```
createInertiaApp({
  resolve: (name) =>
    resolvePageComponent(
      `./Pages/${name}.vue`,
      import.meta.glob<DefineComponent>('./Pages/**/*.vue'),
    ),

  setup({ el, App, props, plugin }) {
    const pinia = createPinia();
    pinia.use(piniaPluginPersistedState);

    const app = createApp({
      render: () => h(App, props),
    });

    app
      .use(plugin)
      .use(pinia)
      .use(router)
      .use(VueQueryPlugin)
      .use(i18n)
      .use(ZiggyVue);

    useAuthStore().initialize();

    router.isReady().then(() => {
      app.mount(el);
    });
  },
});
```

El punto clave está en resolve. Laravel envió el nombre CineVerse; por eso name vale "CineVerse". La plantilla `./Pages/\${name}.vue` se transforma en ./Pages/CineVerse.vue y resolvePageComponent carga ese archivo.

```
Laravel:
Inertia::render('CineVerse')

    -> nombre de página

app.ts:
name = 'CineVerse'

    -> resolve()

./Pages/CineVerse.vue
```

setup() recibe cuatro piezas importantes. el es el elemento HTML donde se montará Vue; App es el componente raíz que proporciona Inertia; props contiene los datos de la página; plugin instala la integración de Inertia dentro de Vue. El App recibido por setup() no debe confundirse con resources/js/cineverse/App.vue: son componentes distintos con nombres parecidos.

createApp({ render: () => h(App, props) }) crea la aplicación Vue que renderiza el componente raíz de Inertia con las props recibidas. Después esa misma instancia registra Pinia, Vue Router, Vue Query, vue-i18n y Ziggy. No existen dos aplicaciones Vue separadas: Inertia y CineVerse terminan dentro de la misma instancia.

3.6 Pages/CineVerse.vue: el adaptador específico de CineVerse

Inertia busca sus páginas bajo resources/js/Pages, pero la arquitectura original de CineVerse vive bajo resources/js/cineverse. Para no mover toda la aplicación a Pages, el proyecto utiliza un adaptador mínimo.

Ubicación en el proyecto: resources/js/Pages/CineVerse.vue · **Página Vue de Inertia** · CineVerse.vue

Lo utiliza: resources/js/app.ts. *Responsabilidad:* adaptar la página que Inertia resuelve a cineverse/App.vue.

```
<script setup lang="ts">
import CineVerseApp from '@cineverse/App.vue';
</script>

<template>
  <CineVerseApp />
</template>
```

Este archivo es la unión más visible entre el convenio de Inertia y la estructura propia del frontend. Inertia puede resolver Pages/CineVerse.vue por nombre y, a su vez, esa página monta resources/js/cineverse/App.vue. Por eso HomeView, MoviesView, ProfileView y las demás vistas pueden conservar su organización original.

3.7 cineverse/App.vue: donde termina el puente y empieza Vue Router

Ubicación en el proyecto: resources/js/cineverse/App.vue · Component raíz de la SPA CineVerse · template con RouterView. Lo utiliza Pages/CineVerse.vue al montar CineVerseApp. AppHeader y AppFooter quedan fuera de RouterView; RouterView es el lugar donde Vue Router coloca la View correspondiente a la URL actual.

```
<div class="app-content">
  <RouterView v-slot="{ Component, route }">
    <component
      :is="Component"
      :key="routeKey(route)"
    />
  </RouterView>
</div>
```

Para /es/movies/550, Vue Router encuentra la ruta MovieDetail y selecciona MovieDetailView; ese trabajo ya no lo realiza Inertia. Ubicación en el proyecto: resources/js/cineverse/core/router/index.ts · router global · definición de la ruta MovieDetail. El fragmento siguiente pertenece a ese archivo y muestra la ruta con :lang y el id numérico de la película.

```
{
  path: "[:lang(en|es)/movies/:id(\\d+)",
  name: "MovieDetail",
  component: MovieDetailView,
}
```

La separación de responsabilidades queda así: Laravel decide qué aplicación entregar en una navegación HTTP completa; Inertia conecta esa respuesta con Vue; Vue Router decide qué View interna mostrar una vez que la aplicación ya está montada.

3.8 Ejemplo completo: Carlos pega /es/movies/550 y presiona Enter

Este recorrido comienza fuera de la SPA. El navegador todavía no tiene CineVerse montado, por lo que la solicitud llega primero a Laravel.

```

1. Navegador -> GET /es/movies/550
2. Laravel compara la URL con routes/web.php
3. Coincide /{locale}/{any}
4. locale = "es" y any = "movies/550"
5. Laravel ejecuta Inertia::render("CineVerse", ["locale" => "es"])
6. HandleInertiaRequests selecciona rootView = "app"
7. Laravel devuelve resources/views/app.blade.php
8. @vite carga resources/js/app.ts
9. @inertia prepara el punto de montaje de la página
10. createInertiaApp recibe component = "CineVerse"
11. resolve() carga ./Pages/CineVerse.vue
12. setup() crea la instancia Vue y registra los plugins
13. CineVerse.vue monta cineverse/App.vue
14. Vue Router lee la URL /es/movies/550
15. Vue Router selecciona MovieDetailView
16. RouterView monta MovieDetailView
17. useMovieDetail inicia las consultas a TMDB

```

La película no fue elegida por Laravel ni por Inertia. Laravel sólo consiguió que la aplicación correcta llegara al navegador. La selección de MovieDetailView ocurre cuando Vue Router interpreta la misma URL después del montaje.

3.9 Qué cambia en una navegación interna sin F5

Supongamos ahora que CineVerse ya está abierto y Carlos pulsa otra MovieCard. En ese caso la aplicación Vue ya existe. Vue Router puede cambiar la ruta y reemplazar la View dentro de RouterView sin volver a pedir a Laravel la página CineVerse para cada pantalla.

```

MovieCard
-> router.push({ name: "MovieDetail", lang, id })
-> Vue Router actualiza la URL
-> RouterView recibe la nueva ruta
-> MovieDetailView se monta o se remonta según routeKey
-> useMovieDetail consulta los datos necesarios

```

En esta navegación interna no se vuelve a ejecutar el recorrido completo routes/web.php -> Inertia::render() -> app.blade.php -> createInertiaApp(). Si el usuario presiona F5 sobre la nueva URL, entonces sí vuelve a comenzar una navegación HTTP completa y Laravel debe entregar nuevamente la aplicación.

3.10 Inertia y Vue Router no son dos routers compitiendo

La posible confusión nace porque Laravel, Inertia y Vue Router participan alrededor de una misma URL. Sin embargo, trabajan en momentos distintos. Laravel resuelve la entrada HTTP; Inertia entrega la página Vue; Vue Router resuelve la pantalla interna.

```

CARGA / RECARGA COMPLETA
Browser -> Laravel -> Inertia -> Vue -> Vue Router -> View

NAVEGACIÓN INTERNA
View / componente -> Vue Router -> RouterView -> nueva View

```

CineVerse eligió mantener Vue Router porque el frontend ya era una SPA completa antes de integrarse con Laravel. Por eso no existen Pages/Home.vue, Pages/Movies.vue y Pages/Profile.vue como páginas Inertia separadas. Existe una única página puente, Pages/CineVerse.vue, y detrás de ella continúa la arquitectura Vue Router original.

3.11 Qué datos puede compartir Laravel mediante Inertia

HandleInertiaRequests comparte auth.user mediante share(). Esas props pueden viajar con la página Inertia y estar disponibles en el frontend. En esta versión, CineVerse además mantiene un contrato explícito GET /user para que useAuthStore reconstruya user y csrfToken a partir de la sesión Laravel después de una recarga.

```
Laravel / Inertia:
share() -> auth.user

CineVerse auth store:
GET /user -> user + csrf_token -> Pinia
```

La presencia de ambas vías no significa que existan dos autenticaciones. La identidad real sigue siendo la sesión de Laravel. Inertia puede compartir contexto de la request inicial y Pinia mantiene una representación reactiva para la interfaz.

3.12 Qué no hace Inertia en CineVerse

Inertia no consulta TMDB, no guarda reseñas, no reemplaza Eloquent, no autentica por sí mismo, no sustituye a Axios y no decide qué película muestra MovieDetailView. Tampoco reemplaza a Vue Router en esta arquitectura.

Su responsabilidad central puede resumirse así: Laravel decide entregar la página CineVerse; Inertia transporta esa decisión hasta Vue y permite montar la aplicación; después Vue Router toma el control de las vistas internas.

4. TMDB: qué es, cómo se obtiene el acceso y cómo lo utiliza CineVerse

TMDB ofrece una API externa que permite a CineVerse consultar información estructurada sobre películas y personas. Una API es una interfaz mediante la cual un programa puede solicitar datos a otro sistema; dentro de esa API, cada dirección concreta que representa una operación se denomina endpoint. CineVerse reúne la configuración común de acceso a TMDB en `tmdbApi.ts` y deja que `movies.service.ts` y `person.service.ts` indiquen después qué endpoint necesitan consultar, con qué parámetros y qué tipo de respuesta esperan recibir.

4.1 Qué es The Movie Database (TMDB)

TMDB significa The Movie Database. CineVerse no instala TMDB ni copia su sitio web: utiliza su APIv3, un servicio externo en Internet que permite consultar automáticamente información de películas y personas mediante solicitudes HTTP. MySQL conserva los datos propios de CineVerse; TMDB aporta el catálogo audiovisual.

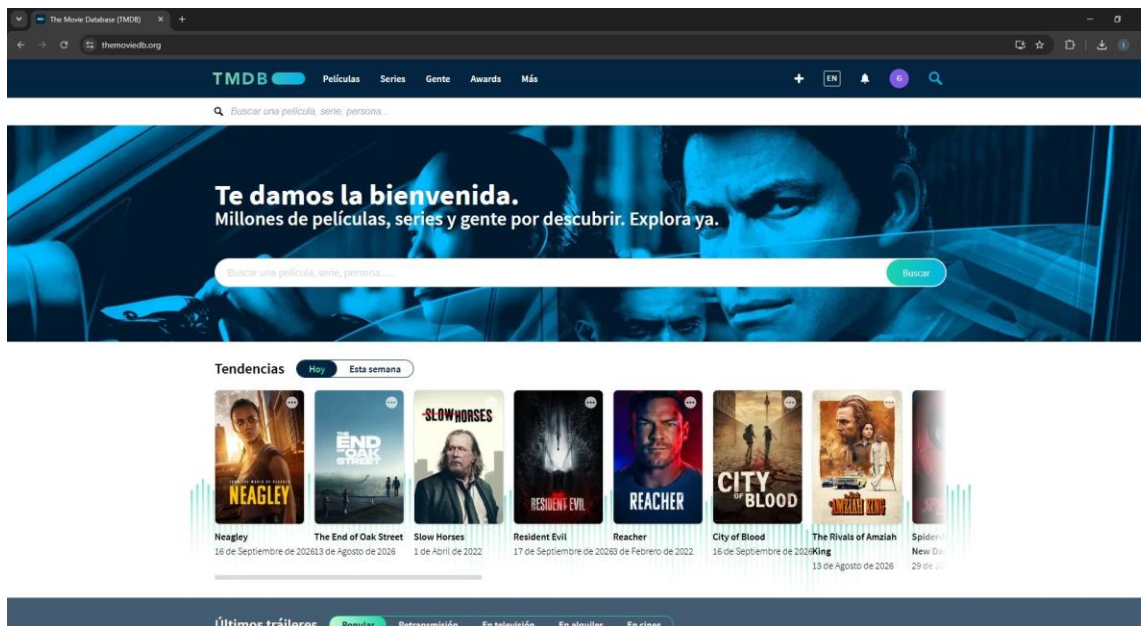


Figura. Sitio público de The Movie Database. CineVerse consume la API, no copia esta página HTML.

Dirección / sistema	Función
www.themoviedb.org	Sitio web que visita una persona.
api.themoviedb.org	API a la que tmdbApi.ts envía solicitudes HTTP.
image.tmdb.org	Servidor de imágenes utilizado para pósters, backdrops y fotos.
MySQL de CineVerse	Usuarios, sesiones, reseñas y demás datos propios; no contiene el catálogo completo de TMDB.

4.2 Qué información obtiene CineVerse de TMDB

TMDB aporta títulos, sinopsis, fechas, géneros, puntuaciones, pósters, fondos, reparto, equipo técnico, videos, películas similares y datos de personas. CineVerse selecciona los campos que necesita y combina varios endpoints cuando una pantalla requiere información de distinta naturaleza.

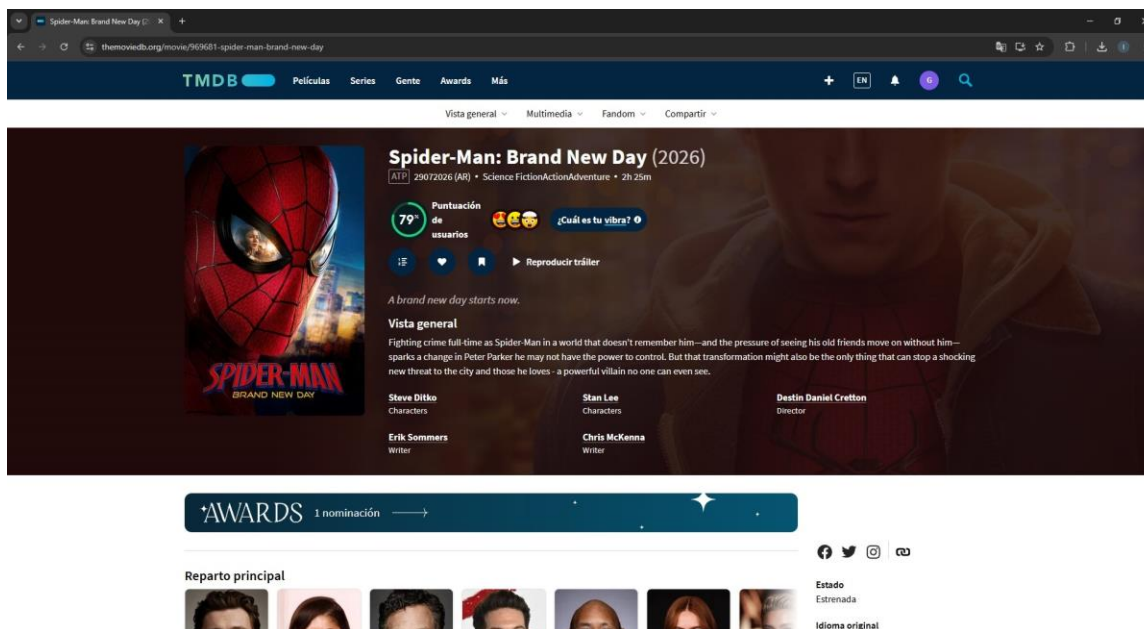


Figura. Ficha pública de una película en TMDB.

La separación con MySQL es visible en reviews: la tabla guarda movie_id y movie_title, pero no existe una tabla local movies con una copia completa del catálogo.

4.3 Cómo obtiene acceso un desarrollador y qué credencial utiliza CineVerse

El acceso a la API se administra desde una cuenta de TMDB. En la configuración de API se obtienen las credenciales que luego se copian al entorno local. Una credencial es un dato que una aplicación presenta ante un servicio para demostrar que dispone de autorización. En CineVerse, VITE_TMDB_TOKEN contiene el API Read Access Token proporcionado por TMDB.

Un token de acceso es una cadena de caracteres que identifica la autorización utilizada para consultar una API. CineVerse no lo envía dentro de la URL: tmdbApi.ts lo coloca en la cabecera HTTP Authorization. Una cabecera HTTP es información adicional que acompaña a la solicitud, separada de la dirección y de los datos principales.

La palabra Bearer no es otro token. Es el nombre del esquema utilizado en la cabecera Authorization para indicar que el valor que sigue es un token de acceso. Por eso Authorization: Bearer <token> significa, en términos simples, “esta solicitud presenta este token como credencial”.

La otra variable importante es VITE_TMDB_BASE_URL. Contiene la URL base https://api.themoviedb.org/3, es decir, la parte inicial común de las direcciones de la API. Los services añaden después un endpoint concreto, por ejemplo /movie/top Rated, para formar la dirección completa de la consulta.

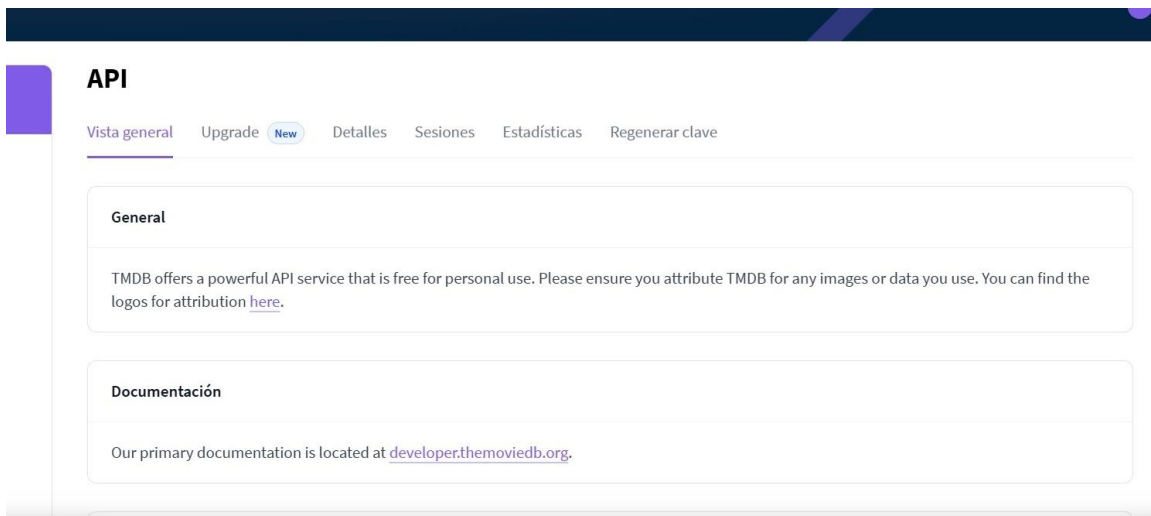


Figura. Sección de configuración de API de una cuenta de TMDB.

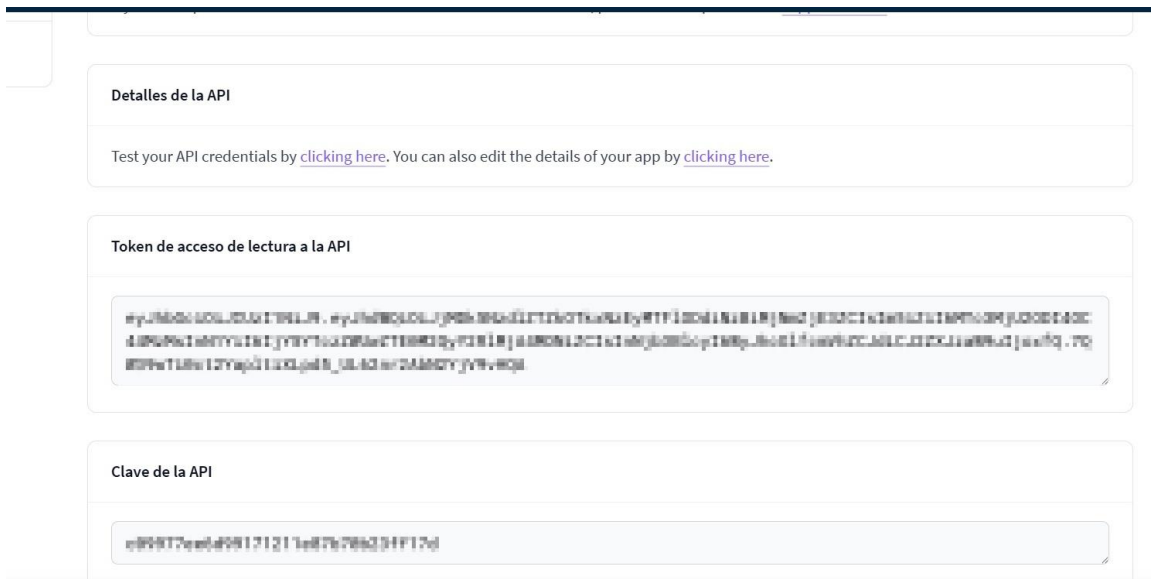


Figura. Credenciales API de TMDB con los valores sensibles censurados.

En la documentación de TMDB también pueden encontrarse ejemplos que utilizan una API Key v3 dentro del parámetro `api_key` de la URL. CineVerse no utiliza ese mecanismo en su código: emplea el API Read Access Token y lo presenta mediante `Authorization: Bearer <token>`.

Ubicación en el proyecto: `.env` y `.env.example` contienen `VITE_TMDB_BASE_URL` y `VITE_TMDB_TOKEN`; `resources/js/cineverse/core/api/tmdbApi.ts` lee esas variables y crea el cliente Axios compartido. El siguiente fragmento muestra cómo la URL base y el token llegan a esa configuración.

```
# .env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

// resources/js/cineverse/core/api/tmdbApi.ts
const api = axios.create({
  baseURL: import.meta.env.VITE_TMDB_BASE_URL,
  headers: {
    Authorization: `Bearer ${import.meta.env.VITE_TMDB_TOKEN}`,
    "Content-Type": "application/json",
  },
});
```

`axios.create()` todavía no busca ninguna película. Sólo crea un cliente HTTP reutilizable. `baseUrl` establece la parte inicial común de las direcciones y `headers` define cabeceras que acompañarán a las solicitudes, entre ellas `Authorization`. Los `services` deciden después qué endpoint y parámetros necesita cada operación.

Resumen: `VITE_TMDB_BASE_URL` indica dónde comienza la API de TMDB; `VITE_TMDB_TOKEN` contiene la credencial de acceso; `Bearer` indica cómo se presenta esa credencial dentro de la cabecera `Authorization`. Como las variables `VITE_*` quedan disponibles en el JavaScript del navegador, no deben tratarse como secretos privados del servidor.

4.4 Qué archivos hablan realmente con TMDB

```
View / Component
-> Composable
-> movies.service.ts / person.service.ts
-> tmdbApi.ts
-> Axios
-> HTTPS
-> TMDB API
-> JSON
-> Vue Query / Composable
-> interfaz
```

Ubicación en el proyecto: `resources/js/cineverse/core/api/tmdbApi.ts` · cliente HTTP compartido.
`resources/js/cineverse/features/movies/services/movies.service.ts` · endpoints de películas.
`resources/js/cineverse/features/person/services/person.service.ts` · detalle y filmografía de personas.

4.5 Ejemplo completo: el usuario abre "Mejor valoradas"

El ejemplo parte de una acción visible. El usuario entra al explorador de películas y elige Mejor valoradas. `useMoviesQuery` detecta `listType="top_rated"` y selecciona el camino de listas predefinidas; después `movies.service.ts` pide la página correspondiente a TMDB.

```
Usuario elige "Mejor valoradas"
-> MoviesView.vue mantiene listType="top_rated"
-> useMoviesQuery.ts ejecuta su función de consulta (queryFn)
-> getMovieList("top_rated", page, language)
-> movies.service.ts
-> api.get("/movie/top_rated", { params: { page, language } })
-> tmdbApi.ts añade baseUrl + Authorization
-> TMDB devuelve page + results + total_pages + total_results
-> Vue Query conserva la página recibida
-> MovieResults une los resultados cargados
-> MovieCard representa cada película
```

La paginación la define TMDB en su respuesta. CineVerse decide qué página solicitar, cuándo pedir la siguiente y cómo conservar juntas las páginas ya obtenidas.

4.5.1 Qué solicitud HTTP construye ese recorrido

Ejemplo didáctico equivalente a una llamada de `getMovieList()`; no es un bloque literal con `"top_rated"` fijo. La implementación real está en `resources/js/cineverse/features/movies/services/movies.service.ts` · Service del frontend · `getMovieList()`.

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  "/movie/top_rated",
  { params: { page: 1, language: "es" } }
);
```

En ese recorrido, `getMovieList()` utiliza la instancia compartida de `Axios`. `VITE_TMDB_BASE_URL` aporta la URL base `https://api.themoviedb.org/3`; el service aporta el endpoint `/movie/top_rated`; y `page=1` junto con `language=es` se agregan como parámetros. El resultado es una solicitud equivalente a GET

`https://api.themoviedb.org/3/movie/top_rated?page=1&language=es`. `tmdbApi.ts` añade además la cabecera `Authorization: Bearer <token>`.

4.5.2 Qué envía CineVerse y qué recibe de TMDB

`tmdbApi.ts` crea una única instancia de `Axios` para TMDB. Allí configura `VITE_TMDB_BASE_URL` como URL base y `Authorization: Bearer <token>` como cabecera común. Después, `movies.service.ts` sólo necesita indicar el endpoint y los parámetros propios de cada consulta.

En una búsqueda `GET` no se envía una película completa ni un objeto JSON con todos los filtros. Los valores como `query`, `page` y `language` se convierten en parámetros de la URL. La credencial viaja aparte, dentro de la cabecera `HTTP Authorization`.

```
GET https://api.themoviedb.org/3/search/movie?query=Batman&page=1&language=es
Authorization: Bearer <VITE_TMDB_TOKEN>
Content-Type: application/json
```

TMDB responde con JSON. Para los endpoints paginados, CineVerse espera una estructura `PaginatedResponse<Movie>`: `page` indica la página recibida, `results` contiene las películas y `total_pages/total_results` describen la paginación informada por TMDB.

El siguiente bloque es un ejemplo reducido de la forma esperada por el código; los valores son ilustrativos y no una captura de una respuesta concreta de TMDB:

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Película de ejemplo",
      "release_date": "2026-01-01",
      "poster_path": "/poster.jpg",
      "backdrop_path": "/backdrop.jpg",
      "vote_average": 7.8,
      "popularity": 120.5,
      "genre_ids": [28, 12],
      "overview": "Sinopsis de ejemplo"
    }
  ],
  "total_pages": 20,
  "total_results": 400
}
```

`Axios` coloca ese JSON en `data`. El tipo `PaginatedResponse<Movie>` ayuda a `TypeScript` a saber qué estructura espera el programa, pero no modifica ni valida por sí mismo la respuesta que llega por la red. El código utiliza después `data.results` para obtener las películas y sus campos.

4.5.3 Prueba manual de un endpoint en Postman

Una llamada de TMDB puede probarse fuera de CineVerse para observar la URL, los parámetros, el estado HTTP y el JSON devuelto. La captura siguiente usa una API Key v3 como parámetro de la URL únicamente para demostrar manualmente el endpoint en Postman. El código real de CineVerse utiliza el mecanismo explicado antes: API Read Access Token dentro de `Authorization: Bearer <token>`.

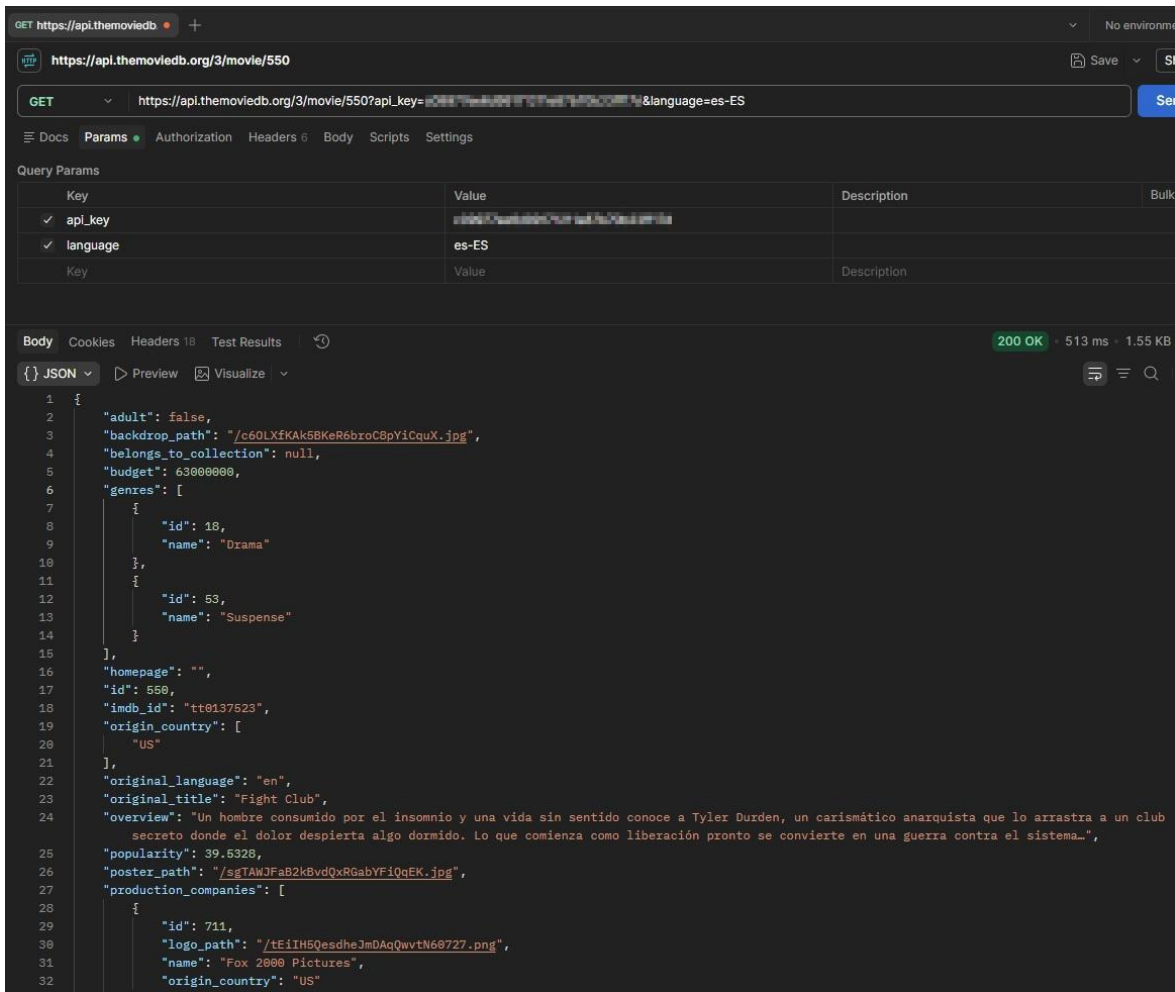


Figura. Prueba manual de GET /3/movie/550 en Postman y respuesta 200 OK.

4.6 Qué devuelve TMDb y cómo lo interpreta TypeScript

TMDb responde con JSON. Un endpoint de detalle devuelve un objeto; los endpoints paginados como `top_rated` o `search/movie` incluyen `page`, `results`, `total_pages` y `total_results`. CineVerse declara tipos TypeScript para expresar la estructura que espera.

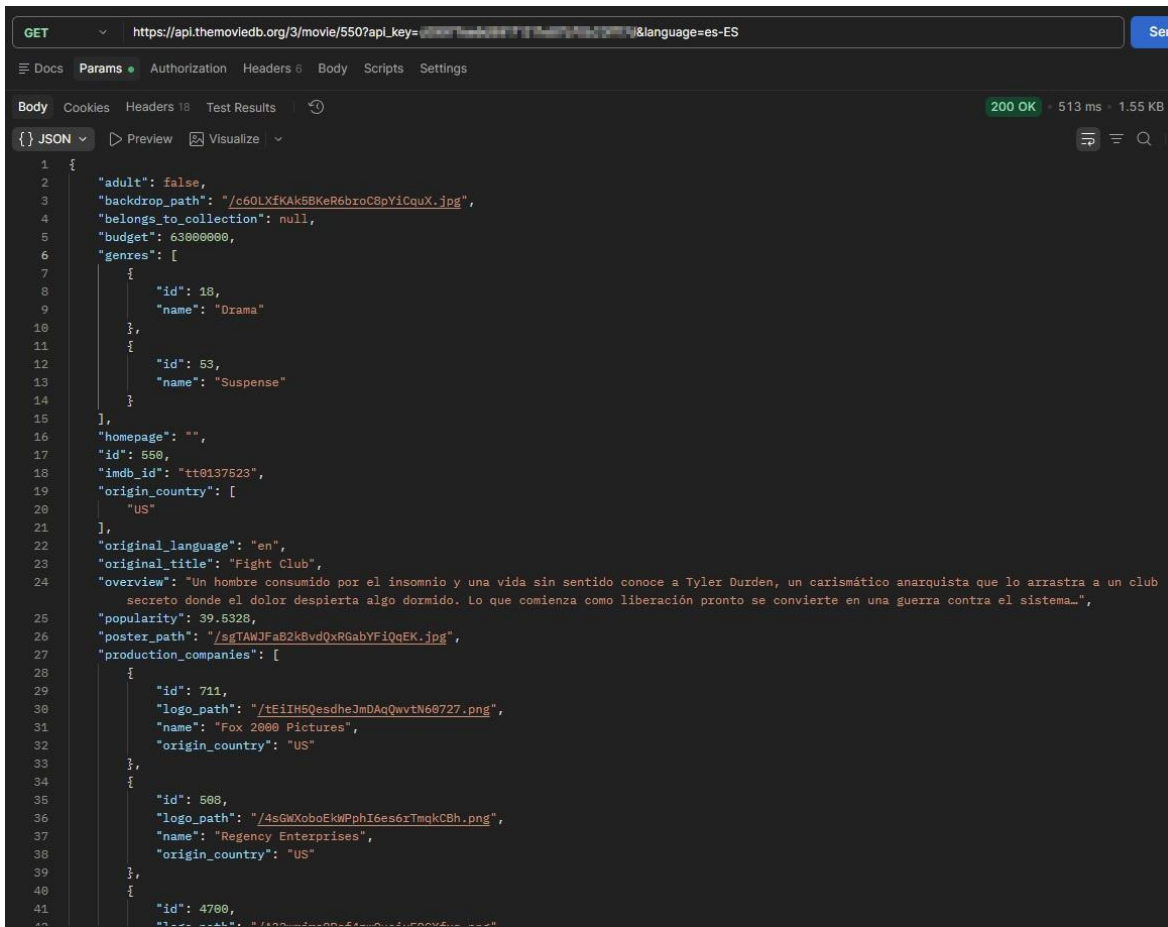


Figura. Fragmento de una respuesta JSON de TMDB con datos de una película.

Ubicación en el proyecto: resources/js/cineverse/shared/types/paginated-response.ts · interface genérica PaginatedResponse<T>. La utilizan movies.service.ts y otros services que tipan respuestas paginadas de TMDB.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}
```

PaginatedResponse<Movie> le dice a TypeScript que results es Movie[]. El tipo ayuda durante el desarrollo, pero no valida el JSON que ya llegó por la red; la aplicación sigue necesitando manejar estados de error y valores opcionales.

4.7 Endpoints utilizados por el proyecto

Endpoint TMDB	Uso
/trending/movie/day	Tendencias de la Home.
/movie/popular	Catálogo popular.
/movie/now_playing	En cartelera.
/movie/upcoming	Próximos estrenos.
/movie/top_rated	Mejor valoradas.
/search/movie	Búsqueda textual.

Endpoint TMDB	Uso
/discover/movie	Filtros avanzados sin texto.
/genre/movie/list	Catálogo de géneros disponibles.
/movie/{id}	Detalle de película.
/movie/{id}/credits	Reparto y equipo técnico (crew).
/movie/{id}/videos	Videos/trailers.
/movie/{id}/similar	Películas similares.
/person/{id}?append_to_response=movie_credits	Detalle de actor + filmografía en una solicitud.

4.8 Imágenes

TMDB no envía la imagen dentro del JSON. Devuelve una ruta relativa, por ejemplo /abc123.jpg. El frontend combina esa ruta con el servidor de imágenes image.tmdb.org y elige un tamaño adecuado. Ese servidor funciona como CDN, una red preparada para entregar archivos de imagen de forma eficiente desde distintas ubicaciones.

4.9 Qué ocurre cuando faltan datos en un idioma

TMDB no siempre tiene todos los campos traducidos. CineVerse aplica varios mecanismos de respaldo para intentar mostrar una imagen, créditos o videos aunque la primera respuesta llegue vacía.

- `getMovieDetail` vuelve a pedir el otro idioma sólo si poster y backdrop están ambos vacíos.
- `getMovieCredits` vuelve a pedir el otro idioma si el reparto (cast) y el equipo técnico (crew) vienen ambos vacíos.
- `getMovieVideos` vuelve a pedir /videos sin language cuando no hay resultados.
- `getSimilarMovies` utiliza actualmente `language="en-US"` fijo, independientemente de la UI.

4.10 Personas y append_to_response

TMDB utiliza el recurso `person` para actores y otras personas vinculadas al cine. CineVerse pide el detalle de una persona y su filmografía en una sola solicitud mediante `append_to_response=movie_credits`. Ese parámetro indica a TMDB que agregue la información de créditos de películas dentro de la misma respuesta principal.

```
GET /person/{id}
?language=es
&append_to_response=movie_credits

TMDB responde:
datos principales de la persona
+ movie_credits dentro del mismo JSON
```

`person.service.ts` devuelve esa estructura a `usePersonDetail`. `PersonHero` consume los datos principales, `PersonKnownFor` selecciona créditos destacados y `PersonFilmography` ordena la filmografía por año.

4.11 Cuenta de CineVerse y cuenta de TMDB no son la misma cosa

La cuenta de TMDB se utiliza únicamente para administrar el acceso del desarrollador a la API y obtener la credencial que CineVerse necesita. La cuenta que crea Carlos dentro de CineVerse se guarda en la tabla `users` de MySQL y se autentica con Laravel. Registrarse en CineVerse no crea una cuenta en TMDB y el login de CineVerse no envía email ni contraseña a TMDB.

```
Cuenta TMDB
-> pertenece al desarrollador / configuración de API
```

-> permite obtener credenciales

Cuenta CineVerse

-> users en MySQL
 -> Laravel Auth + sesión
 -> reseñas y perfil

4.12 Recursos de referencia utilizados al trabajar con TMDB

Para consultar los endpoints y parámetros de la API se utiliza la documentación oficial de TMDB para API v3. El sitio público themoviedb.org sirve para explorar el catálogo como usuario; api.themoviedb.org/3 es la URL base de las solicitudes de datos; image.tmdb.org entrega los archivos de imagen. CineVerse combina esos servicios, pero cada uno tiene una responsabilidad diferente.

5. Arquitectura general de CineVerse Laravel

La forma más sencilla de entender CineVerse Laravel es separar tres recorridos: cómo entra el navegador a la aplicación, cómo se obtienen datos cinematográficos y cómo se trabajan los datos que pertenecen a CineVerse. Esos recorridos pueden ocurrir dentro de una misma pantalla sin confundirse entre sí.

```

ENTRADA DE LA APLICACIÓN
Navegador /es/movies/550
-> routes/web.php
-> Inertia::render("CineVerse")
-> app.blade.php + resources/js/app.ts
-> Pages/CineVerse.vue
-> cineverse/App.vue
-> Vue Router
-> MovieDetailView

CATÁLOGO CINEMATOGRAFICO
MovieDetailView
-> useMovieDetail
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> Vue Query
-> componentes Vue

DATOS PROPIOS
MovieReviews / LoginView / ProfileView
-> service del frontend
-> webApi.ts
-> routes/auth.php o routes/api.php
-> Controller / Request
-> Eloquent
-> MySQL
  
```

5.1 Tres recorridos que conviven en la misma aplicación

La arquitectura se entiende mejor separando tres recorridos. El primero entrega la aplicación al navegador; el segundo obtiene catálogo cinematográfico directamente desde TMDB; el tercero trabaja con identidad y datos propios mediante Laravel y MySQL.

```

ENTRADA
Navegador -> Laravel -> Inertia -> Vue -> Vue Router

CATÁLOGO
View -> Composable -> Service -> tmdbApi -> TMDB -> Vue Query -> UI

DATOS PROPIOS
View -> Service -> webApi -> Laravel -> Eloquent -> MySQL -> JSON -> UI
  
```

Una sola pantalla puede activar más de un recorrido. MovieDetailView obtiene título, póster, reparto y trailer desde TMDB; cuando Carlos abre la pestaña Reseñas, MovieReviews consulta Laravel y MySQL. Vue combina ambos orígenes en la misma interfaz.

5.2 Laravel no renderiza cada pantalla de CineVerse

En una aplicación Inertia tradicional podría existir una página Inertia diferente para cada ruta del sitio. CineVerse no adoptó ese patrón. `routes/web.php` devuelve `Inertia::render("CineVerse")` para la ruta localizada y para el `catch-all`. El componente `Pages/CineVerse.vue` es deliberadamente mínimo y monta `CineVerseApp`. Por eso Laravel no necesita una página Inertia `MovieDetail`, `PersonDetail` o `Favorites`.

La consecuencia es que la URL cumple dos funciones en momentos diferentes. Ante una carga o recarga completa, Laravel debe reconocer `/es/...` y entregar la app. Una vez que Vue está montado, Vue Router interpreta esa misma URL para elegir la vista interna. La sección 3 desarrolla esta frontera con detalle, utilizando el código fuente real del proyecto.

5.3 Una misma pantalla puede usar TMDB y Laravel

El detalle de película es el ejemplo más claro. La sinopsis, el póster, el reparto y los trailers se obtienen desde TMDB. La pestaña Reseñas, en cambio, llama a `/api/movies/{movieId}/reviews` y obtiene promedio, cantidad, reseña propia y comentarios de la comunidad desde MySQL. Vue presenta ambos orígenes como una experiencia única sin que sea necesario copiar la película completa a la base local.

5.4 Dónde vive el estado

No todo estado se guarda en el mismo lugar. La sesión verdadera pertenece a Laravel y se vincula al navegador mediante una cookie. Pinia mantiene una representación reactiva del usuario y el token CSRF para que la interfaz pueda reaccionar. Los favoritos se persisten localmente en el navegador con `pinia-plugin-persistedstate`. Los resultados remotos de TMDB se cachean mediante `Vue Query`. Esta separación evita utilizar MySQL como almacén universal para datos que no lo necesitan.

5.5 Relación entre la arquitectura general y el puente Inertia

La arquitectura general muestra dónde se ubica cada responsabilidad, pero el mecanismo exacto que conecta Laravel con Vue ya no se repite aquí. La sección 3 desarrolla ese puente archivo por archivo y explica qué ocurre tanto en una carga directa como durante la navegación interna de la SPA.

6. Estructura del proyecto

CineVerse Laravel es un único proyecto. No hay una carpeta frontend y otra backend ejecutadas como aplicaciones independientes. Laravel ocupa la raíz y el frontend CineVerse se integra dentro de `resources/js`. Entender esta diferencia evita repetir configuraciones antiguas de Apache, proxy de Vite o dos servidores HTTP propios del proyecto PHP puro.

```
cineverse-laravel/
├── app/
│   ├── Http/Controllers/
│   ├── Http/Middleware/
│   ├── Http/Requests/
│   ├── Models/
│   ├── Notifications/
│   ├── Logging/
│   └── Support/
├── bootstrap/
├── config/
├── database/
│   ├── migrations/
│   └── seeders/
├── public/
├── resources/
│   ├── js/app.ts
│   ├── js/Pages/CineVerse.vue
│   ├── js/cineverse/
│   └── views/
├── routes/
└── web.php
```

```

├── auth.php
├── api.php
├── tests/
├── composer.json
├── package.json
└── vite.config.js

```

6.1 app/: servidor Laravel

app/Http/Controllers/Auth contiene los flujos de cuenta derivados de Breeze y adaptados a CineVerse. app/Http/Controllers/Api contiene endpoints JSON de perfil y reseñas. app/Http/Requests aloja validaciones reutilizables como LoginRequest. app/Models contiene User y Review. Notifications define los emails propios, mientras Support/CineVerseLog y Logging/CineVerseLogFormatter mantienen el formato de diagnóstico del proyecto.

6.2 resources/js/: entrada y aplicación Vue

resources/js/app.ts es el entrypoint de Vite e Inertia. resources/js/Pages/CineVerse.vue es la página que Inertia puede resolver por nombre. Debajo, resources/js/cineverse conserva la organización por core, features, layouts, shared, views y assets. El frontend no fue aplanado dentro de Pages porque el objetivo era mantener la arquitectura modular original.

6.3 database/: estructura y escenario reproducible

database/migrations describe el esquema. Las migraciones incluyen tablas estándar de Laravel, personal_access_tokens, la transición de users desde columnas legacy a la forma compatible con Auth, la tabla reviews y una migración que elimina login_attempts porque el rate limit ya no se apoya en esa tabla. database/seeders carga 5.000 usuarios y 24.322 reseñas desde NDJSON.

6.4 routes/: tres fronteras distintas

Archivo	Qué recibe	Qué devuelve
routes/web.php	Navegación del navegador	Redirects o página Inertia CineVerse.
routes/auth.php	Acciones de cuenta	JSON/redirects de autenticación, verificación y password.
routes/api.php	Datos propios usados por Vue	JSON de perfil y reseñas.

6.5 bootstrap/: arranque del framework

bootstrap/app.php construye la instancia Application de Laravel 12 y registra routing, middleware y manejo de excepciones. bootstrap/providers.php enumera los providers propios que se cargan. Esta carpeta no contiene pantallas de CineVerse; contiene instrucciones de arranque del framework.

6.6 config/: configuración traducida desde .env

Los archivos de config convierten variables de entorno en estructuras PHP que el resto del framework consulta con config(...). database.php define conexiones; session.php define dónde y cuánto vive una sesión; mail.php define el mailer; logging.php define canales de logs; cache.php define el store utilizado por cache y RateLimiter.

6.7 public/: única raíz pública del servidor

public/index.php es el front controller de Laravel. El servidor web debe exponer public/ y no la raíz completa del repositorio. Los assets compilados terminan bajo public/build; favicon.ico y robots.txt también viven aquí. .env, app/, config/ y storage/ quedan fuera de la raíz pública.

6.8 resources/: Vue, CSS y vistas Blade

resources/js/app.ts inicia Inertia y Vue; resources/js/Pages/CineVerse.vue es el adaptador que Inertia resuelve; resources/js/cineverse contiene la aplicación Vue real. resources/css/app.css es una entrada de Vite. resources/views contiene app.blade.php, emails y la vista de error de base de datos.

6.9 storage/: sesiones, caché, logs y archivos de ejecución

storage no contiene código fuente de negocio. Laravel la utiliza para información generada mientras la aplicación está funcionando. Con SESSION_DRIVER=file las sesiones van a storage/framework/sessions; el caché basado en archivos usa storage/framework/cache; los logs están bajo storage/logs si CINEVERSE_LOG_PATH no los desvía a otra carpeta.

storage contiene datos de ejecución, no archivos públicos de la aplicación. No es equivalente a public: los logs y las sesiones no deben servirse como archivos descargables desde el navegador.

6.10 tests/: comprobaciones automáticas

tests contiene pruebas PHPUnit. Feature/Auth proviene en gran parte del scaffolding de Breeze y todavía conserva expectativas del proyecto original que ya no coinciden con todas las adaptaciones de CineVerse. La sección 34 separa lo que los tests intentan comprobar de lo que realmente representa el código actual.

6.11 Ejemplo: qué carpetas atraviesa una reseña

```
resources/js/cineverse/.../MovieReviews.vue
-> resources/js/cineverse/.../reviews.service.ts
-> routes/api.php
-> app/Http/Controllers/Api/ReviewController.php
-> app/Models/Review.php
-> MySQL

database/migrations define la tabla
storage/logs sólo participa si ocurre un fallo técnico registrable
```

7. Archivos de configuración y variables de entorno

Laravel gestiona la configuración sensible al entorno en .env y la traduce a valores de config/*.php. El repositorio incluye .env.example como plantilla, pero .gitignore excluye .env porque puede contener contraseñas de MySQL, credenciales SMTP y el token de TMDB.

7.1 Flujo .env → config → aplicación

```
.env
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
|
v
config/database.php
|
v
DB / Eloquent / migraciones
```

El mismo principio se aplica a sesiones, correo y logs. El código de CineVerse accede a estos valores mediante la configuración de Laravel. Laravel carga las variables de .env y los archivos config/*.php las exponen al resto de la aplicación mediante config(...).

7.2 Ejemplo de configuración mínima para el entorno local

Referencia: `.env.example` está en la raíz del proyecto y muestra qué variables puede configurar una instalación. El bloque siguiente no es una copia literal de ese archivo: es un ejemplo pedagógico adaptado a CineVerse, con marcadores y con algunos valores del entorno utilizado para estas capturas. El `.env` real es local y no se publica.

```
APP_NAME=CineVerse
APP_ENV=local
APP_KEY=base64:...
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
DB_USERNAME=<usuario-local>
DB_PASSWORD=<contraseña-local>

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<token-TMDB>"
```

El puerto 3307 corresponde al entorno local utilizado aquí y no es una regla universal de Laravel. Si MySQL está en 3306 u otro puerto, `.env` debe reflejar la instalación real. Lo mismo ocurre con el usuario, la contraseña y la configuración SMTP. En otras palabras, `.env.example` sirve como plantilla; cada equipo completa su propio `.env` con los valores que realmente utiliza.

7.3 APP_KEY

`APP_KEY` es una clave criptográfica de la aplicación Laravel. No debe copiarse de otro equipo ni inventarse a mano. `php artisan key:generate` crea un valor seguro y lo escribe en `.env`. Laravel la utiliza en operaciones criptográficas del framework; por eso una instalación recién clonada necesita generarla antes de funcionar correctamente.

7.4 Variables VITE_*

Vite hace disponibles en el código del navegador las variables de entorno cuyo nombre comienza con `VITE_`. CineVerse utiliza dos de ellas para configurar el acceso a TMDB: `VITE_TMDB_BASE_URL` y `VITE_TMDB_TOKEN`. La primera contiene la URL base de la API; la segunda contiene el API Read Access Token utilizado como credencial.

```
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"
```

La URL base es la parte común que aparece al comienzo de las consultas. Por ejemplo, `movies.service.ts` puede pedir el endpoint `/movie/top_rated` y Axios lo combina con `https://api.themoviedb.org/3` para obtener la dirección completa `https://api.themoviedb.org/3/movie/top_rated`.

El token cumple otra función: es la credencial que CineVerse presenta ante TMDB. `tmdbApi.ts` lo envía en la cabecera HTTP Authorization con la forma `Bearer <token>`. Bearer no es una segunda credencial; sólo indica el esquema utilizado para presentar el token dentro de esa cabecera.

Como las variables `VITE_*` quedan incorporadas al JavaScript que Vite entrega al navegador, sus valores pueden ser inspeccionados desde el cliente. Por eso no deben utilizarse para contraseñas de MySQL, claves privadas del servidor u otros secretos que deban permanecer exclusivamente en el backend.

Cuando se modifica una variable `VITE_*` mientras `npm run dev` está ejecutándose, Vite debe reiniciarse para volver a leerla. `APP_URL` cumple una función distinta: indica la URL principal de la aplicación Laravel y no es una credencial de TMDB.

7.5 Configuración de sesiones

config/session.php usa env("SESSION_DRIVER", "database"), y .env.example también propone SESSION_DRIVER=database. Con esa configuración, Laravel utiliza la tabla sessions. Si SESSION_DRIVER se configura como file, las sesiones se guardan como archivos en storage/framework/sessions. SESSION_LIFETIME vale 120 minutos en .env.example. En ambos casos la sesión de Laravel es la fuente real de autenticación; Pinia sólo refleja su estado en Vue.

7.6 Configuración de correo

El ejemplo usa SMTP y permite configurar host, puerto, usuario, contraseña, cifrado y remitente. Registro, reenvío de verificación y recuperación de contraseña dependen de esta configuración. Los controladores capturan fallos de entrega en puntos sensibles para registrar el error y devolver mensajes controlados.

7.7 Configuración de base de datos

config/database.php lee DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME y DB_PASSWORD. Eloquent, Query Builder, migraciones y seeders utilizan esa configuración. Cambiar DB_PORT en .env no requiere modificar Review.php ni los Controllers: la conexión se gestiona desde la configuración.

Esquema explicativo del flujo de configuración; no corresponde a un bloque literal de código. Las variables se definen en .env y Laravel las incorpora mediante config/database.php antes de que Eloquent, las migraciones y los seeders utilicen la conexión.

```
.env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cineverse_laravel

config/database.php
-> conexión mysql
-> Eloquent / migrate / db:seed
```

7.8 Configuración del caché y relación con RateLimiter

config/cache.php usa CACHE_STORE=database por defecto en .env.example. El store database utiliza las tablas cache y cache_locks creadas por migraciones. RateLimiter se apoya en el sistema de caché de Laravel; por eso los intentos fallidos de login ya no necesitan la tabla login_attempts del backend anterior.

7.9 Configuración del log propio de CineVerse

CINEVERSE_LOG_PATH se interpreta dentro de config/logging.php. Puede ser una ruta absoluta o una ruta relativa a la raíz del proyecto. El canal cineverse_unexpected utiliza esa carpeta y el nombre base cineverse-backend.log; el driver daily agrega la fecha al archivo final.

```
CINEVERSE_LOG_PATH=storage/logs
-> config/logging.php
-> storage/logs/cineverse-backend.log
-> driver daily
-> cineverse-backend-AAAA-MM-DD.log
```

7.10 Ejemplo guiado: Carlos cambia de computadora y MySQL usa otro puerto

Supongamos que el proyecto fue preparado originalmente con MySQL en 3307 y Carlos instala MySQL en 3306. No modifica los Controllers ni los Models. Actualiza DB_PORT en .env y Laravel vuelve a construir la conexión a partir de config/database.php.

```
Carlos modifica .env
DB_PORT=3306
-> Laravel carga la variable
-> config/database.php arma la conexión
-> Eloquent y php artisan migrate usan el nuevo puerto
```

8. Base de datos MySQL, migraciones y Eloquent

MySQL guarda aquello que pertenece a CineVerse y la infraestructura que Laravel necesita. No almacena una tabla movies ni people porque esos recursos siguen viniendo de TMDB. Esta decisión se refleja en reviews: cada fila conserva movie_id y movie_title para identificar la película reseñada, pero no existe una clave foránea hacia una tabla de películas local.

8.1 Tablas principales

Tabla	Papel
users	Cuenta autenticable: username, email, verificación, password, remember token y timestamps.
reviews	Reseñas de usuarios sobre películas de TMDB.
sessions	Sesiones Laravel cuando SESSION_DRIVER=database.
password_reset_tokens	Tokens del Password Broker.
cache / cache_locks	Cache de Laravel; también puede respaldar RateLimiter según configuración.
jobs / job_batches / failed_jobs	Infraestructura de colas incluida por las migraciones estándar.
personal_access_tokens	Tabla de Sanctum presente aunque el flujo principal usa sesión web.
migrations	Registro de migraciones ya aplicadas.

8.2 users y la transición desde la estructura anterior

La primera migración de esta versión crea users directamente con username, email, email_verified_at, password, remember_token y timestamps. La migración posterior standardize_users_table_for_laravel_auth conserva compatibilidad con una base legacy: agrega las columnas estándar necesarias cuando faltan y, si existen las columnas antiguas, copia name a username y password_hash a password. La migración siguiente elimina password_hash, is_verified y tokens legacy una vez trasladada la información a las convenciones de Laravel.

Este diseño explica por qué el modelo User utiliza el cast password => hashed y email_verified_at => datetime. Laravel Auth y MustVerifyEmail pueden trabajar con columnas estándar en lugar de mantener lógica paralela para is_verified y verification_token.

8.3 reviews y su relación con users

```
reviews
  id          INT UNSIGNED PK
  user_id     FK -> users.id (cascade on delete)
  movie_id    INT UNSIGNED + index
  movie_title VARCHAR
  rating      TINYINT
  comment     TEXT
  created_at
  updated_at
  UNIQUE(user_id, movie_id)
```

La clave foránea garantiza que una reseña siempre pertenezca a un usuario existente. cascadeOnDelete hace que, si se elimina definitivamente una cuenta, sus reseñas también desaparezcan. El índice por movie_id acelera

el listado de reseñas de una película. La restricción `unique_user_movie` impide duplicar una reseña para la misma combinación usuario-película incluso si alguien intentara saltarse la interfaz.

8.4 Migración versus seeder

Una migración define estructura; un seeder carga filas. `php artisan migrate` puede ejecutarse sobre una base vacía y crea las tablas sin cargar las 5.000 cuentas de prueba. `php artisan db:seed` presupone que la estructura ya existe. Para reconstruir deliberadamente un entorno local de pruebas, `php artisan migrate:fresh --seed` elimina las tablas actuales, vuelve a ejecutar las migraciones y después carga los seeders.

8.5 Eloquent

Eloquent es el ORM de Laravel. En lugar de escribir manualmente `SELECT * FROM reviews WHERE user_id = ?`, `ProfileController` puede construir `Review::query()->where("user_id", $userId)->count()`. El modelo `Review` define `fillable` y `casts` y expone `user()` como `belongsTo(User::class)`. Eloquent no reemplaza a MySQL: genera las consultas y transforma resultados en objetos que los controladores pueden utilizar.

8.6 login_attempts ya no existe

La migración `2026_07_11_121000_drop_login_attempts_table.php` elimina la tabla `legacy`. El comentario de `down()` deja explícita la decisión: Laravel usa `RateLimiter/cache` para los intentos de login y no recrea la tabla antigua. Esta es una diferencia arquitectónica real respecto del backend PHP puro.

8.7 Cómo Laravel sabe qué migraciones faltan

La tabla `migrations` registra qué archivos ya fueron aplicados y en qué batch. Cuando se ejecuta `php artisan migrate`, Laravel ordena las migraciones por su nombre/timestamp, compara esa lista con la tabla `migrations` y ejecuta `up()` únicamente en las pendientes. Después registra cada migración aplicada.

```
php artisan migrate
-> leer database/migrations
-> consultar tabla migrations
-> detectar archivos pendientes
-> ejecutar up()
-> MySQL cambia el esquema
-> registrar migración aplicada
```

`down()` describe cómo revertir una migración cuando el flujo de rollback lo requiere. No todas las decisiones son simétricas: la migración que elimina `login_attempts` deja `down()` vacío de forma deliberada porque el sistema actual usa `RateLimiter/cache` y no desea reconstruir esa tabla `legacy`.

8.8 users: adaptación desde el esquema anterior a las convenciones de Laravel

El proyecto contiene migraciones capaces de trabajar con una instalación que hubiera conservado columnas del backend anterior. `standardize_users_table_for_laravel_auth` agrega `username`, `password`, `email_verified_at`, `remember_token` y `timestamps` cuando faltan y copia `name/password_hash` si existen. La migración siguiente transfiere el estado de verificación y elimina columnas legacy como `password_hash`, `is_verified` y `tokens` manuales.

El resultado final permite que `User` extienda `Authenticatable`, implemente `MustVerifyEmail` y use las convenciones que `Auth`, `Password Broker` y `Notifications` esperan.

8.9 Review como modelo Eloquent

`Review` define `fillable` para los campos que pueden asignarse de forma masiva, `casts` para convertir `ids/rating` a enteros y `timestamps` a fechas, y `user()` como relación `belongsTo`. Esa relación expresa que muchas reseñas pueden pertenecer a un usuario.

Ubicación en el proyecto: app/Models/Review.php · clase Eloquent App\Models\Review. La utilizan ReviewController y ProfileController para consultar y persistir reseñas.

```
class Review extends Model {
    protected $fillable = [
        "user_id", "movie_id", "movie_title", "rating", "comment"
    ];

    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

8.10 Ejemplo: Review::create termina en MySQL

Cuando ReviewController llama Review::create(), Eloquent toma los campos permitidos por fillable, prepara el INSERT y utiliza la conexión configurada para MySQL. La restricción UNIQUE(user_id, movie_id) sigue existiendo en la base aunque el Controller ya haya comprobado previamente un duplicado.

```
ReviewController::store()
-> $request->validate(...)
-> comprobar exists()
-> Review::create([...])
-> Eloquent construye INSERT
-> MySQL valida FK + UNIQUE
-> fila persistida
```

9. Datos de prueba y seeders

En Laravel, un seeder es una clase que carga registros automáticamente en tablas que ya existen. CineVerse utiliza seeders para preparar un conjunto amplio de datos de prueba y así recorrer reseñas y perfiles sin crear miles de registros manualmente. En lugar de codificar cada registro dentro de una clase PHP, los seeders leen archivos NDJSON ubicados en database/seeders/data; cada línea contiene un objeto JSON independiente.

9.1 Qué hace un seeder y qué relación tiene con las migraciones

Una migración y un seeder resuelven problemas distintos. La migración define o modifica la estructura de la base de datos: crea tablas, columnas, índices y restricciones. El seeder trabaja después sobre esa estructura ya existente y agrega registros preparados.

En CineVerse, DatabaseSeeder actúa como punto de entrada y coordina los seeders específicos del proyecto. Primero se cargan los usuarios y después las reseñas, porque reviews.user_id depende de users.id.

9.2 DatabaseSeeder

```
DatabaseSeeder
-> UserSeeder
-> ReviewSeeder
```

El orden es importante porque reviews.user_id tiene una clave foránea. ReviewSeeder comprueba explícitamente que existan 5.000 usuarios antes de insertar reseñas.

9.3 Totales esperados

Dataset	Cantidad esperada	Tamaño de batch
users.ndjson	5.000	500
reviews.ndjson	24.322	250

Cada seeder cuenta las líneas realmente procesadas. Si el total no coincide con EXPECTED_COUNT lanza RuntimeException y no presenta el dataset incompleto como si fuera válido.

9.4 Condición de tablas vacías

UserSeeder exige users vacía y ReviewSeeder exige reviews vacía. Esto evita insertar una segunda copia del mismo escenario sobre datos existentes. Cuando se necesita reconstruir un entorno de pruebas, el mensaje de error orienta a `php artisan migrate:fresh --seed`.

9.5 Por qué NDJSON

NDJSON permite leer el archivo línea por línea con `fgets` y procesar lotes sin cargar el dataset completo en memoria. UserSeeder acumula 500 filas y ReviewSeeder 250 antes de cada insert. Para decenas de miles de reseñas, este enfoque es más predecible que decodificar un gran array JSON de una sola vez.

9.6 Datos reproducibles para pruebas, no migración de producción

Los seeders preparan un entorno de prueba local. No describen una operación que deba repetirse sobre datos reales. `migrate:fresh --seed` destruye la estructura existente y por eso sólo corresponde cuando esa destrucción es deliberada y los datos no necesitan conservarse.

9.7 Ejemplo guiado: Carlos reconstruye un entorno local de prueba

Carlos quiere volver a un escenario limpio sin conservar datos anteriores. Ejecuta `migrate:fresh --seed`. Laravel elimina las tablas de esa base, vuelve a ejecutar todas las migraciones y después llama DatabaseSeeder. Al terminar, el esquema y el dataset quedan en el estado esperado por la documentación.

migrate:fresh --seed es destructivo. En este documento se describe para un entorno de prueba cuya información puede borrarse deliberadamente, no como rutina sobre datos reales.

10. Cómo recorre CineVerse una acción del usuario

Seguir acciones completas evita estudiar archivos aislados sin entender para qué existen. Los tres ejemplos siguientes muestran los recorridos más representativos: una consulta sólo a TMDB, un login que usa la sesión de Laravel y una reseña que combina la película de TMDB con datos propios en MySQL.

10.1 Abrir una película

En este recorrido aparece `Promise.all()`. Una Promise representa una operación asíncrona cuyo resultado llegará más adelante; `Promise.all()` permite iniciar varias de esas operaciones y continuar cuando todas hayan terminado. CineVerse lo utiliza para esperar créditos, videos y películas similares en conjunto.

```
Usuario pulsa Fight Club
-> router.push({ name: "MovieDetail", id: 550 })
-> MovieDetailView.vue
-> useMovieDetail(550, language)
-> Promise.all(credits, videos, similar)
-> tmdbApi.ts
-> TMDB
-> Vue Query cache
-> MovieHero / MovieTabs / etc.
```

Laravel no consulta la base para producir sinopsis o póster. La navegación ya ocurre dentro de Vue Router y los services hablan directamente con TMDB. Si sólo cambia el idioma, la query key incorpora `language` y Vue Query puede gestionar un resultado localizado diferente.

10.2 Iniciar sesión

```
LoginView
-> loginUser({ email, password })
-> webApi.post("/login")
-> routes/auth.php
-> AuthenticatedSessionController::store
```

```

-> LoginRequest::authenticate
-> RateLimiter + User + Hash::check
-> Auth::login
-> session()->regenerate()
-> 204
-> GET /user
-> user + csrf_token
-> useAuthStore
-> header / guards / perfil reaccionan

```

Pinia no autentica por sí mismo. La sesión verdadera existe en Laravel. El store conserva una copia reactiva del usuario para que la interfaz sepa qué mostrar. Si la página se recarga, `initialize()` vuelve a consultar `/user` y reconstruye esa representación a partir de la cookie de sesión.

10.3 Crear una reseña

En `createReview(payload, csrfToken)`, `payload` es simplemente el objeto que agrupa los datos que se enviarán a Laravel: `movie_id`, `movie_title`, `rating` y `comment`. El token CSRF viaja aparte, en una cabecera HTTP de protección.

```

MovieReviews
-> createReview(payload, csrfToken)
-> POST /api/reviews + X-CSRF-Token
-> middleware web + auth
-> ReviewController::store
-> validate movie_id/title/rating/comment
-> comprobar duplicado user_id + movie_id
-> Review::create()
-> MySQL
-> REVIEW_CREATED
-> MovieReviews vuelve a cargar estadísticas/listado

```

La interfaz limita el comentario y muestra el formulario apropiado, pero el backend vuelve a validar. La restricción `unique_user_movie` en MySQL añade una tercera capa para la regla “una reseña por usuario y película”. Después de guardar, el frontend no inventa localmente el promedio: vuelve a pedir los datos y muestra el estado confirmado por el servidor.

11. TypeScript en CineVerse

TypeScript es una extensión de JavaScript que permite indicar qué tipo de dato se espera en cada parte del código. Por ejemplo, CineVerse puede declarar que el `id` de una película es un número, que el título es texto y que `poster_path` puede ser una ruta de imagen o `null` cuando TMDB no proporciona un póster. Estas comprobaciones ayudan a detectar errores mientras se desarrolla, antes de abrir la aplicación en el navegador.

En CineVerse los tipos también describen respuestas HTTP, usuarios, reseñas, filtros, propiedades de componentes y valores devueltos por composables. El proyecto activa `strict: true`, por lo que TypeScript realiza comprobaciones más exigentes que en una configuración básica.

11.1 Configuración estricta

Ubicación en el proyecto: `tsconfig.json` · configuración TypeScript del proyecto. La lee el compilador/vue-tsc durante desarrollo y build.

```

{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  }
}

```

Con estas opciones el compilador puede avisar, por ejemplo, que se intenta leer una propiedad que no existe, que un valor podría ser null, que se pasó un string donde se esperaba un número o que quedó una variable declarada pero sin uso.

11.2 Tipos de dominio

Ubicación en el proyecto: `resources/js/cineverse/features/movies/types/movie.ts` · `interface Movie`. La utilizan `movies.service.ts`, `composables` y `componentes` que trabajan con películas de TMDB.

```
export interface Movie {
  id: number;
  title: string;
  release_date: string;
  poster_path: string | null;
  backdrop_path: string | null;
  vote_average: number;
  popularity: number;
  genre_ids?: number[];
  overview: string;
}
```

`interface` es una de las formas que ofrece TypeScript para describir la estructura de un objeto. En `Movie`, `poster_path: string | null` indica que el valor puede ser una ruta de texto o puede no existir. La interfaz debe contemplar ambos casos en lugar de asumir que todas las películas tienen póster.

11.3 Genéricos: `PaginatedResponse<T>`

Ubicaciones en el proyecto: `resources/js/cineverse/shared/types/paginated-response.ts` define `PaginatedResponse<T>`; `resources/js/cineverse/features/movies/services/movies.service.ts` lo usa para tipar respuestas como `/search/movie`.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}

const { data } = await api.get<PaginatedResponse<Movie>>({
  "/search/movie",
  { params: { query, page, language } }
});
```

`PaginatedResponse<T>` es un tipo genérico: `T` representa el tipo concreto de los elementos que contendrá `results`. Cuando se usa `PaginatedResponse<Movie>`, TypeScript sabe que `results` es `Movie[]` y que `page`, `total_pages` y `total_results` son números. La misma estructura de paginación puede reutilizarse con otros tipos sin copiarla.

11.4 Reutilización de tipos en los filtros

Ubicaciones en el proyecto: `resources/js/cineverse/features/movies/types/base-filters.ts` · `BaseFilters`; `discover-filters.ts` · `DiscoverFilters`; `search-with-text-filters.ts` · `SearchWithTextAndFilters`. Las utilizan `movies.service.ts` y `useMoviesQuery.ts`.

```
export interface BaseFilters {
  genres?: number[];
  yearFrom?: number;
  yearTo?: number;
  minRating?: number;
  maxRating?: number;
  sortBy?: string;
}
```

```
export interface DiscoverFilters extends BaseFilters {}

export interface SearchWithTextAndFilters extends BaseFilters {
  query: string;
}
```

BaseFilters reúne las propiedades compartidas por varios tipos de búsqueda. DiscoverFilters reutiliza esas propiedades y SearchWithTextAndFilters añade query. Así género, años, puntuación y orden se definen una sola vez en lugar de duplicarse en cada tipo.

11.5 Obtener un tipo a partir de un composable

Ubicación en el proyecto: `resources/js/cineverse/features/movies/composables/useMoviesQuery.ts` · Composable · tipo `MovieFilters` y función `useMoviesQuery()`. El tipo se deriva de `useMoviesFilters.ts`; el bloque mostrado es un fragmento abreviado.

```
export type MovieFilters = ReturnType<typeof useMoviesFilters>;

export function useMoviesQuery(
  filters: MovieFilters,
  language: Ref<string>
) { ... }
```

`ReturnType` es una utilidad de TypeScript que obtiene automáticamente el tipo que devuelve una función. `MovieFilters` se obtiene a partir de `useMoviesFilters`; si ese composable cambia lo que devuelve, el tipo se actualiza junto con él y reduce la posibilidad de que ambas definiciones se separen.

En el mismo ejemplo aparece `Ref<string>`. `Ref` es un contenedor reactivo de Vue: guarda un valor que puede cambiar y permite que Vue detecte ese cambio. `Ref<string>` indica que ese valor reactivo contiene texto; en `useMoviesQuery` representa el idioma activo.

11.6 TypeScript no reemplaza la validación del backend

Que TypeScript indique `rating: number` protege al propio código durante el desarrollo, pero no convierte al navegador en una barrera de seguridad. Alguien puede enviar manualmente `rating = 999` sin utilizar la interfaz. Por eso `ReviewController` vuelve a validar `rating` con `integer|min:1|max:5` y `comment` con `max:1000`. La migración utiliza `tinyInteger` para `rating`, pero no declara un `CHECK SQL`; la regla 1–5 queda impuesta actualmente por Laravel y por la interfaz.

Cada capa cumple una función distinta

TypeScript ayuda a detectar errores mientras se programa. Laravel vuelve a validar los datos que realmente llegan por HTTP mediante `Request`, `Form Requests` o reglas del controlador. Eloquent construye las consultas sobre MySQL y las restricciones `UNIQUE` y `FOREIGN KEY` mantienen reglas estructurales de la base. En reviews, el rango 1–5 se valida en Laravel; la migración utiliza `tinyInteger` pero no declara un `CHECK SQL`.

11.7 Zonas con tipado menos estricto

El proyecto utiliza una configuración estricta, pero todavía existen zonas menos tipadas. Hay `Record<string, any>`, errores capturados como `any` y algunos arrays de créditos con `any[]`. `any` indica que TypeScript deja de comprobar con precisión ese valor. También debe recordarse que un tipo de `Axios` describe lo que el código espera recibir, pero no valida durante la ejecución que TMDb o el backend hayan enviado exactamente esa estructura.

11.8 Props y eventos tipados entre componentes

TypeScript también protege la comunicación entre componentes Vue. Las props son valores que un componente padre entrega a un hijo. Los eventos permiten el camino inverso: el hijo avisa que ocurrió algo y el padre decide qué hacer.

Ubicación en el proyecto: `resources/js/cineverse/features/movies/components/movie-detail/MovieHero.vue` · Component `Vue · defineProps()/defineEmits()`. Lo utiliza `MovieDetailView.vue` como componente hijo mediante props y eventos.

```
// MovieHero recibe datos del padre
defineProps<{
  movie: MovieDetail;
  cast: CastMember[];
  director: { name: string } | null;
  hasTrailer: boolean;
}>();

// MovieHero avisa al padre
const emit = defineEmits(["openTrailer", "ready"]);
```

Con estas definiciones, TypeScript puede avisar si el padre intenta pasar un tipo incorrecto. En `MovieDetailView`, `openTrailer` se conecta con el evento emitido por `MovieHero` y `ready` se utiliza para retirar el estado de carga cuando el hero terminó de preparar su imagen.

11.9 Tipos de las respuestas del backend

El mismo criterio se utiliza con la API Laravel. `reviews.service.ts` declara que `getMovieReviews()` devuelve `ReviewsResponse` y que `getUserReviews()` devuelve `UserReviewsResponse`. Esto permite que `MovieReviews` sepa que data contiene stats, pagination, user_review y reviews sin trabajar con objetos completamente desconocidos.

El tipo ayuda durante el desarrollo, pero sigue sin validar el JSON real en tiempo de ejecución. Si un servidor devolviera una estructura distinta, TypeScript no podría detener esa respuesta de red; por eso los estados de error y las validaciones del backend siguen siendo necesarios.

12. Cómo está organizado el frontend con Vue

`resources/js/app.ts` es el punto de entrada real del frontend dentro de Laravel. Inicia `Inertia` con `createInertiaApp()`, resuelve `Pages/CineVerse.vue`, crea la aplicación `Vue` y registra `Pinia`, `Vue Router`, `Vue Query`, `vue-i18n` y `Ziggy`. En la versión `Laravel` ya no existe un `main.ts` independiente: `Vite` entra por `app.ts` y desde allí se monta la aplicación `CineVerse` completa.

Montaje principal

Ubicación en el proyecto: `resources/js/app.ts` · `entrypoint` `Vue/Inertia` · `createInertiaApp()`. El fragmento está abreviado para mostrar resolución de `Pages` y registro de plugins; `Pages/CineVerse.vue` monta después la aplicación `CineVerse`.

```
createInertiaApp({
  resolve: name => resolvePageComponent(
    `./Pages/${name}.vue`,
    import.meta.glob("./Pages/**/*.vue")
  ),
  setup({ el, App, props, plugin }) {
    const pinia = createPinia();
    pinia.use(piniaPluginPersistedState);

    const app = createApp({ render: () => h(App, props) });

    app
      .use(plugin)
      .use(pinia)
      .use(router)
```

```

    .use(VueQueryPlugin)
    .use(i18n)
    .use(ZiggyVue);

    useAuthStore().initialize();

    router.isReady().then(() => app.mount(el));
  }
});

```

12.1 Comprobación de la sesión al iniciar la aplicación

Al arrancar, `useAuthStore.initialize()` consulta `GET /user` para saber si el navegador ya tiene una sesión válida. `app.ts` dispara esa comprobación sin bloquear las rutas públicas. Los guards de Vue Router esperan la misma Promise únicamente cuando la ruta requiere saber si hay sesión, por ejemplo Perfil o Login.

Ventaja

Home y Movies no quedan bloqueadas por una consulta a `/user`. Las rutas que sí necesitan conocer el estado de sesión esperan el resultado de forma selectiva.

Como se vio en el recorrido de una película, una Promise representa una operación asíncrona cuyo resultado llegará más adelante. Aquí la implementación reutiliza una única Promise para evitar que `app.ts` y un guard de Vue Router envíen dos `GET /user` al mismo tiempo. `initializePromise` queda guardada incluso después de resolverse; por eso, si la primera consulta fallara por una caída temporal del backend, otra llamada durante la misma vida de la SPA reutilizaría esa promesa ya finalizada en lugar de disparar automáticamente una nueva consulta.

Detalle de integración actual. `app.ts` espera `router.isReady()` antes de montar Vue. Esto permite que los guards de rutas exclusivas para invitados redirijan a un usuario ya autenticado antes del primer render y evita mostrar brevemente el formulario de login o registro. El documento raíz ya no es un `index.html` del frontend independiente: Laravel utiliza `resources/views/app.blade.php` como vista base de Inertia.

12.2 Organización de archivos por funcionalidades (features)

La carpeta `features` agrupa archivos por funcionalidad. `auth` contiene registro, login y recuperación; `movies` reúne exploración, filtros, detalle, reseñas y favoritos; `person` contiene la información de actores; y `profile` reúne las pantallas del perfil. `core` queda reservado para código común utilizado por varias funcionalidades, como Axios, el router y los idiomas.

12.3 Componentes y composables

Una vista representa una pantalla completa. Los componentes dividen esa pantalla en partes más pequeñas y reciben datos o emiten eventos. Los composables son funciones que reúnen lógica de Vue que puede reutilizarse: `useMoviesFilters` controla los filtros, `useMoviesQuery` carga las páginas del explorador, `useMovieDetail` combina los datos de una película y `usePersonDetail` obtiene la información de un actor.

12.3.1 Cómo reconocer View, Component, Composable y Service en el código

La carpeta `resources/js/cineverse` está organizada por funcionalidades. El mismo recorrido suele atravesar varios tipos de archivo, y cada uno resuelve una parte distinta del problema.

Ubicación en el proyecto	Tipo	Responsabilidad en el ejemplo
<code>resources/js/cineverse/features/movies</code> <code>/views/MoviesView.vue</code>	View	Coordina filtros, consulta y grilla del explorador.

Ubicación en el proyecto	Tipo	Responsabilidad en el ejemplo
resources/js/cineverse/features/movies/components/MovieResults.vue	Component	Recibe páginas y construye la lista visible de MovieCard.
resources/js/cineverse/features/movies/composables/useMoviesQuery.ts	Composable	Decide qué consulta ejecutar y configura useInfiniteQuery.
resources/js/cineverse/features/movies/services/movies.service.ts	Service	Construye las llamadas concretas hacia los endpoints de TMDB.
resources/js/cineverse/features/movies/stores/useMoviesFavoritesStore.ts	Store	Mantiene y persiste la lista compartida de favoritos.

```

MoviesView
-> useMoviesFilters()
-> useMoviesQuery()
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> Vue Query devuelve data
-> MovieResults
-> MovieCard

```

La flecha no significa que todos los archivos se importen entre sí directamente. Representa el recorrido conceptual de los datos: la vista coordina, el composable decide, el service solicita y los componentes representan.

12.4 Cómo funciona la reactividad de Vue en CineVerse

Vue mantiene valores reactivos para que la interfaz se actualice cuando cambian los datos sin reconstruir manualmente el HTML. CineVerse usa principalmente `ref`, `computed`, `reactive` y `watch`.

- `ref(value)` guarda un valor reactivo. Ejemplo: `MovieReviews` usa `formRating`, `formComment`, `isSubmitting` y `reviews` como refs.
- `computed(...)` calcula un valor a partir de otros valores reactivos. `favoritesCount` se recalcula a partir de `favorites.length` y `canSubmitForm` depende del `rating` y la longitud del comentario.
- `reactive({...})` agrupa varias propiedades reactivas en un objeto; `useMoviesFilters` lo utiliza para `listType`, `texto`, `géneros`, `años`, `puntuación` y `orden`.
- `watch(...)` ejecuta lógica cuando cambia un valor. CineVerse lo usa, por ejemplo, para esperar brevemente después de que el usuario deja de escribir antes de buscar —técnica llamada `debounce`—, bloquear el scroll al abrir un modal o resetear la grilla al cambiar filtros.

Ejemplo: al ejecutar `favorites.value.push(movie)`, no hace falta buscar el contador en el DOM y modificarlo. `favoritesCount` depende de `favorites`, Vue detecta el cambio y las expresiones del template que usan ese contador se vuelven a evaluar automáticamente.

12.5 Componentes padre, props y emits

Los componentes no comparten variables locales de forma directa. El padre entrega datos mediante `props` y el hijo puede emitir eventos. `MovieTabs` recibe `cast`, `similar`, `movieId` y `movieTitle` desde `MovieDetailView`; luego entrega `movieId` y `movieTitle` a `MovieReviews`. `MovieHero`, en cambio, emite `openTrailer` para que `MovieDetailView` sea quien abra el modal.

```

MovieDetailView
-> props -> MovieHero
<- emit "openTrailer" - MovieHero

MovieDetailView
-> props movieId/movieTitle -> MovieTabs
-> props movieId/movieTitle -> MovieReviews

```

12.6 Ciclo de vida: onMounted y onUnmounted

onMounted se ejecuta cuando el componente ya fue creado y colocado en el DOM. MovieReviews lo utiliza para pedir la primera página de reseñas. ProfileView lo usa para cargar el perfil. MovieDetailView registra allí el listener de la tecla Escape para cerrar el trailer.

onUnmounted se ejecuta cuando el componente se destruye. MovieDetailView elimina el listener global de teclado y MovieResults desconecta su IntersectionObserver. Esta limpieza evita que queden funciones activas después de abandonar una pantalla.

12.7 App.vue, RouterView y cuándo se recrea una vista

App.vue mantiene AppHeader y AppFooter fuera de RouterView para que el marco general permanezca mientras cambia la pantalla central. RouterView recibe el componente asociado a la URL actual.

routeKey(route) construye una key con el nombre de la ruta y sus parámetros, pero excluye lang. Por eso pasar de /es/movies/550 a /es/movies/272 cambia el id y fuerza a recrear la vista, mientras cambiar sólo /es por /en conserva la instancia y deja que la reactividad del idioma y Vue Query actualicen los datos.

12.8 Formularios del frontend: este proyecto no utiliza useForm() de Inertia

Aunque Inertia forma parte de la integración, los formularios visibles de CineVerse no están contruidos con useForm() de @inertiajs/vue3. Login, registro, recuperación y reseñas mantienen su estado con ref/computed/watch, validan en Vue y envían las solicitudes mediante services TypeScript y Axios.

```

LoginView.vue
-> refs: email / password / estados visuales
-> loginUser(payload)
-> auth.service.ts
-> webApi.post("/login", payload)
-> Laravel

```

Por eso no tendría sentido documentar useForm() como si fuera parte del funcionamiento real de esta versión. Inertia entrega la aplicación y comparte contexto, pero los formularios de CineVerse conservan el enfoque Vue + Axios del frontend integrado.

13. Cómo se guardan y reutilizan datos en el frontend

En el frontend aparecen tres conceptos diferentes. Estado es la información que la aplicación necesita recordar mientras está funcionando, por ejemplo qué usuario inició sesión. Caché es una copia temporal de datos ya obtenidos que puede reutilizarse para evitar solicitudes innecesarias. Persistencia significa conservar determinados datos aunque la página se recargue o el usuario vuelva más tarde.

CineVerse usa herramientas distintas para cada necesidad: Pinia mantiene estado compartido, Vue Query administra datos obtenidos mediante solicitudes HTTP y localStorage o sessionStorage conservan ciertos valores dentro del navegador.

13.1 Pinia: autenticación

El store de autenticación de Pinia mantiene el usuario actual, el token CSRF, si existe una sesión autenticada y si ya terminó la comprobación inicial. Como el store está disponible desde distintas pantallas, el header, el router y el perfil pueden consultar el mismo estado sin duplicarlo.

13.1.1 Cómo se instala Pinia y por qué un store es compartido

app.ts crea una única instancia de Pinia con createPinia() y registra pinia-plugin-persistedstate. Esa instancia se instala dentro del setup() real de createInertiaApp(), sobre la misma aplicación Vue que recibe Inertia, Vue Router, Vue Query, vue-i18n y Ziggy. A partir de ese momento, cualquier componente puede llamar useAuthStore() o useFavoritesStore() y trabajar con el mismo estado compartido en lugar de mantener copias independientes.

Ubicación en el proyecto: `resources/js/app.ts` · `entrypoint` `Vue/Inertia` · `setup()` de `createInertiaApp()`. Este código crea la instancia compartida de Pinia e instala `pinia-plugin-persistedstate` antes del montaje de la aplicación.

```
const pinia = createPinia();
pinia.use(piniaPluginPersistedState);
```

Pinia no significa almacenamiento permanente por sí mismo. El store de autenticación vive en memoria y se reconstruye consultando `GET /user`. El store de favoritos, en cambio, tiene `persist: true` y por eso `pinia-plugin-persistedstate` puede conservarlo en `localStorage`.

13.1.2 Store de autenticación por dentro

`useAuthStore` mantiene `user`, `csrfToken` e `isInitialized`. `isAuthenticated` es un `computed` que vale `true` cuando `user` no es `null`. Después de un login correcto, `LoginView` llama `setUser()` y `setCsrfToken()`; al cerrar sesión o detectar que la sesión ya no es válida, `clear()` vuelve ambos valores a `null`.

Esquema resumido del estado de `useAuthStore`; no es un bloque literal del archivo. La implementación real está en `resources/js/cineverse/features/auth/stores/useAuthStore.ts` y es consumida por `app.ts`, `guards` y vistas autenticadas.

```
user: AuthUser | null
csrfToken: string | null
isAuthenticated = computed(() => user !== null)
isInitialized: boolean

login correcto
-> setUser(response.data.user)
-> setCsrfToken(response.data.csrf_token)
```

`initialize()` es especial: llama `GET /user` para reconstruir el estado a partir de la cookie de sesión. Mantiene una `initializePromise` compartida para que `app.ts` y un `guard` no disparen dos consultas al mismo tiempo. La implementación actual conserva esa `Promise` incluso después de terminar; si la primera consulta falló por una caída temporal, una nueva llamada durante la misma vida de la SPA no fuerza automáticamente otro intento.

13.2 Pinia: favoritos

El store de favoritos conserva una lista de objetos `Movie`, evita duplicados y limita la colección a 20. `pinia-plugin-persistedstate` guarda ese store en `localStorage`, por lo que los favoritos pueden sobrevivir a una recarga del navegador. No se envían al backend ni se asocian con la cuenta.

13.3 Vue Query

Las pantallas de Inicio, Películas, detalle y actores utilizan `TanStack Vue Query` para pedir datos y controlar estados como carga, error y datos disponibles. Cada consulta tiene una `queryKey`, es decir, una clave que identifica qué información representa. Cuando cambian valores importantes como el `id`, el idioma o los filtros, cambia la clave y `Vue Query` sabe que debe tratar el resultado como otra consulta.

Detalle de implementación actual. En `useMoviesQuery.ts`, la opción `enabled` termina con `|| true`. Por eso la expresión siempre resulta verdadera y la consulta queda habilitada en todos los casos. Las condiciones escritas antes de ese operador no llegan a decidir el valor final de `enabled` en esta versión.

Área	Estrategía observada
Home	Tres consultas independientes. Los datos se consideran recientes durante 5 minutos y se mantienen temporalmente los anteriores al cambiar de idioma.
Explorador	Consulta paginada con <code>useInfiniteQuery</code> . Se fuerza una carga nueva al volver a montar el explorador.
Detalle de película	Consulta por <code>id</code> e idioma. No reintenta automáticamente y obtiene créditos, videos y similares después del detalle principal.
Actor	Consulta por <code>id</code> e idioma. No reintenta automáticamente.

13.3.1 Qué guarda realmente Vue Query

Vue Query no reemplaza a Pinia. Se usa para datos obtenidos mediante solicitudes: películas de TMDB, detalle de actores y resultados de la Home. Una `queryKey` identifica el resultado. Si cambia una parte importante de esa clave —por ejemplo el id, el idioma o un filtro—, Vue Query considera que se trata de otra consulta.

En Home los datos son considerados recientes durante cinco minutos. En el explorador `staleTime` y `gcTime` valen 0, por lo que el comportamiento busca datos más actuales y no conserva la caché al desconectarse. En detalle de película y actor la key incluye id e idioma.

13.3.2 queryKey y queryFn: identidad y trabajo de una consulta

Una consulta de Vue Query tiene dos ideas centrales. `queryKey` identifica qué información representa; `queryFn` es la función que obtiene esa información cuando hace falta.

Pseudocódigo didáctico; no es un bloque literal. La implementación real está en `resources/js/cineverse/features/movies/composables/useMoviesQuery.ts`, donde `useInfiniteQuery` construye `queryKey` y `queryFn` a partir de `useMoviesFilters`.

```
queryKey: ["movies", listType, debouncedQuery, genres, yearFrom, yearTo, minRating, maxRating,
  sortBy, language]

queryFn: ({ pageParam }) => {
  // decide entre search/movie, discover/movie o movie/{listType}
}
```

Si cambia una parte importante de `queryKey` —por ejemplo el idioma, Batman por Matrix o un género— Vue Query deja de tratar el resultado como la misma consulta. Esto permite que el explorador reaccione a filtros sin que `MovieResults` conozca cómo se construyó la request.

13.3.3 useQuery y useInfiniteQuery no resuelven el mismo problema

Herramienta	Dónde se usa	Qué aporta
<code>useQuery</code>	Home, detalle de película, detalle de persona	Una consulta identificada por una key. Expone data, estados de carga/error y opciones de cache.
<code>useInfiniteQuery</code>	Explorador de películas	Mantiene varias páginas bajo una misma consulta y expone <code>fetchNextPage</code> / <code>hasNextPage</code> .

13.3.4 Estados que una vista recibe de Vue Query

Valor	Significado práctico
<code>data</code>	Datos confirmados que devolvió la query.
<code>isLoading</code> / <code>isPending</code>	La primera carga todavía no produjo datos utilizables.
<code>isError</code> / <code>error</code>	La operación terminó con error y la vista puede mostrar un estado controlado.
<code>fetchNextPage()</code>	Solicita la página siguiente en una consulta infinita.
<code>hasNextPage</code>	Indica si <code>getNextPageParam</code> encontró otra página disponible.
<code>isFetchingNextPage</code>	Evita mostrar o disparar varias cargas de página simultáneas.

13.3.5 Pinia, Vue Query y Axios: tres responsabilidades diferentes

Herramienta	Pregunta que responde	Ejemplo en CineVerse
Pinia	¿Qué estado compartido propio de la app necesito recordar?	Usuario autenticado y favoritos.

Herramienta	Pregunta que responde	Ejemplo en CineVerse
Vue Query	¿Qué datos remotos obtuve y en qué estado está la consulta?	Películas, personas y páginas de resultados.
Axios	¿Cómo envío la solicitud HTTP?	tmdbApi.ts y webApi.ts.
localStorage / sessionStorage	¿Qué dato quiero conservar en el navegador?	Favoritos, preferencias visuales y último estado de filtros.

Axios no reemplaza a Vue Query y Vue Query no reemplaza a Pinia. Axios transporta; Vue Query administra datos remotos; Pinia mantiene estado compartido de la aplicación.

13.4 sessionStorage y localStorage

Dato	Dónde se guarda	Cuánto tiempo permanece
Último query de filtros	sessionStorage	Aislado por pestaña; permite volver a Movies con la URL limpia.
Favoritos	Persistencia de Pinia / localStorage	Sobrevive a recargas y nuevas sesiones del navegador.
Banner de Firefox descartado	localStorage	No vuelve a mostrarse después de cerrarlo.
Orden/estado visual de paneles	localStorage	Se recupera entre visitas según el componente de filtros.

sessionStorage y localStorage son almacenamientos que ofrece el navegador. sessionStorage vive durante la pestaña actual; localStorage puede mantenerse después de cerrar y volver a abrir el sitio. CineVerse usa sessionStorage para recordar filtros y evitar repetir el skeleton completo de Home dentro de la misma pestaña, y localStorage para favoritos y preferencias visuales.

14. Navegación sin recargar la página: SPA y Vue Router

Como CineVerse es una SPA, cambiar de sección no implica descargar una página HTML completa desde el servidor cada vez. Vue Router decide qué vista debe mostrarse según la URL y Vue reemplaza el contenido correspondiente dentro de la aplicación ya cargada.

Una ruta es la relación entre una URL y una pantalla. Por ejemplo, /es/movies muestra el explorador y /es/movies/550 muestra el detalle de la película cuyo id es 550. CineVerse utiliza createWebHistory, por lo que las URLs se ven como direcciones normales y no utilizan el símbolo #.

El idioma también forma parte de la ruta: las secciones funcionales comienzan con /es o /en. Esto permite compartir una URL que conserva tanto la pantalla como el idioma seleccionado.

Ruta	Vista / propósito
/:lang	HomeView
/:lang/movies	Explorador de películas
/:lang/movies/:id	Detalle de película; id sólo numérico
/:lang/person/:id	Detalle de actor; id sólo numérico
/:lang/favorites	Favoritos locales
/:lang/profile	Perfil; requiresAuth
/:lang/profile/reviews	Mis reseñas; requiresAuth
/:lang/login	Login; requiresGuest

Ruta	Vista / propósito
/:lang/register	Registro; requiresGuest
/:lang/forgot-password	Recuperación; requiresGuest
/:lang/reset-password	Formulario de nueva contraseña

14.1 Detección inicial del idioma

En una carga directa del sitio, la raíz / llega primero a Laravel. `routes/web.php` redirige / a /es, por lo que el comportamiento observable del servidor es comenzar en español. Una vez montada la SPA, Vue Router controla las rutas `/:lang/...` y el guard global sincroniza el parámetro `lang` con `vue-i18n`. La ruta raíz definida dentro de Vue Router conserva lógica de detección del idioma del navegador, pero una entrada HTTP normal a / ya fue resuelta antes por Laravel.

```
Carga directa en /
-> Laravel routes/web.php
-> redirect("/es")
-> Inertia entrega CineVerse
-> Vue Router resuelve /es
-> vue-i18n locale = "es"
```

14.2 Protecciones de navegación (guards)

Un guard es una función que se ejecuta antes de completar una navegación. `requiresAuth` evita entrar al perfil o a las reseñas propias sin una sesión válida. `requiresGuest` hace lo contrario: evita mostrar Login, Register y ForgotPassword a quien ya inició sesión. Ambos esperan `initialize()` únicamente cuando necesitan conocer el estado de autenticación.

14.3 Qué ocurre cuando una ruta no existe: error 404

Una ruta 404 se muestra cuando la URL no coincide con ninguna pantalla válida. CineVerse tiene rutas de captura general para conservar /es o /en cuando sea posible. Las rutas de detalle aceptan únicamente ids numéricos, por lo que `/es/movies/abc` termina en `NotFoundView` sin enviar una solicitud innecesaria a TMDB.

14.4 createWebHistory, RouterView y routeKey

`createWebHistory(import.meta.env.BASE_URL)` hace que CineVerse utilice URLs normales, sin #. Vue Router observa la URL y RouterView muestra la vista asociada. En la integración Laravel, una recarga directa de `/es/movies/550` no depende de un mecanismo de respaldo hacia `index.html`: el catch-all localizado de `routes/web.php` devuelve `Inertia::render("CineVerse")` y, una vez montada la app, Vue Router resuelve `MovieDetailView`.

`App.vue` añade una segunda decisión mediante `routeKey()`: un cambio de id recrea la vista; un cambio únicamente de idioma no. Esto evita conservar los datos de otra película al cambiar de `/movies/550` a `/movies/272`, pero permite cambiar ES/EN sin destruir todo el estado visual del componente.

14.5 Ejemplo completo de navegación a otra película

```
Usuario pulsa una MovieCard con id 550
-> MovieCard.goToMovie(550)
-> router.push({ name: "MovieDetail", lang, id: 550 })
-> beforeEach sincroniza el idioma
-> Router resuelve /es/movies/550
-> RouterView recibe MovieDetailView
-> routeKey incluye id=550
-> Vue monta la vista
-> useMovieDetail(550, locale)
-> TMDB
```

`scrollBehavior()` devuelve `{ top: 0 }`, por lo que una navegación comienza arriba de la nueva pantalla en lugar de conservar la posición vertical de la vista anterior.

15. Idiomas: vue-i18n y cambio de idioma

i18n es una abreviatura utilizada en desarrollo para internationalization, es decir, internacionalización. CineVerse utiliza vue-i18n para mantener los textos de la interfaz en español e inglés sin duplicar la lógica de cada componente. Las traducciones se guardan en `resources/js/cineverse/core/i18n/es.json` y `resources/js/cineverse/core/i18n/en.json`.

Ubicación en el proyecto: `resources/js/cineverse/core/i18n/index.ts` · módulo de configuración global de vue-i18n. La instancia exportada se instala desde `resources/js/app.ts` y es utilizada por los componentes mediante `useI18n()`.

```
const i18n = createI18n({
  legacy: false,
  locale: "es",
  fallbackLocale: "en",
  messages: { en, es },
});
```

`legacy: false` hace que vue-i18n utilice su API moderna para Vue 3. `locale` indica el idioma activo y `fallbackLocale` define un idioma de respaldo. Los componentes no escriben una misma frase dos veces: solicitan una clave como `t("nav.movies")` y vue-i18n busca el texto correspondiente en el diccionario activo.

15.1 Qué ocurre al cambiar de español a inglés

El idioma forma parte de la URL. Si el usuario está en `/es/movies/550` y pulsa EN, `AppHeader` ejecuta `switchLang("en")` y Vue Router navega a la misma ruta con `lang="en"`. Antes de completar la navegación, el guard global lee `to.params.lang` y asigna ese valor a `i18n.global.locale.value`. A partir de ese momento las claves `t(...)` se resuelven contra `en.json`.

Ubicación en el proyecto: `resources/js/cineverse/core/router/index.ts` · router global · `beforeEach()`. Lo utiliza Vue Router en cada navegación para sincronizar `:lang` con vue-i18n y aplicar los guards.

```
router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  // ... guards de autenticación
});
```

El cambio de idioma no modifica solamente los botones y títulos de la interfaz. Inicio, Películas, detalle de película y detalle de actor incluyen el idioma en sus consultas, por lo que TMDB recibe el language correspondiente. `useLocalizedDate` utiliza `es-AR` o `en-US` para decidir el orden visual de día y mes mediante `Intl.DateTimeFormat`.

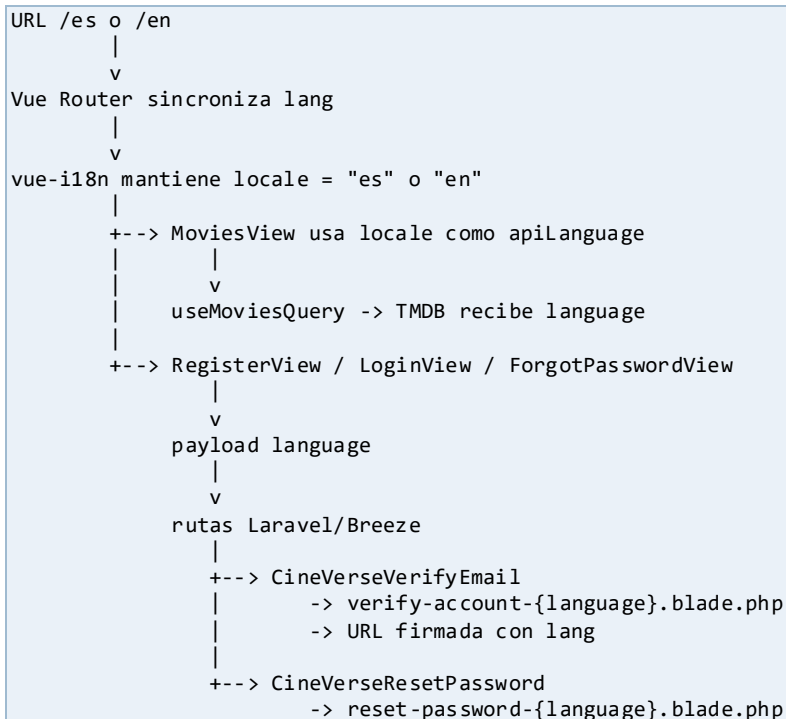
Dos particularidades forman parte de la implementación actual. `useLocalizedDate` convierte primero el texto recibido con `new Date(dateStr.replace(" ", "T"))`; cuando el origen es una fecha sin hora, la interpretación de `Date` y la zona horaria local pueden desplazar el día mostrado. Además, `PersonHero.vue` mantiene los textos "Leer más" y "Leer menos" escritos directamente en español, por lo que esas dos etiquetas no cambian al seleccionar EN.

El idioma también se envía a Laravel en los flujos de registro, reenvío de verificación y recuperación de contraseña. Las notificaciones `CineVerseVerifyEmail` y `CineVerseResetPassword` leen ese valor y seleccionan las vistas Blade `resources/views/emails/verify-account-es.blade.php`, `verify-account-en.blade.php`, `reset-password-es.blade.php` o `reset-password-en.blade.php`.

Caso límite actual: switchLang conserva parámetros de la ruta, como el id de una película, pero no copia route.query. route.query contiene los parámetros que aparecen después del signo ? en una URL. Por eso, si el usuario cambia de idioma desde una URL del explorador que contiene búsqueda, géneros, años u otros filtros, esos parámetros se pierden en la navegación actual.

15.1.1 Cómo el idioma llega a TMDb y a los correos

El idioma no se decide de forma independiente en cada parte del sistema. Vue Router y vue-i18n mantienen un único idioma activo y distintas funciones reutilizan ese valor. Por eso conviene seguir el recorrido completo desde la URL hasta las consultas y los correos.



En MoviesView, `const { locale } = useI18n()` obtiene el idioma actual y `apiLanguage` apunta a ese mismo valor reactivo. `useMoviesQuery` incluye `language.value` dentro de su `queryKey`. Por eso, cuando el usuario cambia de /es a /en, Vue Query considera que la consulta cambió y vuelve a pedir los datos con el nuevo idioma.

En los formularios que pueden producir un correo sucede algo parecido. RegisterView toma `locale.value` y lo envía como `language` dentro del objeto enviado a POST /register. ForgotPasswordView lo manda a POST /forgot-password y LoginView lo usa cuando solicita el reenvío del correo de verificación.

Ubicación en el proyecto: `resources/js/cineverse/features/auth/views/RegisterView.vue` · View Vue · submit del registro. La vista toma `locale` y llama `registerUser()` de `auth.service.ts` con `language` dentro del payload.

```

const lang = locale.value as "es" | "en";

await registerUser({
  username,
  email,
  password,
  password_confirmation,
  language: lang
});

```

En Laravel no existe un AuthService o MailService propio para esta tarea. RegisteredUserController y PasswordResetLinkController conservan `language` dentro de la solicitud; cuando User dispara CineVerseVerifyEmail o CineVerseResetPassword, cada notificación toma `request()->input("language", app()->getLocale())`, acepta únicamente es o en y utiliza en como respaldo. La verificación incluye además `lang` dentro de la URL firmada para que el regreso a la SPA conserve el idioma. El flujo completo de registro, verificación y Password Broker se desarrolla en la sección 26.

15.1.2 Ejemplo guiado: Carlos cambia el menú de español a inglés

Carlos está usando CineVerse en español. AppHeader muestra Inicio, Películas y Favoritos. Al pulsar EN, el botón no reemplaza palabras manualmente: ejecuta `switchLang("en")` y pide a Vue Router navegar a la misma ruta con otro valor para `lang`.



Figura. Encabezado de CineVerse antes del cambio de idioma, con ES activo.

Ubicación en el proyecto: `resources/js/cineverse/layouts/components/AppHeader.vue` · Component · selector de idioma y función `switchLang()`.

```
function switchLang(lang: string) {
  router.push({
    name: route.name as string,
    params: { ...route.params, lang },
  });
}
```

Carlos pulsa EN

```
-> AppHeader ejecuta switchLang("en")
-> /es/... pasa a /en/...
-> router.beforeEach lee lang="en"
-> i18n.global.locale.value = "en"
-> t(...) se vuelve a resolver
-> Vue actualiza los textos visibles
```

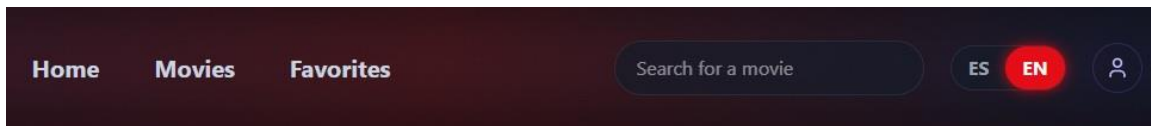


Figura. El mismo encabezado después del cambio a EN.

15.1.3 Cómo `t("nav.movies")` termina mostrando Películas o Movies

Los componentes solicitan una clave, no una frase fija. AppHeader utiliza `t("nav.movies")`; `vue-i18n` busca esa clave dentro del diccionario activo. Ubicación en el proyecto: `resources/js/cineverse/core/i18n/es.json` y `resources/js/cineverse/core/i18n/en.json` · diccionarios de traducción. El bloque siguiente muestra sólo un fragmento parcial de `nav` para ilustrar la misma clave en ambos idiomas; los objetos reales contienen más claves.

```
// es.json
"nav": {
  "home": "Inicio",
  "movies": "Películas",
  "favorites": "Favoritos"
}

// en.json
"nav": {
  "home": "Home",
  "movies": "Movies",
  "favorites": "Favorites"
}

t("nav.movies")
-> locale="es" -> es.json -> "Películas"
-> locale="en" -> en.json -> "Movies"
```

15.1.4 Ejemplo guiado: Carlos cambia el idioma de la ficha de Matt Damon

La biografía de una persona no sale de `en.json`. Carlos abre la ficha con ES activo; `PersonDetailView` entrega el `id` y `locale` a `usePersonDetail()`. Al cambiar a EN, el idioma forma parte de la identidad de la consulta y TMDB vuelve a recibir una request localizada.

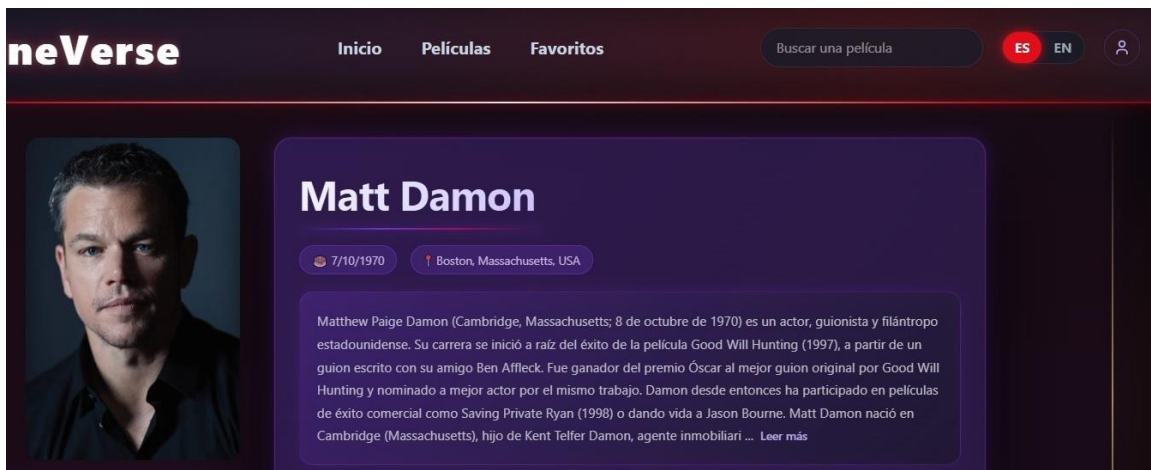


Figura. Ficha de Matt Damon con la interfaz y la biografía en español.

Ubicación en el proyecto: `resources/js/cineverse/features/person/composables/usePersonDetail.ts` · Composable · consulta identificada por persona e idioma.

```
return useQuery<PersonDetail>({
  queryKey: ["person-detail", id, language],
  enabled: Number.isFinite(id) && id > 0,
  retry: false,
  queryFn: async () => getFullPersonDetail(id, language.value),
});
```

Ubicación en el proyecto: `resources/js/cineverse/features/person/services/person.service.ts` · Service del frontend · llamada a TMDB.

```
api.get(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

Carlos pulsa EN
 -> locale cambia
 -> cambia queryKey
 -> Vue Query ejecuta otra vez queryFn
 -> person.service.ts envía language=en
 -> TMDB devuelve datos localizados
 -> PersonHero muestra person.biography



Figura. La misma ficha después de solicitar los datos de TMDB en inglés.

15.1.5 Traducir interfaz, localizar datos, formatear fechas y elegir un email

Qué cambia	De dónde sale	Mecanismo
Inicio → Home	es.json / en.json	vue-i18n y t(...)
Biografía de una persona	TMDB	language dentro de la consulta
Formato de fecha	JavaScript	Intl.DateTimeFormat con es-AR / en-US
Correo de verificación / reset	Laravel	Notification selecciona una vista Blade ES/EN

Cambiar una traducción en en.json no modifica la biografía que entrega TMDB. Son orígenes de datos diferentes que comparten el mismo idioma activo.

15.2 Diccionarios, claves e idioma de respaldo

Los dos archivos de idioma no contienen exactamente las mismas claves. Por ejemplo, t("nav.movies") busca nav.movies dentro del diccionario activo; para que una clave tenga traducción propia en ambos idiomas debe existir en es.json y en en.json.

En esta versión, es.json contiene 210 claves terminales y en.json 208. En inglés no están definidas auth.success.resendSuccessTitle y auth.success.resendSuccessSubtitle, aunque LoginView las utiliza en el estado exitoso del reenvío de verificación. fallbackLocale: "en" no puede suplir una clave que falta precisamente en el diccionario inglés.

fallbackLocale: "en" define el inglés como idioma de respaldo: si una clave falta en el idioma activo, vue-i18n intenta buscarla allí. En esta versión, cuando una clave falta precisamente en en.json, el fallback no proporciona otra traducción inglesa y la ausencia queda expuesta en ese flujo.

16. Comunicación entre Vue, TMDB y Laravel

CineVerse mantiene separadas dos comunicaciones de datos y un puente de aplicación. Vue consulta TMDB directamente para catálogo y personas; Vue consulta Laravel para cuenta, sesión, perfil y reseñas; y Laravel/Inertia entrega la aplicación Vue cuando el navegador entra por una ruta web. Laravel usa Eloquent para leer y escribir MySQL.

El navegador elige el camino según la acción. La ficha de una película puede obtener sinopsis, reparto o imágenes desde TMDB sin consultar MySQL. En cambio, iniciar sesión, recuperar una contraseña, abrir el perfil o publicar una reseña necesita al backend Laravel y a la base propia.

16.1 De Vue a TMDB: recorrido resumido

Cuando una pantalla necesita películas o actores, un service del frontend utiliza tmdbApi.ts para consultar TMDB directamente. Ese cliente aporta la URL base y la cabecera Authorization que contiene Bearer <token>; la respuesta vuelve al frontend para que Vue la represente. La sección 4 desarrolla paso a paso el caso concreto de Mejor valoradas.

```
Pantalla Vue
-> service de movies o person
-> tmdbApi.ts
-> TMDB
-> respuesta JSON
-> Vue Query / vista
```

En este camino PHP y MySQL no intervienen: el Bearer token identifica a CineVerse frente a TMDB y la respuesta vuelve directamente al frontend.

16.2 De Vue a Laravel: ejemplo con el inicio de sesión

Cuando el usuario envía el formulario de login, LoginView llama a auth.service.ts. webApi envía POST /login con JSON, X-Requested-With y withCredentials:true. routes/auth.php entrega la solicitud a AuthenticatedSessionController, que utiliza LoginRequest. Ese Form Request valida los datos, aplica RateLimiter, busca User con Eloquent, comprueba Hash::check, exige correo verificado y abre una sesión con Auth::login(). Después Vue consulta GET /user para reconstruir user y csrf_token en Pinia.

```

Usuario pulsa "Ingresar"
-> LoginView.vue
-> auth.service.ts
-> webApi.ts
-> POST /login
-> routes/auth.php
-> AuthenticatedSessionController::store()
-> LoginRequest::authenticate()
-> User / Eloquent / MySQL
-> sesión Laravel
-> GET /user
-> useAuthStore / Pinia

```

Aquí sí intervienen la API propia, MySQL y la sesión Laravel. Una vez que existe una sesión autenticada, las operaciones de reseñas y el cierre de sesión combinan la cookie de sesión con un X-CSRF-Token enviado explícitamente por CineVerse.

16.3 HTTP, JSON y los dos clientes Axios

Ambos caminos utilizan HTTP y JSON. Authorization: Bearer identifica a la aplicación frente a TMDB. Las peticiones al backend propio permanecen en el mismo origen de Laravel y utilizan sus cookies web. La protección CSRF no depende siempre de GET /user: antes de iniciar sesión, rutas como /login, /register, /forgot-password y /reset-password siguen el mecanismo normal de Laravel y Axios, que utiliza la cookie XSRF-TOKEN y la cabecera X-XSRF-TOKEN. Después de autenticarse, GET /user devuelve además csrf_token; CineVerse lo conserva en Pinia y lo envía explícitamente como X-CSRF-Token al crear, editar o borrar reseñas y al cerrar la sesión.

Axios es la librería utilizada para enviar estas solicitudes. tmdbApi.ts habla con TMDB y webApi.ts habla con Laravel. Mantener dos instancias evita mezclar el Bearer token de TMDB con las reglas de sesión del backend propio.

tmdbApi.ts crea el cliente que habla con TMDB y webApi.ts crea el cliente que habla con Laravel. webApi usa baseUrl "/", Accept application/json, X-Requested-With XMLHttpRequest y withCredentials:true.

Característica	TMDB	Laravel / CineVerse
Archivo	resources/js/cineverse/core/api/tmdbApi.ts	resources/js/cineverse/core/api/webApi.ts
baseUrl	VITE_TMDB_BASE_URL	"/" (mismo origen de Laravel)
Autenticación	Authorization: Bearer <token TMDB>	Cookie de sesión del guard web
Envío de cookies (withCredentials)	No se habilita	true
CSRF	No aplica	X-CSRF-Token en escrituras protegidas
Datos	Películas, actores, videos, géneros	Cuenta, sesión, perfil y reseñas

Ubicaciones en el proyecto: `resources/js/cineverse/core/api/tmdbApi.ts` crea el cliente Axios de TMDB; `resources/js/cineverse/core/api/webApi.ts` crea el cliente del backend Laravel. Los services importan una u otra instancia según el destino.

```
// TMDB: cliente externo
const api = axios.create({
  baseURL: import.meta.env.VITE_TMDB_BASE_URL,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json"
  }
});

// Laravel: mismo origen que la aplicación
const webApi = axios.create({
  baseURL: "/",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
    "X-Requested-With": "XMLHttpRequest"
  },
  withCredentials: true
});
```

16.4 Laravel y Vite durante el desarrollo

En la versión Laravel integrada ya no existe el proxy del antiguo frontend independiente hacia `cineverse.local`. Laravel sirve la aplicación en su propio origen —por ejemplo `http://127.0.0.1:8000`— y Vite actúa como servidor de assets durante el desarrollo. `webApi` usa rutas relativas, por lo que `/login`, `/user` y `/api/reviews` llegan al mismo Laravel que entregó la página.

Estado actual

Los dos clientes Axios no tienen un tiempo máximo de espera global ni un interceptor global de errores. Un interceptor es una función común que puede revisar todas las solicitudes o respuestas antes de que lleguen al resto del código. Actualmente cada service o composable recibe el error y decide cómo tratarlo en su propio flujo.

16.5 Qué partes tiene una solicitud en CineVerse

Una solicitud puede transportar información en lugares diferentes. CineVerse utiliza parámetros de URL para paginación y filtros, un cuerpo JSON para datos de formularios —el payload—, cabeceras para Authorization/CSRF y cookies para la sesión Laravel.

- Parámetros de URL: en `GET /api/movies/550/reviews?page=2`, `page=2` viaja en la query string y Laravel lo obtiene con `$request->query('page', 1)`.
- Cuerpo JSON: `POST /api/reviews` envía `movie_id`, `movie_title`, `rating` y `comment`. Laravel decodifica el JSON y `$request->validate()` aplica las reglas del controlador.
- Cabeceras: Authorization lleva el Bearer token a TMDB. En las rutas web de Laravel, Axios puede enviar X-XSRF-TOKEN a partir de la cookie XSRF-TOKEN. Además, CineVerse envía explícitamente X-CSRF-Token en las operaciones autenticadas de reseñas y logout utilizando el token devuelto por `GET /user`.

La cookie de sesión es distinta: no forma parte del JSON ni del token CSRF. El navegador la administra como cookie HTTP y `webApi` usa `withCredentials:true` para mantener el comportamiento de credenciales de sesión.

16.6 Cookie, CSRF y Bearer no son lo mismo

```

TMDB
Authorization: Bearer <token TMDB>
-> identifica la aplicación frente a TMDB

Backend CineVerse / Laravel
Cookie de sesión
-> permite que el guard web recupere al usuario
XSRF-TOKEN / X-XSRF-TOKEN
-> protección CSRF web normal de Laravel/Axios

Reseñas y logout autenticados
Cookie de sesión + X-CSRF-Token
-> identidad de sesión + token explícito obtenido desde GET /user

```

Por ejemplo, GET /api/profile necesita sesión porque está dentro de middleware(['web', 'auth']). GET /api/movies/{movieId}/reviews es público, aunque usa el grupo web para que, si existe una sesión, pueda devolver la reseña propia separada del listado comunitario.

16.7 Cómo llega un error de Axios a una vista

Cuando Laravel responde con un estado HTTP de error, la Promise de Axios termina en estado de rechazo y el código pasa al manejo de errores. Las vistas pueden inspeccionar error.response?.data?.code o los errores de validación para convertirlos a mensajes localizados. El backend, por su parte, reserva el log técnico para fallos inesperados y no necesita mostrar un stack trace al usuario.

17. Explorador de películas y sistema de búsqueda

El explorador no utiliza una única forma de buscar. useMoviesQuery elige uno de tres caminos según lo que haya seleccionado el usuario: una lista predefinida de TMDB, Discover con filtros enviados a TMDB, o una búsqueda por texto en la que algunos filtros se aplican después en el navegador.

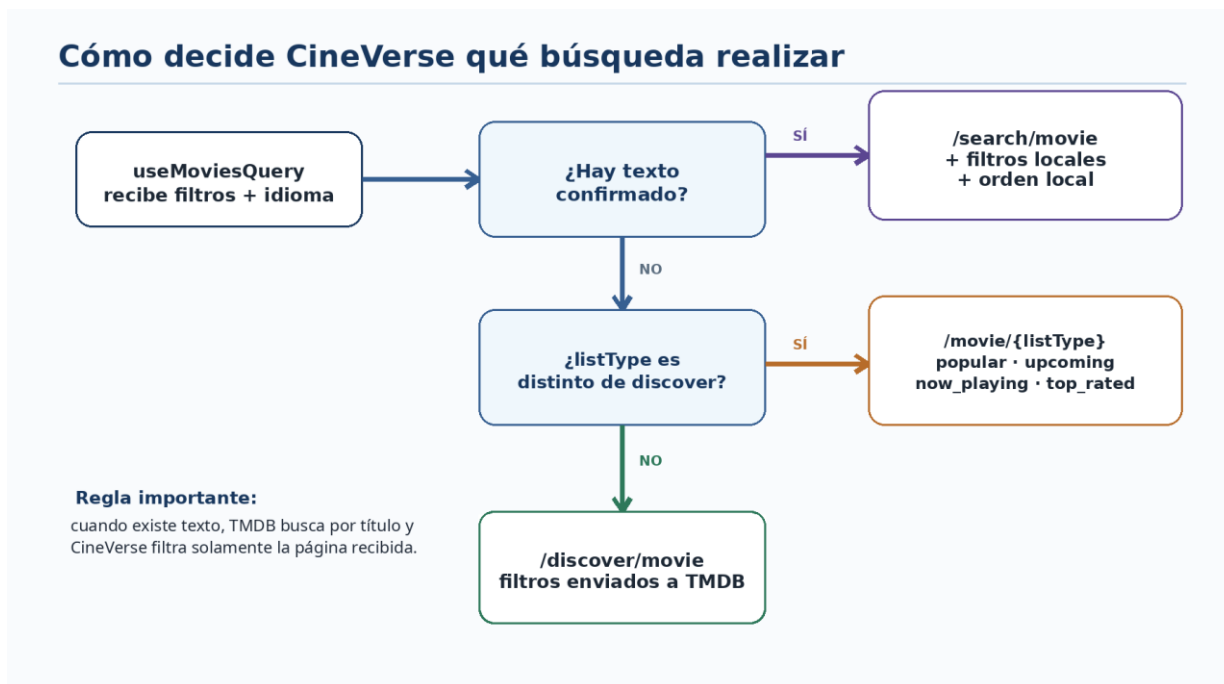


Figura. Decisión de useMoviesQuery entre texto, listas predefinidas y Discover.

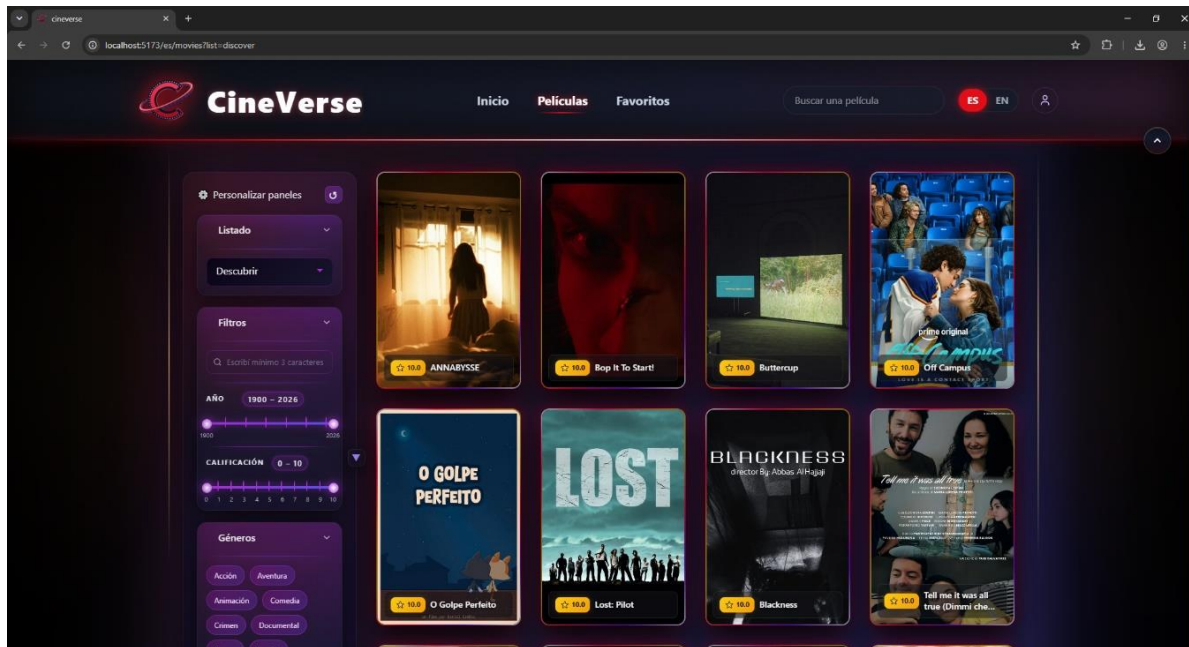


Figura. Explorador de películas con panel lateral y grilla de resultados.

17.1 Ejemplo guiado: Carlos entra a Películas y se construye la primera grilla

Carlos entra a /es/movies. MoviesView se monta y llama useMoviesFilters(). Si la URL no trae parámetros y tampoco existe un estado previo en sessionStorage, listType adopta el valor por defecto popular. A partir de ahí useMoviesQuery crea una consulta infinita cuya primera página vale 1.

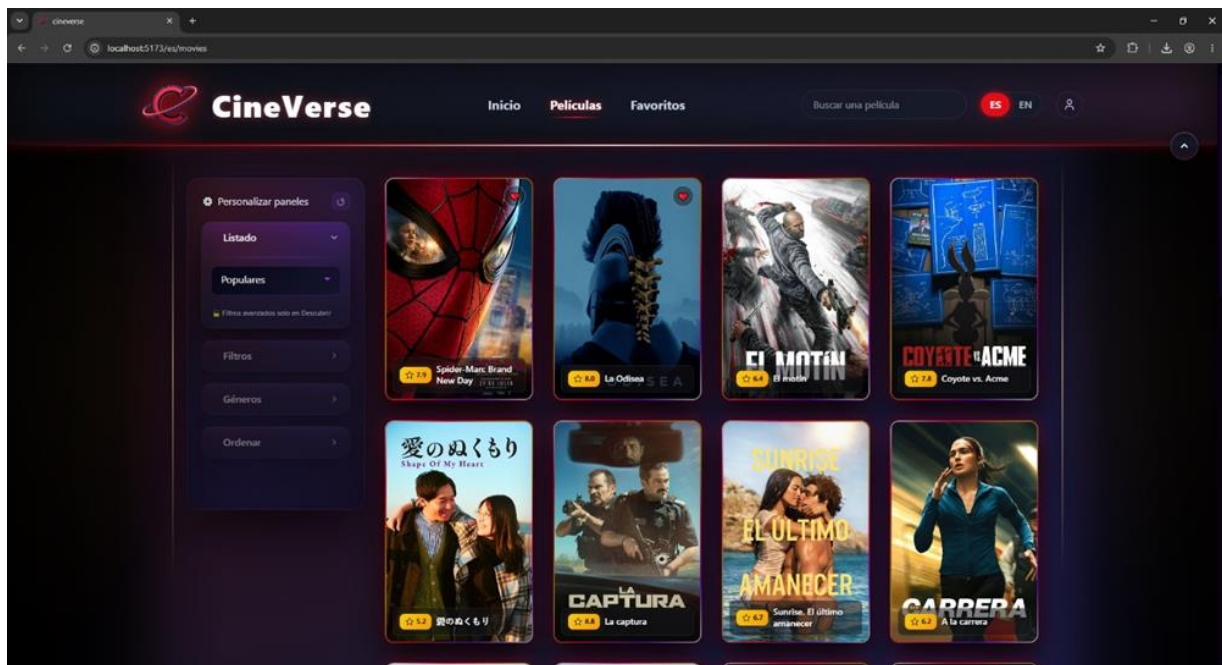


Figura. Explorador de Películas con catálogo Popular y grilla inicial.

```

Carlos entra a /es/movies
-> MoviesView.vue
-> useMoviesFilters()
-> sin URL ni estado anterior: listType="popular"
-> useMoviesQuery(filters, apiLanguage)
-> useInfiniteQuery(initialPageParam=1)
-> getMovieList("popular", 1, "es")
-> movies.service.ts
-> GET /movie/popular?page=1&language=es

```

```
-> TMDB devuelve PaginatedResponse<Movie>
-> Vue Query guarda la página en data.pages
-> MovieResults procesa los resultados
```

Ubicación en el proyecto: `resources/js/cineverse/features/movies/components/MovieResults.vue` · Component
· transforma las páginas recibidas en una lista de películas utilizables.

```
const pages = data?.pages;
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

Luego el template recorre `validMovies`. Cada vuelta del `v-for` crea una `MovieCard` distinta y le entrega el objeto `movie` correspondiente. `:delay="index * 30"` escalona la animación de entrada de las tarjetas.

Ubicación en el proyecto: `resources/js/cineverse/features/movies/components/MovieResults.vue` · **Component Vue** · template `v-for` de `MovieCard`

Lo utiliza: `MovieResults.vue`. *Responsabilidad:* renderizar una tarjeta por película válida.

```
<MovieCard
  v-for="(movie, index) in validMovies"
  :key="movie.id"
  :movie="movie"
  :delay="index * 30"
/>
```

La cantidad de tarjetas y la cantidad de columnas son problemas diferentes. `validMovies` determina cuántas `MovieCard` existen; CSS Grid decide cómo se distribuyen en la pantalla.

Ubicación en el proyecto: `resources/js/cineverse/features/movies/components/MovieResults.vue` · **CSS scoped del componente** · `.movies-grid`

Lo utiliza: `MovieResults.vue`. *Responsabilidad:* definir columnas responsivas de la grilla.

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 32px;
}

> 1024 px: 4 columnas
<= 1024 px: 3 columnas
<= 768 px: 2 columnas
<= 480 px: 1 columna
```

MIN_MOVIES vale 20. Si después de descartar películas sin póster quedan menos de 20 y existe otra página, MovieResults puede emitir load-more automáticamente antes de que Carlos pulse Mostrar más.

17.1.1 El JSON puede haber llegado aunque el póster todavía esté cargando

Cada `MovieCard` tiene un segundo proceso visual. El objeto `Movie` ya puede existir en `validMovies` mientras el navegador todavía descarga la imagen desde `image.tmdb.org`. Por eso la tarjeta muestra un `skeleton` hasta recibir el evento `load` del `<img>`.

Ubicación en el proyecto: `resources/js/cineverse/features/movies/components/MovieCard.vue` · **Component Vue** · template + `onPosterLoad()`

Lo utiliza: `MovieResults.vue`. *Responsabilidad:* mantener `skeleton` hasta que el navegador termine de cargar el póster.

```
<div v-if="!imageLoaded" class="poster-skeleton"></div>



function onPosterLoad() {
  imageLoaded.value = true;
}

```

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

datos de TMDB ya disponibles
-> MovieCard se monta
-> poster-skeleton visible
-> navegador solicita el póster con loading="lazy"
-> @load
-> imageLoaded=true
-> desaparece el skeleton
-> aparece el póster

```

17.2 Ejemplo guiado: Carlos pasa el mouse sobre una película



Figura. MovieCard durante hover: título, año, sinopsis y botón Ver detalles.

La descripción no provoca otra llamada a TMDB. MovieCard ya recibió el objeto movie desde la grilla y ese objeto contiene title, release_date, overview y vote_average. El hover sólo revela información que ya estaba en memoria.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Carlos coloca el cursor sobre La Odisea
-> MovieCard ya posee movie.overview
-> CSS activa .movie-card:hover .poster-overlay
-> overlay pasa de opacity 0 a visible
-> Vue muestra title / year / overview
-> no se envía una nueva request HTTP

```

MovieResults además escucha mousemove sobre la grilla. handleGridMouseMove() identifica la .movie-card bajo el cursor, calcula la posición relativa dentro de la tarjeta y aplica rotateX/rotateY y el glare. requestAnimationFrame agrupa la actualización con el ritmo de pintado del navegador.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

mousemove en .movies-grid
-> handleGridMouseMove(event)
-> closest(".movie-card")
-> getBoundingClientRect()
-> posición del cursor respecto del centro
-> rotateX / rotateY / glare
-> requestAnimationFrame
-> efecto 3D visible

```

Al pulsar Ver detalles no existe un @click específico en ese botón. El clic se propaga al contenedor .movie-card, cuyo @click ejecuta goToMovie(movie.id). El corazón usa @click.stop precisamente para evitar esa navegación cuando Carlos sólo quiere guardar un favorito.

17.3 Ejemplo guiado: Carlos busca Batman desde el encabezado

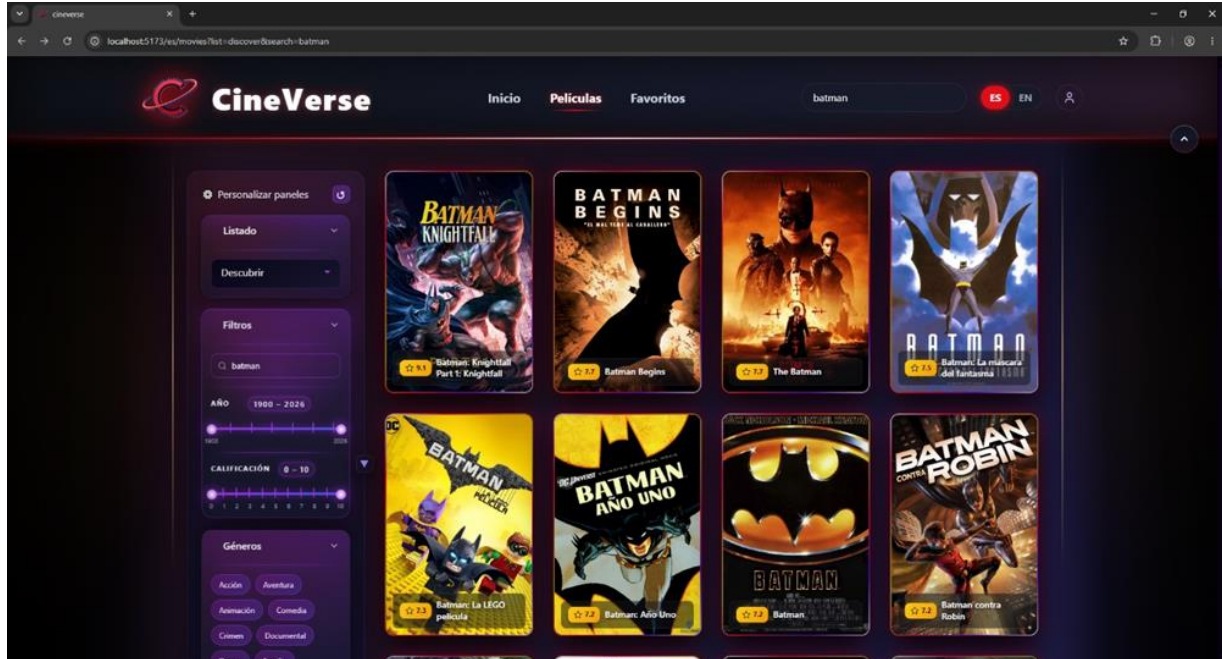


Figura. Búsqueda de Batman desde el encabezado y resultados en la grilla.

El buscador superior y el buscador del panel lateral representan el mismo criterio, pero llegan a él por caminos distintos. En AppHeader, Carlos escribe Batman y pulsa Enter; el formulario ejecuta handleSearch(), que valida un mínimo de tres caracteres y navega hacia Movies con query parameters.

Ubicación en el proyecto: resources/js/cineverse/layouts/components/AppHeader.vue · **Component Vue** · handleSearch()

Lo utiliza: buscador superior del header. *Responsabilidad:* validar el texto y navegar al explorador con query params.

```
function handleSearch() {
  const query = searchQuery.value.trim();
  if (!query || query.length < 3) return;

  router.push({
    name: "Movies",
    params: { lang: route.params.lang },
    query: {
      search: query,
      list: "discover",
      sort: "vote_average.desc"
    },
  });
}
```

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos escribe Batman arriba
-> AppHeader.handleSearch()
-> router.push(...)
-> URL contiene search=Batman
-> useMoviesFilters lee route.query
-> txtMovie / debouncedQuery toman ese valor
```

```

-> useMoviesQuery detecta texto
-> searchByTextAndFilters()
-> GET /search/movie
-> TMDB
-> MovieResults reemplaza la grilla

```

En este recorrido la URL actualiza el estado del panel lateral. El input del encabezado funciona de manera distinta: `searchQuery` toma `route.query.search` sólo cuando se crea `AppHeader` mediante `getInitialSearchQuery()`, y el componente no tiene un `watch` de `route.query` que vuelva a escribir ese ref. Por eso, si después Carlos cambia el texto desde el panel lateral, el campo superior puede conservar el valor anterior durante la misma sesión de la SPA.

Cuando el término llega confirmado desde la URL, el panel lateral puede reconstruir el mismo estado. No son dos consultas independientes: `Vue Router` y `useMoviesFilters` permiten que ambas interfaces representen el mismo criterio de búsqueda.

17.4 Camino 1: listas predefinidas

Si no hay texto escrito y el tipo elegido es `popular`, `now_playing`, `upcoming` o `top_rated`, CineVerse consulta directamente `/movie/{listType}`. TMDB decide qué películas forman la página y en qué orden llegan. En `upcoming`, CineVerse elimina además resultados cuya fecha de estreno ya haya pasado.

17.5 Camino 2: Discover

Si el usuario elige `Discover` y no escribe un texto, CineVerse convierte los filtros de la interfaz en parámetros de `/discover/movie`. En este camino TMDB recibe género, fechas, puntuación y orden, por lo que filtra los datos antes de devolver la página al navegador.

17.6 Camino 3: búsqueda por texto más filtros

Si hay un texto de búsqueda confirmado, CineVerse utiliza `/search/movie`. TMDB recibe el texto, la página y el idioma. Después, el navegador aplica sobre los resultados recibidos los filtros adicionales de género, año, puntuación y orden. Es importante distinguirlo de `Discover`: en este camino esos filtros no se ejecutan en TMDB.

17.7 Ejemplo guiado: Carlos busca Batman desde el panel lateral

```

Texto confirmado: "Batman"
↓
GET /search/movie?query=Batman&page=1&language=es
↓
TMDB devuelve la página 1
↓
CineVerse aplica, si existen:
- género
- año
- rating
- orden
↓
MovieResults muestra la página ya filtrada

```

Limitación esencial

Texto + filtros se aplica sobre cada página recibida, no sobre el universo completo de coincidencias de TMDB. `total_results` se reemplaza por `results.length` de esa página, mientras `total_pages` conserva la paginación original de TMDB.

17.7.1 Recorrido completo de la búsqueda "Batman"

El ejemplo anterior muestra la URL final. El recorrido real dentro del frontend es más largo y permite ver qué archivo participa en cada paso:

1. El usuario escribe Batman en el campo de búsqueda. `useMoviesFilters` guarda el texto en `txtMovie`.
2. El watcher de búsqueda espera 500 milisegundos. Si el texto limpio tiene al menos tres caracteres, copia Batman a `debouncedQuery`.
3. El cambio de `debouncedQuery` modifica `queryKey`. `useMoviesQuery`, construido con `useInfiniteQuery` de Vue Query, vuelve a ejecutar su `queryFn`.
4. Como existe texto confirmado, `queryFn` elige `searchByTextAndFilters()` y le entrega la página, el idioma activo y los filtros.
5. `searchByTextAndFilters()` llama primero a `searchByText()`, que ejecuta `api.get("/search/movie", { params: { query, page, language } })`.
6. `tmdbApi.ts` completa la dirección con `VITE_TMDB_BASE_URL` y agrega el Bearer token. TMDB recibe la solicitud HTTP.
7. TMDB responde con una página de resultados. Axios devuelve data con la forma `PaginatedResponse<Movie>`.
8. Si hay género, año, puntuación u orden, `searchByTextAndFilters()` los aplica en el navegador sobre results de esa página. Después reemplaza `total_results` por `results.length`, pero conserva `total_pages` informado originalmente por TMDB.
9. Vue Query conserva las páginas obtenidas y expone data, estados de carga, error y `fetchNextPage`.
10. `MoviesView` pasa esos valores a `MovieResults`. `MovieResults` une las páginas cargadas y muestra las películas en la grilla.

```
Batman
-> useMoviesFilters
-> debounce 500 ms
-> debouncedQuery
-> useMoviesQuery
-> searchByTextAndFilters
-> searchByText
-> tmdbApi / Axios
-> TMDB
-> JSON PaginatedResponse<Movie>
-> filtros locales
-> Vue Query
-> MoviesView
-> MovieResults
```

El idioma también participa en este recorrido. `MoviesView` pasa locale como `apiLanguage` y `useMoviesQuery` añade `language.value` a `queryKey`. Cambiar ES por EN provoca una nueva consulta con `language=en`; no se limita a traducir los botones de la pantalla.

17.8 Cómo se interpretan varios géneros seleccionados

La misma selección de géneros no se interpreta exactamente igual en los dos caminos. En Discover se envía `with_genres=28,12` a TMDB. En la búsqueda por texto, el código revisa `genre_ids` con `some(...)`, por lo que una película se conserva si coincide con al menos uno de los géneros elegidos. Esta diferencia forma parte del comportamiento actual del proyecto.

18. Cómo se guardan los filtros y se reflejan en la URL

Los filtros no empiezan en una URL ni en `sessionStorage`: empiezan cuando Carlos cambia una opción visible en el explorador. `MoviesView` crea un único objeto de filtros con `useMoviesFilters()`, lo entrega a `MovieFilters` para que la interfaz pueda modificarlo y, al mismo tiempo, se lo entrega a `useMoviesQuery` para que los cambios terminen afectando la consulta de películas.

18.1 Qué piezas intervienen y dónde están

Ubicación principal: `resources/js/cineverse/features/movies/composables/useMoviesFilters.ts` · **Composable** · función exportada `useMoviesFilters()`

`MoviesView.vue` crea este composable una sola vez y pasa el objeto resultante como prop a `MovieFilters.vue`. El mismo objeto también se entrega a `useMoviesQuery()`. Por eso el panel visual, la URL y la consulta no mantienen tres copias independientes: comparten el mismo estado reactivo.

18.2 Ejemplo guiado: Carlos selecciona filtros y la URL cambia

Carlos entra a Películas, elige Discover, selecciona Acción y Aventura, fija año desde 2000 y una puntuación mínima de 7. `MovieFilters` modifica refs que pertenecen al objeto creado por `useMoviesFilters()`; un watcher espera 300 ms, escribe sólo los parámetros necesarios y usa `router.replace()` para actualizar la URL sin llenar el historial del navegador con cada movimiento intermedio.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos modifica MovieFilters.vue
-> props.movieFilters cambia refs de useMoviesFilters()
-> watcher de useMoviesFilters.ts espera 300 ms
-> writeStoredQuery(query) guarda moviesFilters:lastQuery en sessionStorage
-> router.replace({ query }) actualiza la URL
-> filters.queryKey cambia
-> useMoviesQuery.ts vuelve a ejecutar la consulta
-> MovieResults recibe la nueva data
-> Carlos ve la grilla correspondiente a sus filtros
```

18.3 Datos que controla el sistema de filtros

- Texto escrito y `debouncedQuery` confirmado.
- Tipo de catálogo.
- IDs de géneros seleccionados.
- Año desde / hasta.
- Rango de rating [0,10].
- Ordenamiento.

18.4 Espera antes de lanzar una búsqueda (debounce)

Para no enviar una solicitud por cada tecla, la búsqueda espera 500 milisegundos desde el último cambio. Esta técnica se conoce como `debounce`. Además, el texto debe tener al menos tres caracteres: escribir “ba” no consulta TMDB; escribir “bat” y dejar de teclear durante medio segundo sí.

18.5 Cómo se escriben los filtros en la URL

Los cambios de filtros también esperan 300 milisegundos antes de actualizar la URL. Se utiliza `router.replace` en lugar de `router.push` para no crear una nueva entrada del historial por cada movimiento intermedio de un control deslizante (slider).

El ejemplo usa el orden por puntuación como valor predeterminado. Como `vote_average.desc` ya representa ese orden, el código que convierte los filtros en parámetros de la URL no necesita escribir `sort` mientras el usuario no cambie esa opción.

```
/es/movies
?list=discover
&genres=28,12
&yearFrom=2000
&yearTo=2020
&ratingMin=7
```

18.6 Qué información tiene prioridad al recuperar los filtros

URL con parámetros de consulta

→ la URL es la fuente de verdad

URL limpia al volver a Películas

→ intenta recuperar `sessionStorage["moviesFilters:lastQuery"]`

Sin URL ni datos guardados

→ utiliza valores por defecto

Si la URL ya trae filtros, esos valores tienen prioridad. Si la URL está limpia, CineVerse intenta recuperar `sessionStorage["moviesFilters:lastQuery"]`. Si tampoco existe ese dato, utiliza los valores por defecto. Cuando recupera filtros desde `sessionStorage`, los vuelve a escribir en la URL para que la dirección pueda copiarse, compartirse o guardarse.

18.7 Paneles personalizables

El panel de filtros permite abrir o cerrar secciones y cambiar su orden mediante arrastrar y soltar. Parte de esa disposición se guarda en el navegador para recuperarla en visitas posteriores. Durante el modo de personalización, los paneles se cierran temporalmente para facilitar el movimiento.

Casos	límite	actuales
El listener de <code>resize</code> se registra con una función anónima y se intenta retirar con otra referencia, por lo que no se elimina realmente al desmontar el componente. También queda un listener de <code>scroll</code> sin retirar y un <code>scrollInterval</code> puede permanecer activo después de abandonar la vista. Si se abandona la vista durante el modo de personalización, el estado temporal de paneles cerrados puede llegar a persistirse.		

18.8 Ejemplo completo: filtros -> URL -> consulta -> regreso

El estado de los filtros se sincroniza en los dos sentidos. El composable no sólo escribe la URL: también puede reconstruir su estado desde una URL compartida, desde los botones Atrás/Adelante o desde `sessionStorage` cuando se vuelve a Películas con una dirección limpia.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Usuario elige:
  Discover
  géneros 28 y 12
  año desde 2000
  rating mínimo 7

useMoviesFilters
  -> espera 300 ms
  -> router.replace(...)

URL:
/es/movies?list=discover&genres=28,12&yearFrom=2000&ratingMin=7

queryKey cambia
  -> useMoviesQuery
  -> searchByFilters(...)
  -> GET /discover/movie con los filtros
  -> TMDB responde
  -> MovieResults muestra la página
  
```

Si después se copia esa URL y se abre directamente, useMoviesFilters lee route.query al iniciar, convierte géneros a números, recupera yearFrom/ratingMin y crea el mismo estado reactivo. Si la URL no trae parámetros, intenta recuperar moviesFilters:lastQuery desde sessionStorage.

syncingFromRoute y la comparación de arrays evitan un bucle lectura -> escritura -> lectura y evitan resetear la grilla cuando un array nuevo contiene exactamente los mismos valores que el anterior.

19. Cómo se cargan más películas

La carga incremental se entiende mejor desde la interfaz: Carlos recibe una primera grilla, pulsa “Mostrar más” y, desde ese momento, CineVerse puede continuar pidiendo páginas cuando el usuario se acerca al final. La paginación se reparte entre useMoviesQuery(), MoviesView y MovieResults; ninguna de esas piezas hace por sí sola todo el trabajo.

CineVerse utiliza useInfiniteQuery<PaginatedResponse<Movie>> para conservar y gestionar las páginas de resultados que ya fueron obtenidas desde TMDB. La función getNextPageParam determina cuál debe ser la siguiente página solicitada comparando la página actual con la cantidad total de páginas informada por la API. Cuando ya no existen más páginas disponibles, devuelve undefined y finaliza la carga incremental.

19.1 Qué piezas participan

Ubicación: resources/js/cineverse/features/movies/composables/useMoviesQuery.ts · Composable · useMoviesQuery(). Decide qué endpoint de TMDB utilizar y, mediante useInfiniteQuery(), conserva las páginas recibidas y calcula la siguiente con getNextPageParam().

Ubicación: resources/js/cineverse/features/movies/views/MoviesView.vue · View. Recibe fetchNextPage y hasNextPage desde useMoviesQuery(); entrega esos valores a MovieResults y conecta @load-more="fetchNextPage".

Ubicación: resources/js/cineverse/features/movies/components/MovieResults.vue · Component. Une data.pages, descarta resultados sin póster utilizable, muestra el botón “Mostrar más”, controla autoScrollEnabled e instala el IntersectionObserver.

19.2 Ejemplo guiado: Carlos pulsa “Mostrar más”

Carlos ya ve la primera página. Al pulsar el botón, MovieResults.enableScroll() activa el modo de carga automática y emite load-more. MoviesView escucha ese evento y ejecuta fetchNextPage(), función de TanStack Vue Query. useMoviesQuery calcula page=2 a partir de la última respuesta y vuelve a recorrer el mismo camino de búsqueda que se utilizó para la página 1.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa "Mostrar más"
-> MovieResults.vue :: enableScroll()
-> autoScrollEnabled = true
-> emit("load-more")
-> MoviesView.vue :: @load-more="fetchNextPage"
-> useInfiniteQuery ejecuta getNextPageParam(lastPage)
-> pageParam = 2
-> queryFn usa el mismo camino activo: lista / texto / Discover
-> TMDB devuelve página 2
-> data.pages conserva página 1 + página 2
-> MovieResults hace flatMap y actualiza validMovies
-> Carlos ve nuevas tarjetas debajo de las anteriores
```

19.3 De dónde sale la página siguiente

La decisión no se toma en MovieResults. Está en useMoviesQuery.ts: getNextPageParam compara lastPage.page con lastPage.total_pages. Si todavía quedan páginas devuelve page + 1; cuando llega a la última devuelve undefined y TanStack Vue Query deja de ofrecer una página siguiente.

Ubicación en el proyecto: resources/js/cineverse/features/movies/composables/useMoviesQuery.ts · **Composable** · useMoviesQuery() · opción getNextPageParam

Lo utiliza: MoviesView.vue a través de useMoviesQuery(). *Responsabilidad:* decidir el número de la próxima página de TMDB.

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.total_pages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

19.4 Cómo se unen las páginas de resultados

MovieResults une las películas de todas las páginas ya cargadas y las presenta como una lista continua. El usuario no navega manualmente a /page/2: las nuevas películas aparecen debajo de las anteriores.

19.5 Qué ocurre cuando faltan pósters

Algunas tarjetas no se muestran cuando la película no tiene póster. Para evitar una grilla inicial demasiado corta, el flujo puede pedir otra página si después de descartar esas películas quedaron pocos elementos utilizables.

19.6 Botón “Mostrar más” y carga automática al hacer scroll

Primero la vista ofrece el botón “Mostrar más”. Después utiliza IntersectionObserver, una función del navegador que avisa cuando un elemento se aproxima al área visible. Al acercarse al final de la lista, CineVerse puede pedir automáticamente otra página.

Detalle técnico de la paginación con texto y filtros
Como total_pages conserva el valor original de /search/movie, la consulta paginada de Vue Query puede seguir avanzando por las páginas de TMDB aunque alguna página concreta quede con muy pocos resultados después del filtrado local.

19.7 Qué hace MovieResults con las páginas recibidas

MovieResults recibe data.pages desde useInfiniteQuery. Cada elemento de pages es una respuesta paginada de TMDB. El componente las une con flatMap y después descarta películas cuyo poster_path no existe o contiene un valor inválido. El resultado final queda en validMovies, que es la lista que realmente se recorre con v-for.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
pages[0].results + pages[1].results + ...
-> flatMap
-> todas las películas cargadas
-> filtrar poster_path inválido
-> validMovies
-> MovieCard v-for
```

MIN_MOVIES vale 20. Como TMDB pagina antes de que CineVerse descarte películas sin póster, una página podría dejar menos de 20 tarjetas utilizables. Si todavía existe hasNextPage y no hay otra carga en curso, MovieResults emite load-more para completar automáticamente una cantidad inicial razonable.

19.8 Botón Mostrar más e IntersectionObserver

El scroll infinito no se habilita desde el primer momento. autoScrollEnabled comienza en false. El usuario pulsa Mostrar más, enableScroll() lo cambia a true y además pide una página. Desde entonces un IntersectionObserver vigila un elemento invisible situado al final de la grilla.

El observer usa rootMargin: "2500px" para comenzar a cargar antes de que el usuario llegue físicamente al final. loadingMore e isFetchingNextPage impiden varias solicitudes simultáneas. Existe además un setTimeout de 200 ms que puede pedir una página adicional como prefetch si el usuario avanza muy rápido.

19.9 Qué ocurre cuando cambia la búsqueda

MoviesView incrementa resetSignal cuando cambia la identidad de la consulta. MovieResults observa ese número, bloquea temporalmente cargas automáticas, desactiva el scroll infinito, vacía validMovies y vuelve al principio de la página. Después los nuevos datos de Vue Query vuelven a poblar la grilla.

20. Página de inicio

La página de inicio muestra tres grupos de películas: tendencias, cartelera y mejor valoradas. Cada grupo realiza su propia consulta a TMDb, por lo que una fila puede terminar de cargar y mostrarse aunque las otras todavía estén esperando su respuesta.

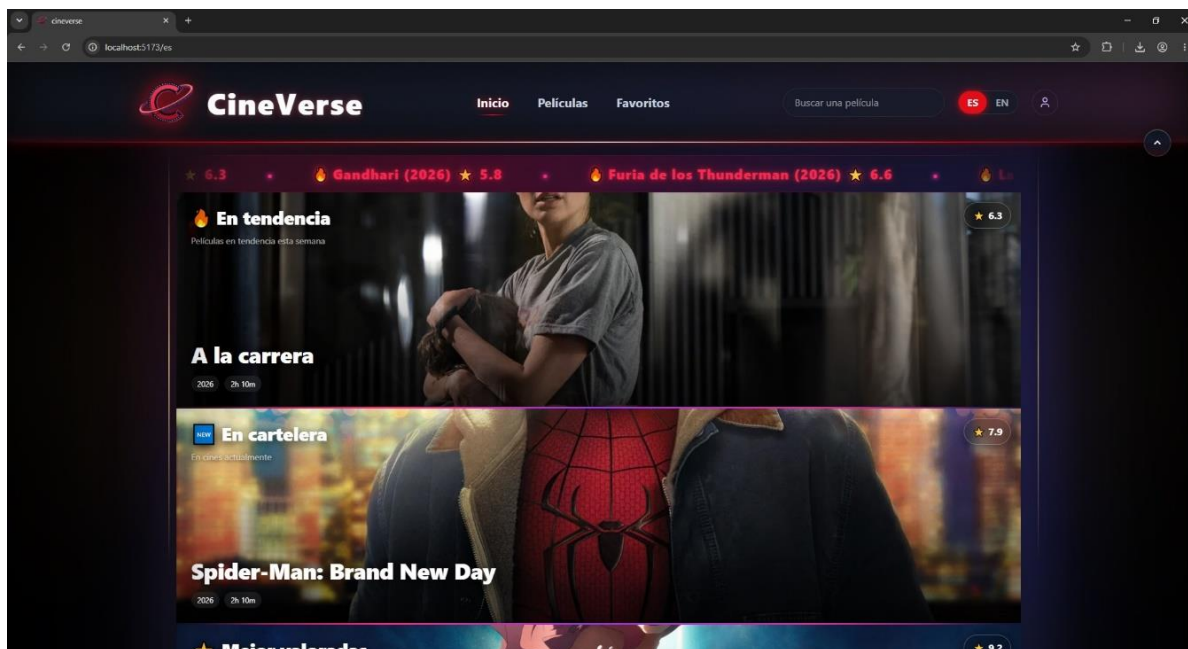


Figura. Página de inicio con tendencias, cartelera y mejor valoradas.

Bloque	Endpoint	Cómo se carga
En tendencia	/trending/movie/day	Consulta independiente.
En cartelera	/movie/now_playing	Consulta independiente.
Mejor valoradas	/movie/top Rated	Consulta independiente.

20.1 Cómo reutiliza datos la página de inicio

Los resultados de Home se consideran recientes durante cinco minutos. Durante ese tiempo Vue Query puede reutilizarlos sin tratarlos como datos vencidos. Al cambiar de idioma, keepPreviousData permite conservar temporalmente la versión anterior mientras llega la nueva respuesta, evitando que la fila quede vacía.

20.2 Duración visible en tarjetas

Estado actual

MovieCardCarousel muestra "2h 10m" como duración fija. La duración real sólo se obtiene en el detalle de cada película mediante /movie/{id}; las tarjetas de Home no realizan esa consulta adicional.

20.3 Estado visual de carga

HomeRow recibe una propiedad loading, pero actualmente no la utiliza directamente. La experiencia de carga depende de sessionStorage["home-loaded"], showOverlay, isReady y el evento ready del carrusel. Por eso una respuesta vacía o un error de TMDB no siempre tiene un mensaje de error específico para cada fila.

20.4 Recorrido completo de la Home

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

HomeView se monta
-> useHomeMovies(locale)
-> useQuery trending
-> useQuery topRated
-> useQuery nowPlaying
-> las 3 solicitudes salen de forma independiente
-> movies.service.ts
-> tmdbApi.ts / Axios
-> TMDB
-> cada query actualiza su propia data
-> HomeView toma hasta 10 películas por grupo
-> HomeRow -> HomeCarousel -> MovieCardCarousel
  
```

Ninguna de las tres filas espera a las otras. Si tendencias llega primero, esa información ya puede alimentar el ticker y su fila mientras las demás continúan cargando. La queryKey de cada grupo incluye el idioma; al cambiar ES/EN se solicita la versión localizada correspondiente.

HomeRow no usa directamente la prop loading para decidir todo el estado visual. También consulta sessionStorage["home-loaded"], espera el evento ready del carrusel y oculta el overlay después de un pequeño delay. HomeCarousel sólo emite ready cuando Swiper fue inicializado y la primera imagen visible terminó de cargar y pintar.

El ticker se construye con las primeras cinco películas de tendencias: título, año y vote_average. Luego agrega tres frases traducidas y duplica el array para crear una cinta continua.

20.5 Ejemplo guiado: Carlos entra a Inicio

Carlos entra a /es. HomeView ejecuta useHomeMovies(locale) y crea tres consultas independientes: tendencias, mejor valoradas y cartelera. Ninguna necesita esperar a las otras para comenzar.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Carlos entra a Inicio
-> HomeView
-> useHomeMovies(locale)
-> useQuery trending
-> useQuery topRated
-> useQuery nowPlaying
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> cada query actualiza su propia data
-> HomeRow
-> HomeCarousel
-> MovieCardCarousel
  
```

El loader visual tampoco depende únicamente de que haya llegado JSON. HomeCarousel emite ready cuando Swiper está inicializado y la primera imagen visible terminó de cargar y pintar. HomeRow utiliza ese evento junto con showOverlay y sessionStorage["home-loaded"] para retirar el estado inicial.

Resultado práctico: una fila puede estar lista mientras otra todavía carga, porque las tres consultas son independientes.

21. Detalle de una película

La pantalla de detalle necesita datos de varios endpoints. Primero consulta `/movie/{id}` para confirmar que la película existe. Sólo después solicita créditos, videos y películas similares. Esas tres solicitudes se lanzan juntas con `Promise.all`. `Promise.all` espera varias operaciones asíncronas al mismo tiempo, de modo que CineVerse no necesita terminar una consulta antes de comenzar la siguiente.

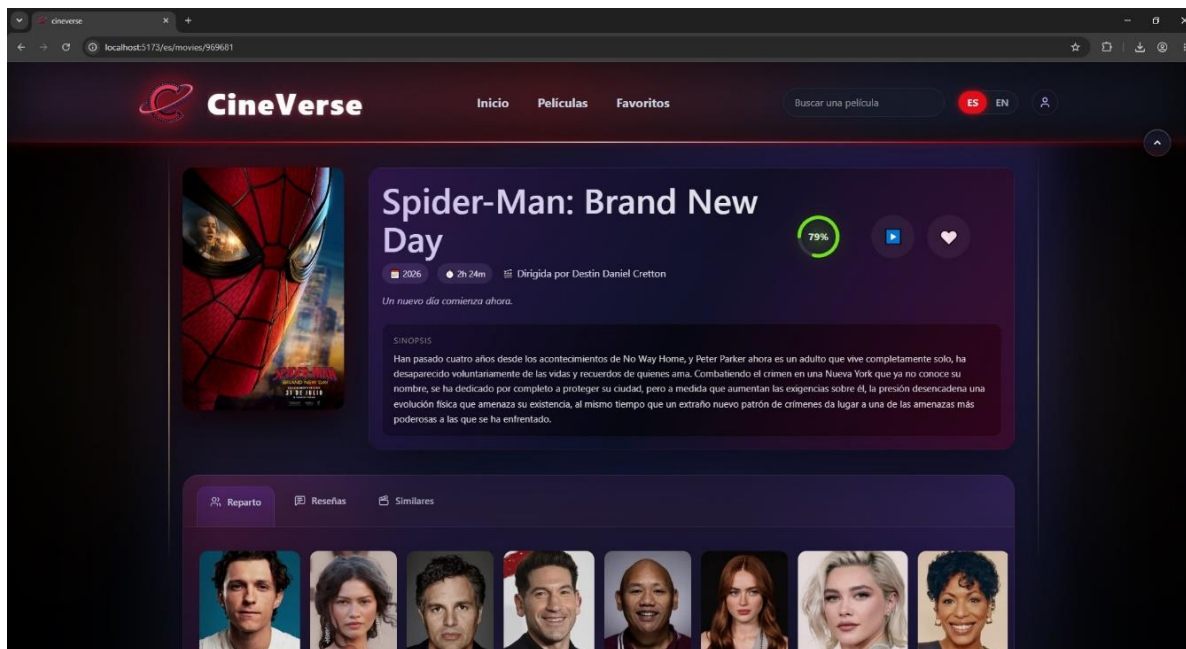


Figura. Detalle de película con ficha principal, trailer y reparto.

Coordinación de solicitudes

Ubicación en el proyecto: `resources/js/cineverse/features/movies/composables/movie-detail/useMovieDetail.ts` · Composable · `queryFn` de `useMovieDetail()`. Lo utiliza `MovieDetailView.vue` para coordinar detalle, créditos, videos y similares.

```
const movieData = await getMovieDetail(id, language.value);

const [creditsData, videosData, similarData] = await Promise.all([
  getMovieCredits(id, language.value),
  getMovieVideos(id, language.value),
  getSimilarMovies(id, 1)
]);
```

21.1 Cómo se preparan los datos para mostrarlos

`useMovieDetail` transforma las respuestas de TMDb en una estructura más cómoda para la pantalla. Limita el reparto a diez integrantes, busca dentro del equipo técnico (crew) el registro cuyo campo `job` vale "Director", selecciona un trailer de YouTube y devuelve las películas similares. Esta preparación evita repetir la misma lógica dentro de varios componentes visuales.

Propiedad final	Origen
movie	<code>/movie/{id}</code>
cast	<code>credits.cast.slice(0,10)</code>
director	Primer registro del equipo técnico (crew) cuyo campo <code>job</code> es Director

Propiedad final	Origen
trailer	Primer video cuyo campo type es Trailer y cuyo campo site es YouTube
similar	results de /similar

21.2 Cuándo se realiza la consulta y cómo se tratan los errores

La consulta se ejecuta únicamente cuando el id es un número válido mayor que cero. `retry=false` evita reintentos automáticos. La limitación actual es visual: distintos fallos de la consulta pueden terminar mostrando una experiencia de “no encontrado”, por lo que un id inexistente y una caída temporal de TMDb no siempre se distinguen claramente.

Detalle de código
`useMovieDetail` contiene una expresión suelta ``trailer: videosData.results[0]?.key ?? null`` antes de construir el objeto final. Esa expresión no asigna el valor a ninguna variable ni modifica el objeto devuelto; el trailer efectivo se obtiene después mediante `find(...)`.

21.3 Recorrido completo al abrir /es/movies/550

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Vue Router
-> MovieDetailView obtiene id=550
-> useMovieDetail(550, locale)
-> GET /movie/550
-> si existe:
    Promise.all(
      GET /movie/550/credits,
      GET /movie/550/videos,
      GET /movie/550/similar
    )
-> limitar cast a 10
-> buscar crew.job === "Director"
-> buscar primer video Trailer + YouTube
-> MovieDetailResult
-> MovieHero + MovieTabs
```

La consulta principal se hace primero porque permite comprobar que la película existe antes de lanzar las tres solicitudes secundarias. `Promise.all` hace que créditos, videos y similares se esperen en conjunto en vez de encadenarlos uno detrás de otro.

`MovieDetailView` mantiene `heroReady` separado de `isLoading`. El loader no desaparece simplemente cuando llegó el JSON: `MovieHero` emite `ready` cuando el póster ya está cargado y su entrada visual puede mostrarse sin un salto brusco.

21.3.1 Ejemplo guiado: Carlos abre Fight Club y espera el hero

Carlos pulsa una `MovieCard` con id 550. `Vue Router` cambia la URL y `MovieDetailView` obtiene el id. `useMovieDetail` primero consulta `/movie/550`. Sólo si ese detalle existe lanza créditos, videos y similares en paralelo mediante `Promise.all`.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Fight Club
-> MovieCard.goToMovie(550)
-> /es/movies/550
-> MovieDetailView
-> useMovieDetail(550, locale)
```

```
-> GET /movie/550
-> Promise.all(credits, videos, similar)
-> preparar cast / director / trailer / similares
-> MovieHero + MovieTabs
```

MovieDetailView mantiene heroReady separado del estado de la request. El JSON puede haber llegado, pero el loader principal espera además a que MovieHero termine de cargar su imagen y emita ready. Así la ficha no aparece con un salto brusco mientras el póster todavía está entrando.

21.4 Cuándo se cargan Reparto, Reseñas y Similares

MovieTabs comienza con activeTab="cast". Reparto se muestra primero. SimilarMovies recibe los resultados que useMovieDetail ya obtuvo desde TMDB, pero MovieReviews funciona distinto: está detrás de un v-if y su onMounted(loadInitial) se ejecuta cuando el usuario entra por primera vez en la pestaña Reseñas.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
abrir detalle
-> pestaña activa: Reparto
-> todavía no se ejecuta MovieReviews.onMounted()

usuario pulsa Reseñas
-> Vue crea MovieReviews
-> onMounted(loadInitial)
-> GET /movies/550/reviews?page=1
-> Laravel / ReviewController / MySQL
```

MovieTabs envuelve los tres componentes en KeepAlive. Después de haber creado una pestaña, Vue puede conservar su instancia al alternar entre Reparto, Reseñas y Similares en lugar de destruirla inmediatamente. Esto conserva, por ejemplo, el estado ya cargado de MovieReviews durante esos cambios.

22. Trailers, películas similares e imágenes

La página de detalle no muestra solamente los datos principales de una película. También utiliza información adicional de TMDB para reproducir trailers, proponer títulos similares y mostrar imágenes de respaldo cuando faltan recursos. Esta sección sigue esos datos desde la respuesta de TMDB hasta lo que termina viendo el usuario.

22.1 Selección del trailer

CineVerse recorre los videos devueltos por TMDB y toma el primer elemento cuyo type sea Trailer y cuyo site sea YouTube. No comprueba la propiedad official, por lo que la descripción correcta es “primer trailer de YouTube encontrado” y no “trailer oficial”.

Ubicación en el proyecto: resources/js/cineverse/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · selección del trailer dentro de queryFn. El resultado se entrega a MovieDetailView.vue y luego a MovieHero.vue.

```
const trailerData = videosData.results.find(
  (v) => v.type === "Trailer" && v.site === "YouTube"
);

trailer: trailerData ? trailerData.key : null
```

La propiedad key identifica el video dentro de YouTube. CineVerse la utiliza para construir el reproductor embebido dentro de un modal, de manera que el trailer pueda verse sin abandonar la página de detalle.

22.1.1 Ejemplo guiado: Carlos pulsa Ver trailer

useMovieDetail ya examinó videosData.results y guardó la key del primer video cuyo type es Trailer y site es YouTube. Cuando Carlos pulsa el botón del hero, MovieHero no crea el iframe por sí mismo: emite openTrailer y MovieDetailView controla el modal.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Ver trailer
-> MovieHero emite openTrailer
-> MovieDetailView pone isTrailerOpen=true
-> Vue monta el modal
-> iframe src = https://www.youtube.com/embed/{key}?autoplay=1
-> watch bloquea el scroll del body

Cerrar / Escape
-> isTrailerOpen=false
-> iframe se desmonta
-> la reproducción se detiene
```

La key viene de TMDb; el video se reproduce desde YouTube. CineVerse no descarga ni almacena el archivo de video.

22.2 Similares

getSimilarMovies consulta /movie/{id}/similar. En el código actual fija language="en-US" sin utilizar el idioma activo de la interfaz, por lo que algunas películas similares pueden mostrar título o sinopsis en inglés mientras el resto de CineVerse está en español.

22.3 Imágenes de respaldo cuando TMDb no proporciona una

CineVerse incluye no-image-poster.jpg y no-image-avatar.jpg para mostrar una imagen alternativa cuando TMDb no proporciona póster o fotografía. El manejo no es totalmente uniforme: algunos componentes utilizan estos archivos de forma explícita y otros construyen directamente la URL de image.tmdb.org.

22.4 Tamaños de imagen utilizados

Contexto	Tamaño observado / intención
Tarjetas de película	w500
Backdrop de Home	w1280
Reparto	w185
Similares	w300
Hero de detalle	original
Actor	w500

22.5 Desde la key de TMDb hasta el reproductor de YouTube

TMDb no devuelve el video completo. En results devuelve una key. useMovieDetail conserva la key del primer video cuyo type sea Trailer y cuyo site sea YouTube. Cuando el usuario pulsa el botón de trailer, MovieHero emite openTrailer y MovieDetailView abre el modal.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
trailer key = "abc123"
-> MovieDetailView isTrailerOpen = true
-> iframe src =
  https://www.youtube.com/embed/abc123?autoplay=1
```

Cerrar el modal cambia isTrailerOpen a false; al desmontarse el iframe se detiene la reproducción. También puede cerrarse con Escape. Mientras está abierto, un watch cambia document.body.style.overflow a hidden para impedir que la página del fondo siga desplazándose.

22.6 Cómo se construye una URL de imagen

TMDB devuelve rutas como /abc123.jpg. Los componentes anteponen https://image.tmdb.org/t/p/ y el tamaño que necesitan. Por ejemplo, una tarjeta puede usar w500 mientras el hero utiliza original. Si falta poster_path o profile_path, ciertos componentes muestran archivos locales como no-image-poster.jpg o no-image-avatar.jpg.

No todos los componentes aplican exactamente el mismo mecanismo de respaldo. MovieResults descarta películas sin póster antes de construir la grilla, mientras MovieHero sí puede mostrar no-image-poster.jpg. Esta diferencia forma parte del comportamiento actual.

23. Actores y filmografía

Cuando el usuario selecciona un actor desde el reparto, PersonDetailView utiliza usePersonDetail y person.service.ts. TMDB llama person a este tipo de recurso dentro de su API. CineVerse pide los datos del actor y su filmografía en una sola solicitud utilizando append_to_response=movie_credits, un parámetro de TMDB que permite incluir los créditos de películas en la misma respuesta.

Ubicación en el proyecto: resources/js/cineverse/features/person/services/person.service.ts · Service del frontend · getFullPersonDetail(). Lo utiliza usePersonDetail.ts para obtener la persona y movie_credits desde TMDB.

```
const { data } = await api.get<PersonDetail>(`
  /person/${personId}`,
  {
    params: {
      language,
      append_to_response: "movie_credits"
    }
  }
);
```

23.1 Componentes

Componente	Responsabilidad
PersonHero	Muestra los datos principales del actor, su fotografía y biografía.
PersonKnownFor	Selecciona películas destacadas a partir de sus créditos.
PersonFilmography	Organiza la filmografía para mostrarla en la vista.

La clave de Vue Query incluye el id del actor y el idioma, de modo que cada combinación tenga su propio resultado. La consulta no se reintenta automáticamente cuando falla y tampoco vuelve a ejecutarse sólo porque el usuario regrese a la pestaña. Si TMDB no proporciona profile_path, la interfaz muestra el avatar local de respaldo.

23.2 Recorrido completo de un actor

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Usuario pulsa un actor del reparto
-> router.push(name="Person", id)
-> PersonDetailView obtiene el id
-> usePersonDetail(id, locale)
-> person.service.ts
-> GET /person/{id}
    language=<idioma>
    append_to_response=movie_credits
-> TMDB responde PersonDetail + movie_credits
-> Vue Query guarda el resultado
-> PersonHero
    -> PersonKnownFor
    -> PersonFilmography

```

`append_to_response=movie_credits` evita una segunda solicitud separada para la filmografía: TMDB incorpora `movie_credits` dentro de la misma respuesta del actor.

23.3 Cómo se construye "Conocido por"

`PersonKnownFor` clona los créditos para no modificar la prop original, los ordena por `popularity` de mayor a menor y toma los ocho primeros. Al pulsar una película usa Vue Router para abrir `MovieDetail` manteniendo el idioma actual.

23.4 Cómo se organiza la filmografía

`PersonFilmography` descarta créditos sin `release_date`, ordena los restantes por fecha de estreno desde la más reciente hasta la más antigua y después los agrupa por año. El resultado se transforma en una línea temporal por años. La foto principal usa `w500` y, si `profile_path` falta o es inválido, `PersonHero` utiliza el avatar local.

23.5 Ejemplo guiado: Carlos abre un actor y cambia el idioma

Carlos pulsa una persona del reparto. Vue Router conserva `lang` y navega a `/:lang/person/:id`. `PersonDetailView` obtiene el id, `usePersonDetail` crea la consulta y `person.service.ts` usa `append_to_response=movie_credits` para pedir datos principales y filmografía en la misma respuesta.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Carlos pulsa un actor
-> router.push(name="Person", id)
-> PersonDetailView
-> usePersonDetail(id, locale)
-> person.service.ts
-> GET /person/{id}?language=<locale>&append_to_response=movie_credits
-> TMDB
-> Vue Query
-> PersonHero + PersonKnownFor + PersonFilmography

```

Si Carlos cambia ES por EN, `language` forma parte de la `queryKey`. Vue Query trata el nuevo idioma como otro resultado y `person.service.ts` vuelve a consultar TMDB; los botones cambian por `vue-i18n` y la biografía externa cambia porque TMDB recibió otro `language`.

24. Favoritos

Los favoritos se guardan únicamente en el navegador. No requieren una cuenta, no se envían al backend y no crean registros en MySQL. El store de Pinia conserva objetos `Movie` y `pinia-plugin-persistedstate` los guarda en `localStorage`.

Regla	Implementación
Sin duplicados	<code>isFavorite()</code> y <code>addFavorite()</code> comparan <code>movie.id</code> .
Máximo	<code>MAX_FAVORITES = 20</code> .
Eliminar	<code>removeFavorite(id)</code> .

Regla	Implementación
Vaciar	clearFavorites().
Persistencia	persist: true en el store.

Implicación práctica

Los favoritos pertenecen al navegador actual, no a la cuenta de CineVerse. Si el usuario cambia de navegador, utiliza otro perfil o borra el almacenamiento local, la lista puede desaparecer. Las reseñas no se ven afectadas porque sí están guardadas en MySQL y asociadas a la cuenta.

24.1 Qué es Pinia en este flujo

Pinia es el sistema de estado compartido del frontend. Un estado local dentro de MovieCard sólo existiría en esa tarjeta; un store de Pinia puede ser consultado por MovieCard, MovieHero, AppHeader o MoviesFavoritesView y todos ven la misma lista de favoritos.

app.ts crea Pinia una sola vez, registra pinia-plugin-persistedstate y después instala Pinia en la aplicación Vue. Por eso useFavoritesStore() no crea una lista independiente por cada componente: devuelve acceso al store identificado como "favorites".

24.2 Cómo está construido useFavoritesStore

Ubicación en el proyecto: resources/js/cineverse/features/movies/stores/useMoviesFavoritesStore.ts · **Store** Pinia · useMoviesFavoritesStore

Lo utiliza: MovieCard.vue, MovieHero.vue, AppHeader.vue y MoviesFavoritesView.vue. Responsabilidad: mantener y persistir favoritos locales.

```
const favorites = ref<Movie[]>([]);

const favoritesCount = computed(() =>
  favorites.value.length
);

isFavorite(id)
addFavorite(movie)
removeFavorite(id)
clearFavorites()

persist: true
```

favorites es el estado principal y contiene objetos Movie completos, no solamente ids. favoritesCount es derivado: se recalcula a partir de favorites.length. isFavorite() usa some() para buscar una película por id. addFavorite() devuelve true si pudo agregarla y false si ya existía o si se alcanzó el máximo de 20.

removeFavorite(id) construye un nuevo array excluyendo ese id. clearFavorites() reemplaza el array por uno vacío. Como favorites es reactivo, cada cambio se refleja inmediatamente en todos los componentes que consumen el store.

24.3 Ejemplo completo: guardar Fight Club como favorito

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

MovieCard recibe un objeto Movie:
{
  id: 550,
  title: "Fight Club",
  poster_path: "/...",
  release_date: "1999-10-15",
  vote_average: ...,
  ...
}

usuario pulsa el corazón
-> onFavoriteClick(movie)
-> isFavorite(550) == false
-> addFavorite(movie)
-> favorites.value.push(movie)
-> devuelve true
-> Vue actualiza el corazón y favoritesCount
-> el plugin de persistencia guarda el nuevo estado en localStorage

```

No se envía POST al backend y no se crea ninguna fila en MySQL. Tampoco se vuelve a pedir el detalle de la película para guardarla: se persiste el objeto Movie que el componente ya tenía disponible.

24.4 Qué ocurre al recargar el navegador

`persist: true` activa `pinia-plugin-persistedstate` para este store. Cuando el estado cambia, el plugin lo serializa en el almacenamiento del navegador. Al crear nuevamente la aplicación después de F5, el plugin restaura el estado persistido y favoritos vuelve a contener las películas guardadas.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

F5 / nueva carga
-> app.ts crea Pinia
-> se registra pinia-plugin-persistedstate
-> useFavoritesStore se instancia
-> se restaura el estado persistido
-> favoritos contiene nuevamente las películas
-> favoritosCount y corazones se recalculan

```

La persistencia pertenece al navegador, no al usuario autenticado. Iniciar sesión con otra cuenta en el mismo perfil del navegador no crea otra lista independiente. Usar otro navegador, otro perfil o borrar `localStorage` puede hacer desaparecer esos favoritos.

24.5 Límite de 20 y comportamiento de los botones

`MAX_FAVORITES` vale 20. `addFavorite()` comprueba primero duplicados y luego `favorites.length >= MAX_FAVORITES`. En `MovieCard` se utiliza el booleano devuelto: si `addFavorite()` devuelve `false` por el límite, la tarjeta muestra temporalmente un tooltip durante 1500 ms.

`MovieHero` también alterna favoritos, pero actualmente no utiliza el valor `false` devuelto por `addFavorite()`. Por eso el límite se respeta igualmente, aunque desde el hero no aparece el mismo aviso visual de límite alcanzado. Es una diferencia actual entre ambas interfaces.

24.6 Eliminar un favorito

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

usuario pulsa el corazón de una película ya guardada
-> isFavorite(id) == true
-> removeFavorite(id)
-> filter(m => m.id !== id)
-> favorites cambia
-> Vue actualiza la interfaz
-> el estado persistido también queda actualizado

```

24.7 Página Favoritos y "Limpiar todo"

MoviesFavoritesView obtiene directamente favoritos y favoritesCount del store. Muestra el contador actual sobre 20 y crea una MovieCard por cada objeto guardado. Si la lista está vacía, en lugar de la grilla muestra un estado vacío con un enlace para volver al explorador.

El botón Limpiar todo no borra inmediatamente. openConfirm() abre un modal, un watch bloquea el scroll del body y confirmClear() llama clearFavorites(). Al vaciarse el store, la grilla desaparece, favoritesCount pasa a 0 y el plugin persiste el estado vacío.

24.8 Consecuencia de guardar el objeto Movie completo

MoviesFavoritesView reutiliza los objetos Movie guardados para construir las tarjetas. La vista no recorre favoritos haciendo una consulta nueva a TMDB por cada película. Esto hace que abrir Favoritos sea inmediato y funcione con los datos persistidos, pero también significa que esos datos son una copia local del momento en que se guardó cada película.

25. Cómo está organizado el backend Laravel

El backend actual ya no usa un Router propio, servicios manuales y PDO directo como columna vertebral. Laravel recibe las peticiones, decide middleware, ejecuta controladores o Form Requests, trabaja con Eloquent y construye una respuesta HTTP. El frontend continúa consumiendo JSON para cuenta, perfil y reseñas, pero ahora esos contratos viven dentro del framework.

25.1 routes/web.php, auth.php y api.php

routes/web.php se ocupa de la navegación y de entregar las páginas mediante Inertia. routes/auth.php contiene las rutas relacionadas con la autenticación del usuario, como registro, inicio y cierre de sesión, recuperación de contraseña y verificación de correo electrónico. routes/api.php contiene los endpoints JSON propios de la aplicación. Separarlos permite reconocer la función de una URL por su propósito: /es/movies/550 corresponde a navegación de la SPA, /login al proceso de autenticación y /api/reviews al acceso a datos mediante la API.

Ruta	Middleware	Controlador/acción	Uso
GET /user	auth	closure en auth.php	Reconstruir sesión y CSRF en Pinia.
POST /login	guest	AuthenticatedSessionController@store	Iniciar sesión.
POST /register	guest	RegisteredUserController@store	Crear cuenta y enviar verificación.
GET /verify-email/{id}/{hash}	signed + throttle	VerifyEmailController	Confirmar correo desde enlace.
POST /forgot-password	guest	PasswordResetLinkController@store	Pedir recuperación.
POST /reset-password	guest	NewPasswordController@store	Guardar nueva contraseña.
GET /api/profile	web + auth	Api\ProfileController@show	Resumen del perfil.
GET /api/reviews	web + auth	ReviewController@index	Mis reseñas.
POST /api/reviews	web + auth	ReviewController@store	Crear reseña.
PUT /api/reviews/{id}	web + auth	ReviewController@update	Editar reseña propia.
DELETE /api/reviews/{id}	web + auth	ReviewController@destroy	Eliminar reseña propia.
GET /api/movies/{movieId}/reviews	web	ReviewController@movieReviews	Reseñas públicas + contexto de usuario si existe.

25.2 bootstrap/app.php

Laravel 12 concentra parte de la configuración de arranque en bootstrap/app.php. withRouting registra web.php, api.php, console.php y la ruta /up, utilizada como comprobación básica del estado de la aplicación (health check). withMiddleware añade HandleInertiaRequests y AddLinkHeadersForPreloadedAssets al grupo web. withExceptions personaliza el reporte y la respuesta cuando MySQL no está disponible.

25.3 Controllers versus Requests

Un Controller coordina una acción HTTP; un Form Request puede encapsular autorización y reglas de validación. LoginRequest es el ejemplo más importante porque no sólo valida email/password: también normaliza el email, aplica RateLimiter, comprueba el hash, exige correo verificado y ejecuta Auth::login. ProfileUpdateRequest concentra reglas del perfil. ReviewController utiliza \$request->validate() directamente porque sus reglas están acotadas a cada operación.

25.4 Eloquent y respuestas JSON

Los controladores de API trabajan con Review::query() y User. ReviewController combina select, join, where, orderByDesc, forPage, count y selectRaw para construir reseñas de comunidad, promedio y paginación. Laravel serializa los resultados de Eloquent a JSON cuando se incluyen en response()->json().

25.5 Rutas web tradicionales heredadas de Breeze

web.php conserva /profile GET/PATCH/DELETE porque forman parte de la estructura que Breeze espera. La experiencia visible de CineVerse usa /es/profile mediante Vue Router y /api/profile para el resumen. Mantener las rutas tradicionales permite conservar parte de la infraestructura y pruebas originales, aunque no todas las pruebas heredadas siguen alineadas con la adaptación actual; la sección 34 lo detalla.

25.6 Mapa resumido: una petición JSON del frontend llega a Laravel

Los endpoints propios de CineVerse no siguen el mismo camino que TMDB. webApi.ts usa el mismo origen de Laravel, envía Accept: application/json y conserva las cookies de sesión con withCredentials: true. Las rutas de Laravel aplican middleware antes de ejecutar el Controller.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Vue
-> service TypeScript
-> webApi.ts / Axios
-> ruta Laravel
-> middleware
-> Controller / Form Request
-> Eloquent
-> MySQL
-> response()->json(...)
-> Axios response.data
-> Vue actualiza la interfaz
```

Cliente Axios	Destino	Credencial / contexto
tmdbApi.ts	TMDB	Authorization: Bearer <VITE_TMDB_TOKEN>
webApi.ts	Laravel de CineVerse	Cookie de sesión y, en mutaciones protegidas, X-CSRF-Token

25.7 public/index.php: el front controller de Laravel

Cuando el servidor web recibe una navegación o una request que debe procesar Laravel, el punto público de entrada es public/index.php. Este archivo no contiene reglas de CineVerse: carga el autoloader de Composer, obtiene la instancia Application desde bootstrap/app.php y entrega el Request al framework.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Navegador / Axios
-> public/index.php
-> require vendor/autoload.php
-> require bootstrap/app.php
-> Application recibe Request
-> routing + middleware + controller
```

A este patrón se lo conoce como front controller: muchas URLs distintas pasan por un único punto de entrada y es el framework el que decide qué código debe responder.

25.8 bootstrap/app.php: construir la Application

Laravel 12 concentra en bootstrap/app.php una parte importante del arranque que en versiones anteriores podía estar distribuida de otra forma. CineVerse utiliza `Application::configure()` y encadena `withRouting()`, `withMiddleware()` y `withExceptions()`.

- `withRouting` registra `routes/web.php`, `routes/api.php`, `routes/console.php` y la comprobación de estado `up`.
- `withMiddleware` agrega `HandleInertiaRequests` y `AddLinkHeadersForPreloadedAssets` al grupo web.
- `withExceptions` registra el reporte del canal de diagnóstico y la respuesta especial cuando MySQL no está disponible.

Este archivo se ejecuta antes de que una ruta concreta llegue a un Controller. Por eso puede definir comportamiento transversal para muchas requests.

25.9 Router de Laravel: método HTTP + URL + acción

Una ruta relaciona un método HTTP y una URL con una acción. El mismo texto `/reviews` puede tener significados distintos según el método: GET lista, POST crea. Dentro de `routes/api.php` las rutas se declaran como `/reviews`, pero `bootstrap/app.php` registra ese archivo mediante `withRouting(api: ...)`; Laravel le aplica el prefijo `/api`, por eso el frontend llama `/api/reviews`. `routes/api.php` agrupa además las rutas protegidas y aplica middleware `web` + `auth` antes de sus Controllers.

Ubicación en el proyecto: `routes/api.php` - rutas API Laravel. Este grupo usa middleware `web` + `auth` y conecta el CRUD `/reviews` con `App\Http\Controllers\Api\ReviewController` (`index`, `store`, `update` y `destroy`).

```
Route::middleware(["web", "auth"])->group(function () {
    Route::get("/reviews", [ReviewController::class, "index"]);
    Route::post("/reviews", [ReviewController::class, "store"]);
    Route::put("/reviews/{id}", [ReviewController::class, "update"]);
    Route::delete("/reviews/{id}", [ReviewController::class, "destroy"]);
});
```

El parámetro `{id}` se extrae desde la URL y Laravel lo entrega al método del Controller. En `movieReviews` ocurre algo parecido con `{movieId}`.

25.10 Middleware: cortar o permitir la request antes del Controller

Middleware se ejecuta alrededor de una ruta. `auth` exige un usuario autenticado; `guest` impide utilizar determinadas acciones de autenticación cuando ya existe una sesión; `signed` valida la firma de una URL; `throttle` limita la frecuencia de una ruta; `web` activa el conjunto de middleware de sesión, cookies, CSRF y otros comportamientos web.

```
POST /api/reviews
-> grupo api.php con web + auth
-> web recupera sesión/cookies y aplica CSRF
-> auth exige request()->user()
-> sólo entonces ReviewController::store()
```

Esto explica por qué la seguridad no depende de que Vue oculte un botón. Una request manual sigue pasando por los middleware del servidor.

25.11 Request y Form Request

`Illuminate\Http\Request` representa la solicitud actual. Permite leer query parameters, JSON, usuario, sesión y cabeceras. Un `Form Request` hereda de esa clase y agrega reglas de autorización/validación en una clase propia.

LoginRequest es el ejemplo más completo del proyecto: prepareForValidation() recorta el email; rules() valida forma y presencia; authenticate() consulta User, usa Hash::check, exige email verificado, llama Auth::login y limpia el RateLimiter; ensureIsNotRateLimited() aplica el límite de tres intentos por email + IP.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
POST /login
-> Laravel crea LoginRequest
-> prepareForValidation()
-> rules()
-> AuthenticatedSessionController::store(LoginRequest $request)
-> $request->authenticate()
-> session()->regenerate()
```

25.12 Controller: coordinar una acción HTTP

El Controller queda en la frontera entre HTTP y la lógica de la aplicación. Decide qué datos necesita leer del Request, qué modelos o facades utiliza y qué Response devuelve. En CineVerse algunos Controllers son muy pequeños porque el framework ya resolvió parte del trabajo; otros, como ReviewController, contienen consultas Eloquent y reglas propias del caso de uso.

Un Controller no es una página Vue. Vue vive en el navegador; el Controller atiende una request HTTP en el servidor.

25.13 Models y Eloquent

User y Review son Models Eloquent. User además extiende Authenticatable porque participa en el sistema de autenticación. Los Models describen asignación de atributos, casts y relaciones; las consultas se construyen mediante Review::query(), User::query(), where(), join(), count(), first(), get() y otros métodos del Query Builder/Eloquent.

25.14 Cómo Laravel construye las respuestas JSON

Los endpoints propios de CineVerse devuelven JsonResponse mediante response()->json(). ReviewController mantiene un contrato con success, code y, cuando hace falta, data. Laravel serializa arrays, modelos y colecciones a JSON y establece el status HTTP indicado.

Ubicación en el proyecto: app/Http/Controllers/Api/ReviewController.php · clase ReviewController · método store(Request \$request). Este bloque responde 409 cuando ya existe una reseña del mismo usuario para la película.

```
return response()->json([
    "success" => false,
    "code" => "REVIEW_ALREADY_EXISTS",
], 409);
```

El frontend recibe status 409 y ese JSON. Axios rechaza la Promise por el status de error; MovieReviews puede inspeccionar response.data.code y decidir qué mensaje mostrar. No existe en esta versión una clase Response.php propia como en el backend PHP anterior.

25.15 Notifications y Blade en el backend actual

User utiliza el trait Notifiable y reemplaza las notificaciones estándar con CineVerseVerifyEmail y CineVerseResetPassword. Una Notification decide por qué canal se envía el mensaje; en este proyecto via() devuelve mail. MailMessage selecciona una vista Blade distinta para español o inglés.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
User
-> notify(new CineVerseVerifyEmail())
-> Notification::via() = mail
-> toMail()
-> MailMessage
```

```
-> emails.verify-account-es.blade.php / -en.blade.php
-> mailer configurado en config/mail.php
```

25.16 Ejemplo guiado: Carlos edita una reseña de punta a punta

Carlos ya tiene una reseña y cambia su puntuación. Este ejemplo reúne las piezas anteriores sin saltarse las fronteras entre navegador, framework y base de datos.

Ubicación en el proyecto: resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue · **Component Vue** · submitForm()

Lo utiliza: MovieTabs.vue cuando la pestaña Reseñas está activa. Responsabilidad: recoger rating/comment y llamar createReview() o updateReview().

```
Carlos pulsa Guardar cambios
-> MovieReviews.vue
-> reviews.service.ts
-> webApi.put("/api/reviews/15", { rating, comment }, { X-CSRF-Token })
-> public/index.php
-> Laravel Router encuentra PUT /reviews/{id}
-> middleware web + auth
-> ReviewController::update($request, 15)
-> normalizeReviewComment()
-> $request->validate()
-> Review::query()->find(15)
-> comprobar review.user_id == request()->user()->id
-> $review->update(...)
-> Eloquent -> MySQL
-> JSON { success:true, code:"REVIEW_UPDATED" }
-> Axios
-> MovieReviews vuelve a cargar datos
-> Carlos ve el estado confirmado
```

26. Autenticación y cuenta: Breeze, sesiones y recuperación

La autenticación de CineVerse es un buen ejemplo de cómo se adaptó Breeze sin abandonar los mecanismos estándar de Laravel. Las pantallas son propias de la SPA, pero el servidor utiliza Auth, guard web, sesiones, hashing, RateLimiter, MustVerifyEmail, Password Broker y middleware. Explicar sólo “usa Breeze” ocultaría qué ocurre realmente.

26.1 Qué aporta Breeze y qué ejecuta Laravel realmente

Breeze no es un servicio que quede corriendo detrás de CineVerse. En este proyecto aparece en require-dev y aportó la estructura inicial de código —el scaffolding— para rutas de autenticación, controladores, LoginRequest y pruebas Feature/Auth. A partir de esa base, CineVerse adaptó el comportamiento y conectó sus propias pantallas Vue. Cuando una persona inicia sesión, lo que se ejecuta en tiempo real son componentes del framework Laravel: el guard web, Auth, Hash, RateLimiter, la sesión, cookies, middleware, MustVerifyEmail y el Password Broker.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Breeze (estructura inicial de código / scaffolding)
-> routes/auth.php + controllers Auth + LoginRequest + tests

Ejecución real
Vue propio de CineVerse
-> rutas Laravel
-> Auth / Hash / RateLimiter / sesiones / middleware
-> User / Eloquent / MySQL
-> respuestas JSON de vuelta a Vue
```

Por eso “CineVerse usa Breeze” describe el origen de la estructura, pero no basta para explicar la autenticación. También explica por qué las pantallas visibles no son las vistas predeterminadas de Breeze: LoginView.vue, RegisterView.vue, VerifyEmailView.vue y las demás forman parte de la SPA de CineVerse y consumen las rutas Laravel mediante auth.service.ts y webApi.ts.

26.1.1 Ejemplo guiado: Carlos crea una cuenta

Carlos completa RegisterView con username, email, password y confirmación. Vue valida la experiencia básica, pero la creación real de la cuenta ocurre en Laravel. RegisterView agrega además language para que la notificación de verificación utilice la plantilla del idioma activo.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Crear cuenta
-> RegisterView
-> registerUser(payload)
-> auth.service.ts
-> webApi.post("/register")
-> routes/auth.php
-> RegisteredUserController::store
-> normalizar username / email / language
-> validar username / email / password
-> User::create()
-> Eloquent -> MySQL
-> sendEmailVerificationNotification()
-> CineVerseVerifyEmail
-> Blade ES/EN
-> SMTP
```

A diferencia del comportamiento inicial de Breeze, este controlador no ejecuta Auth::login después del registro. Carlos debe verificar el correo y luego iniciar sesión.

26.2 Login: desde el formulario hasta la sesión

LoginView valida el formato básico y llama loginUser(). auth.service.ts envía POST/login mediante webApi. Como la petición declara Accept: application/json, el controlador responde 204 cuando el acceso es correcto. Antes de eso, LoginRequest ejecuta las comprobaciones de servidor.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
LoginRequest::authenticate()
1. ensureIsNotRateLimited()
2. buscar User por email
3. Hash::check(password, user.password)
4. exigir user->hasVerifiedEmail()
5. Auth::login(user, remember)
6. RateLimiter::clear(throttleKey)

AuthenticatedSessionController
7. session()->regenerate()
8. responder 204 a Vue
```

La regeneración del identificador de sesión después de autenticar reduce el riesgo de fijación de sesión. El navegador conserva la cookie que Laravel emite; esa cookie no contiene la contraseña ni reemplaza la base de datos, sino que permite vincular futuras peticiones con la sesión almacenada por el driver configurado.

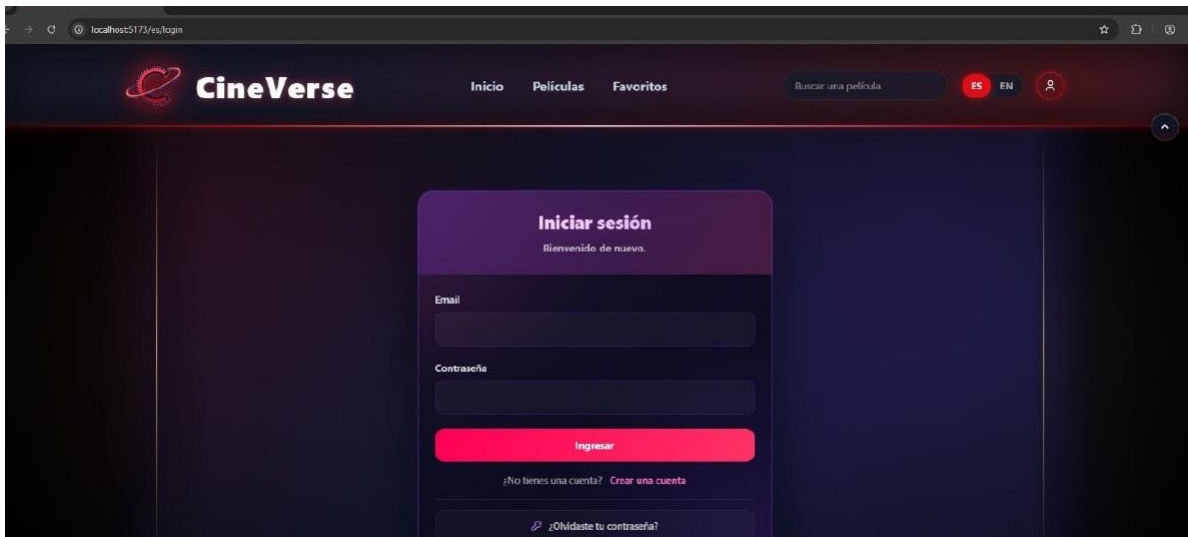


Figura. Pantalla de inicio de sesión de CineVerse.

26.2.1 Ejemplo guiado: Carlos inicia sesión

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos escribe email y contraseña
-> LoginView valida formato
-> loginUser()
-> POST /login
-> LoginRequest::authenticate()
-> ensureIsNotRateLimited()
-> User por email
-> Hash::check()
-> hasVerifiedEmail()
-> Auth::login()
-> session()->regenerate()
-> 204
-> frontend consulta GET /user
-> Laravel devuelve user + csrf_token
-> useAuthStore actualiza Pinia
```

El 204 confirma el login pero no contiene el perfil completo. CineVerse consulta después /user para reconstruir el objeto reactivo que utilizan el header, los guards y el perfil.

26.3 Reconstruir la sesión después de F5

Una vez que el usuario está autenticado, recargar la página destruye el estado JavaScript en memoria. Por eso app.ts llama authStore.initialize(). El store pide GET /user. Laravel toma request()->user() desde la sesión y devuelve id, username, email, created_at y csrf_token. Pinia vuelve a poblar user y csrfToken sin pedir nuevamente email/password.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
F5
-> app.ts
-> useAuthStore.initialize()
-> GET /user
-> cookie de sesión viaja con el request
-> middleware auth
-> request()->user()
-> { user, csrf_token }
-> Pinia reconstruye el estado
```

26.4 Registro y verificación de correo

RegisteredUserController normaliza username, email y language antes de validar. username debe tener 3–20 caracteres y usar letras, números o guion bajo; email debe ser válido, máximo 80 y único. password utiliza

Rules\Password::defaults() y confirmación. En app/Providers/AppServiceProvider.php, Password::defaults() está configurado con mínimo 8 y máximo 32 caracteres, combinación de mayúsculas y minúsculas, al menos un número y al menos un símbolo. Después User::create() guarda la contraseña hasheada y se llama sendEmailVerificationNotification().

A diferencia del Breeze inicial, el registro no ejecuta Auth::login. La cuenta queda creada pero necesita verificar el correo antes del login. User implementa MustVerifyEmail y reemplaza la notificación estándar por CineVerseVerifyEmail, que usa plantillas HTML separadas en español e inglés.

26.4.1 Enlace firmado

CineVerseVerifyEmail crea una URL temporal firmada con verification.verify, id, hash del email y lang. routes/auth.php aplica middleware signed y throttle:6,1. VerifyEmailController vuelve a cargar User, compara el hash con sha1(getEmailForVerification()), marca email_verified_at y redirige a /es/verify-email?verified=1 o su equivalente inglés.

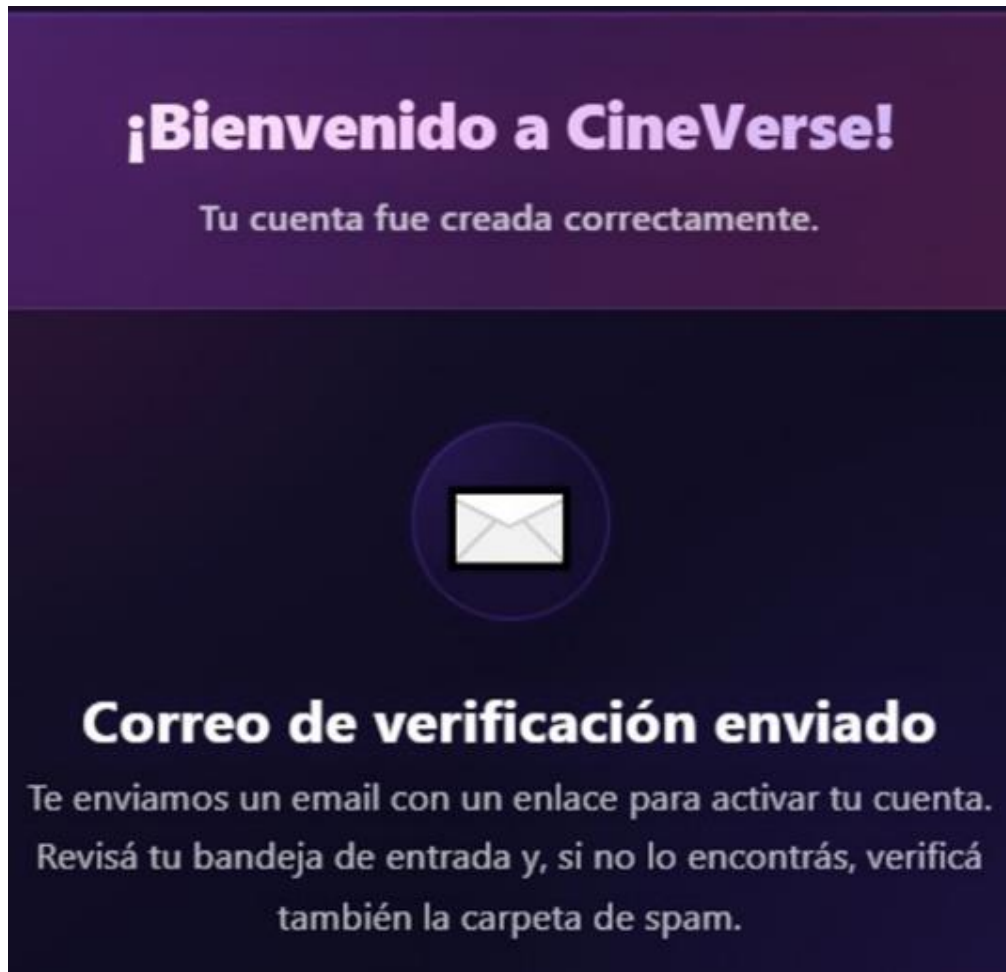


Figura. Pantalla de verificación de correo de CineVerse.

26.4.2 Duración real del enlace de verificación en esta versión

CineVerseVerifyEmail genera la URL con now()->addMinutes(config("auth.verification.expire", 60)). config/auth.php no define una clave auth.verification.expire; por eso esta llamada utiliza su valor de respaldo: 60 minutos. Las plantillas verify-account-es.blade.php y verify-account-en.blade.php muestran actualmente ese mismo tiempo de validez, por lo que el texto del correo coincide con la expiración efectiva del enlace.

26.5 Reenvío de verificación

La ruta POST /email/verification-notification está dentro de middleware auth y usa throttle:6,1. Al mismo tiempo, LoginRequest impide completar Auth::login() cuando el correo todavía no está verificado, mientras LoginView ofrece un botón de reenvío después de ese rechazo. Por lectura del flujo completo, el frontend puede intentar el reenvío sin disponer de la sesión que el endpoint protegido exige. Esta es una inconsistencia del comportamiento actual entre la interfaz y la protección de la ruta.

26.6 Recuperación de contraseña y Password Broker

ForgotPasswordView envía POST /forgot-password. PasswordResetLinkController delega en Password::sendResetLink(), que utiliza el Password Broker configurado como users en config/auth.php y la tabla password_reset_tokens. El Password Broker es el mecanismo de Laravel que coordina los tokens de restablecimiento: los crea, los guarda, controla su vencimiento —60 minutos— y aplica un throttle de 60 segundos. User personaliza la notificación mediante CineVerseResetPassword para conservar el diseño e idioma de CineVerse.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
solicitud
ForgotPasswordView -> POST /forgot-password
-> Password::sendResetLink()
-> password_reset_tokens
-> CineVerseResetPassword -> email

cambio final
ResetPasswordView -> POST /reset-password
-> Password::reset()
-> nuevo hash de password
-> nuevo remember_token
```

26.6.1 Qué valida realmente ResetPasswordView al abrir el enlace

Los comentarios internos de ResetPasswordView indican que el token se valida contra el backend al montar la vista. El código ejecutado actualmente no realiza esa consulta: onMounted sólo comprueba que token y email estén presentes en la URL y, si existen, cambia tokenStatus a valid. La validez real y la expiración del token se comprueban cuando Carlos envía POST /reset-password y Laravel ejecuta Password::reset().

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
abrir /reset-password?token=...&email=...
-> ResetPasswordView.onMounted()
-> ¿existen token y email?
    no -> tokenStatus="invalid"
    sí -> tokenStatus="valid" (sin request de validación)

Carlos envía nueva contraseña
-> POST /reset-password
-> Password::reset()
-> Laravel valida token / expiración
-> éxito o error
```

26.7 Logout

logoutUser() envía POST /logout con X-CSRF-Token. AuthenticatedSessionController llama Auth::guard("web")->logout(), invalida la sesión y regenera el token CSRF. Después Pinia limpia su representación local. Cerrar sesión, por lo tanto, no consiste sólo en ocultar el nombre del header: destruye la identidad autenticada del lado Laravel.

26.8 Guard web, sesión y cookie: tres conceptos relacionados

config/auth.php define el guard web con driver session. El guard es la estrategia que Laravel utiliza para saber qué usuario corresponde a la request. La sesión guarda estado en el servidor o en el driver configurado; la cookie del navegador transporta el identificador necesario para vincular requests posteriores con esa sesión.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
login correcto
-> Auth::login($user)
-> guard web asocia usuario con la sesión
-> session()->regenerate() rota el id
-> navegador recibe/actualiza cookie de sesión

request posterior
-> cookie viaja automáticamente
-> middleware web recupera la sesión
-> auth puede resolver request()->user()
```

Pinia no sustituye este mecanismo. useAuthStore guarda una representación reactiva para la interfaz, pero la identidad verdadera se reconstruye desde Laravel mediante GET /user.

26.9 Hash y la contraseña almacenada

User tiene un cast password => hashed y los Controllers también utilizan Hash::make al crear o restablecer credenciales. Durante el login, LoginRequest usa Hash::check para comparar la contraseña escrita con el hash almacenado. La contraseña en texto plano no se guarda en la tabla users.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos escribe password
-> LoginRequest
-> User::where("email", ...)->first()
-> Hash::check(password escrita, user.password hash)
-> true: puede continuar
-> false: ValidationException + RateLimiter::hit()
```

26.10 RateLimiter del login por email + IP

LoginRequest::throttleKey() combina el email en minúsculas con la IP. ensureIsNotRateLimited() permite hasta tres intentos antes de bloquear. Cada fallo de credenciales ejecuta RateLimiter::hit(); un login correcto ejecuta RateLimiter::clear(). Cuando se supera el límite se emite Lockout y se lanza una ValidationException con status 429.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
clave = strtolower(email) + "|" + ip

si tooManyAttempts(clave, 3)
-> Lockout
-> 429
si credenciales incorrectas
-> RateLimiter::hit(clave)
si login correcto
-> RateLimiter::clear(clave)
```

26.11 Password Broker: recuperación de contraseña en Laravel

Password Broker es el mecanismo de Laravel que coordina tokens de restablecimiento. Password::sendResetLink() localiza al usuario, crea o reemplaza el token correspondiente y llama sendPasswordResetNotification() en el modelo. User redirige esa notificación a CineVerseResetPassword, que construye el enlace hacia la pantalla Vue.

Al guardar una contraseña nueva, Password::reset() valida token, email y vencimiento según la configuración del broker. Si el token es válido, NewPasswordController aplica Hash::make, rota remember_token, guarda User y emite el evento PasswordReset.

26.11.1 Ejemplo guiado: Carlos olvidó su contraseña

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Paso 1: solicitar correo
Carlos abre /es/forgot-password
```

```

-> ForgotPasswordView
-> POST /forgot-password { email, language }
-> PasswordResetLinkController::store()
-> Password::sendResetLink()
-> User::sendPasswordResetNotification($token)
-> CineVerseResetPassword
-> Blade ES/EN
-> SMTP

Paso 2: abrir enlace
Carlos pulsa el enlace
-> /es/reset-password?token=...&email=...
-> ResetPasswordView comprueba que token y email existan
-> muestra el formulario

Paso 3: guardar nueva contraseña
-> POST /reset-password
-> NewPasswordController::store()
-> validar token/email/password
-> Password::reset()
-> Hash::make() + remember_token nuevo
-> 204
-> Carlos vuelve a Login

```

En la implementación actual ResetPasswordView no comprueba remotamente el token al montar. Aunque sus comentarios internos dicen lo contrario, sólo verifica que token y email estén presentes; la validez real se decide cuando se envía POST /reset-password.

26.12 remember_token

La tabla users incluye remember_token porque forma parte de las convenciones de autenticación de Laravel. En el reset de contraseña CineVerse lo rota con Str::random(60). Este token no es el token de recuperación ni el CSRF token; son valores con propósitos distintos.

26.13 Verificación de correo: firma, hash y estado del usuario

CineVerseVerifyEmail genera una URL temporal firmada con id, sha1 del email y lang. El middleware signed comprueba que los parámetros protegidos no hayan sido alterados. VerifyEmailController vuelve a cargar el usuario, compara el hash esperado y marca email_verified_at. Sólo después LoginRequest permitirá el acceso.

La URL firmada usa config("auth.verification.expire", 60) y config/auth.php no define esa clave, por lo que se utiliza el valor alternativo de 60 minutos. Las plantillas del email muestran actualmente ese mismo tiempo de validez.

27. Reseñas y perfil

Las reseñas son el dato de dominio más completo de CineVerse porque atraviesan interfaz, sesión, CSRF, rutas Laravel, Controller, Eloquent y MySQL. Para evitar dar esas piezas por sabidas, esta sección sigue siempre el recorrido desde lo que hace Carlos hasta la consulta que se ejecuta y la respuesta que vuelve a Vue. La película continúa perteneciendo a TMDB; MySQL guarda únicamente la actividad propia de CineVerse.

27.1 Reglas de una reseña

Campo	Validación en Laravel
movie_id	required integer min:1
movie_title	nullable string max:255
rating	required integer min:1 max:5
comment	required string min:20 max:1000

El comentario debe tener entre 20 y 1000 caracteres tanto al crear como al actualizar. La interfaz puede impedir envíos inválidos, pero ReviewController vuelve a validar porque el navegador no es una frontera de seguridad. movie_id debe ser positivo, movie_title es opcional en la request y rating queda limitado al rango 1–5. Hay una diferencia importante entre validación HTTP y persistencia: la migración define movie_title como una columna string no nullable y ReviewController::store() guarda \$validated["movie_title"] ?? ""; por eso, si el cliente no envía título, la fila se persiste con una cadena vacía y no con NULL.

27.1.1 Ejemplo guiado: Carlos crea una reseña

Carlos abre la pestaña Reseñas de una película, elige una puntuación y escribe un comentario. MovieReviews valida el formulario para la experiencia de usuario; al guardar, reviews.service.ts envía la mutación a Laravel junto con el token CSRF de la sesión.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Guardar
-> MovieReviews
-> createReview(payload, csrfToken)
-> reviews.service.ts
-> POST /api/reviews
    cookie de sesión
    X-CSRF-Token
-> middleware web + auth
-> ReviewController::store
-> validar movie_id / movie_title / rating / comment
-> comprobar duplicado user_id + movie_id
-> Review::create()
-> MySQL
-> JSON REVIEW_CREATED
-> MovieReviews vuelve a cargar stats y listado
```

El frontend puede impedir un envío inválido, pero Laravel vuelve a validar porque una request también puede construirse fuera de la interfaz.

27.1.2 Mapa de archivos que participan al crear una reseña

Frontend: MovieReviews.vue contiene el formulario y submitForm(); reviews.service.ts define createReview(); webApi.ts crea el cliente Axios que habla con Laravel y mantiene cookies de sesión.

Backend: routes/api.php registra POST /api/reviews dentro de middleware web + auth; app/Http/Controllers/Api/ReviewController.php, clase ReviewController, método store(), normaliza y valida el payload; app/Models/Review.php es el modelo Eloquent utilizado para crear la fila.

Base de datos: database/migrations/2026_07_11_120000_create_reviews_table.php define reviews, la foreign key hacia users y la restricción UNIQUE(user_id, movie_id).

27.2 Una reseña por usuario y película

Antes de insertar, ReviewController busca una fila con user_id igual al usuario de la sesión y movie_id igual al payload. Si existe responde REVIEW_ALREADY_EXISTS con 409. Además, la migración define UNIQUE(user_id, movie_id). La regla queda protegida tanto en la lógica de aplicación como en la base.

27.3 Editar y eliminar sólo la propia

update() y destroy() cargan la reseña por id. Si no existe devuelven 404. Si review.user_id no coincide con request()->user()->id devuelven 403 NOT_REVIEW_OWNER. El hecho de que Vue esconda botones ajenos mejora la experiencia, pero la verdadera autorización se hace en el servidor.

27.4 Reseñas de una película

La lectura de reseñas es pública, pero puede aprovechar una sesión existente. Cuando Carlos abre la pestaña Reseñas, MovieTabs monta MovieReviews; MovieReviews ejecuta loadInitial(), que llama fetchReviews(1), y

reviews.service.ts envía GET /api/movies/{movieId}/reviews?page=1. La ruta sólo exige middleware web: un invitado puede leer, mientras que una cookie válida permite a Laravel reconocer opcionalmente al usuario.

La respuesta separa cuatro conceptos: estadísticas globales de la película, datos de paginación, reseña propia del usuario si existe y listado comunitario. Esa separación explica por qué la reseña de Carlos puede mostrarse destacada sin aparecer duplicada entre las tarjetas de la comunidad.

27.4.1 Ejemplo guiado: Carlos abre la pestaña Reseñas de Fight Club

Fight Club tiene movieId=550. La pestaña Reparto es la inicial; por eso esta consulta no ocurre al abrir el detalle. Recién cuando Carlos pulsa “Reseñas”, Vue crea MovieReviews y se ejecuta onMounted(loadInitial).

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos abre Fight Club (id 550)
-> MovieTabs.vue cambia activeTab a "reviews"
-> Vue monta MovieReviews.vue
-> onMounted(loadInitial)
-> fetchReviews(1)
-> reviews.service.ts :: getMovieReviews(550, 1)
-> webApi GET /api/movies/550/reviews?page=1
-> routes/api.php :: middleware web
-> ReviewController::movieReviews($request, 550)
-> Eloquent consulta reviews + users
-> JSON stats/pagination/user_review/reviews
-> MovieReviews actualiza refs
-> Carlos ve promedio, total, su reseña y la comunidad
```

27.4.2 Dónde está cada paso

Componente que activa la pestaña: resources/js/cineverse/features/movies/components/movie-detail/MovieTabs.vue · Component Vue · activeTab y render condicional de MovieReviews.

Componente que pide y representa datos: resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue · Component Vue · loadInitial(), fetchReviews() y loadMore().

Service HTTP: resources/js/cineverse/features/movies/services/reviews.service.ts · getMovieReviews(movieId, page).

Ruta: routes/api.php · GET /api/movies/{movieId}/reviews con middleware web.

Backend: app/Http/Controllers/Api/ReviewController.php · clase ReviewController · método movieReviews(Request \$request, int \$movieId).

Modelo consultado: app/Models/Review.php · clase Review · Eloquent sobre la tabla reviews.

27.5 Perfil y mis reseñas

Perfil y Mis reseñas usan la misma identidad de sesión, pero responden a preguntas distintas. GET /api/profile construye un resumen corto: total de reseñas y las seis más recientes. GET /api/reviews?page=N recorre el historial completo en páginas de 12. El primero alimenta ProfileView; el segundo alimenta MyReviewsView.

27.5.1 Ejemplo guiado: Carlos abre Perfil

Ubicación en el frontend: resources/js/cineverse/features/profile/views/ProfileView.vue · View · onMounted(loadProfile).

Ubicación del service: resources/js/cineverse/features/profile/services/profile.service.ts · getProfile() · GET /api/profile.

Ubicación en Laravel: app/Http/Controllers/Api/ProfileController.php · clase ProfileController · método show(Request \$request). La ruta está en routes/api.php dentro de middleware web + auth.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos entra a /es/profile
-> ProfileView.vue :: onMounted(loadProfile)
```

```

-> profile.service.ts :: getProfile()
-> GET /api/profile
-> routes/api.php :: web + auth
-> Api\ProfileController::show()
-> $request->user()->id identifica a Carlos
-> Review::where(user_id)->count()
-> Review::where(user_id)->orderByDesc(created_at)->limit(6)->get()
-> JSON reviews_count + latest_reviews
-> ProfileView actualiza reviewsCount y latestReviews
-> Carlos ve el total y sus seis reseñas más recientes

```

Ubicación en el proyecto: app/Http/Controllers/Api/ProfileController.php · **Controller API Laravel** · App\Http\Controllers\Api\ProfileController::show()

Lo utiliza: GET /api/profile desde routes/api.php; la respuesta es consumida por profile.service.ts.

Responsabilidad: contar todas las reseñas propias y recuperar sólo las seis más recientes.

```

$reviewsCount = Review::query()
    ->where('user_id', $userId)
    ->count();

$latestReviews = Review::query()
    ->where('user_id', $userId)
    ->orderByDesc('created_at')
    ->limit(6)
    ->get();

```

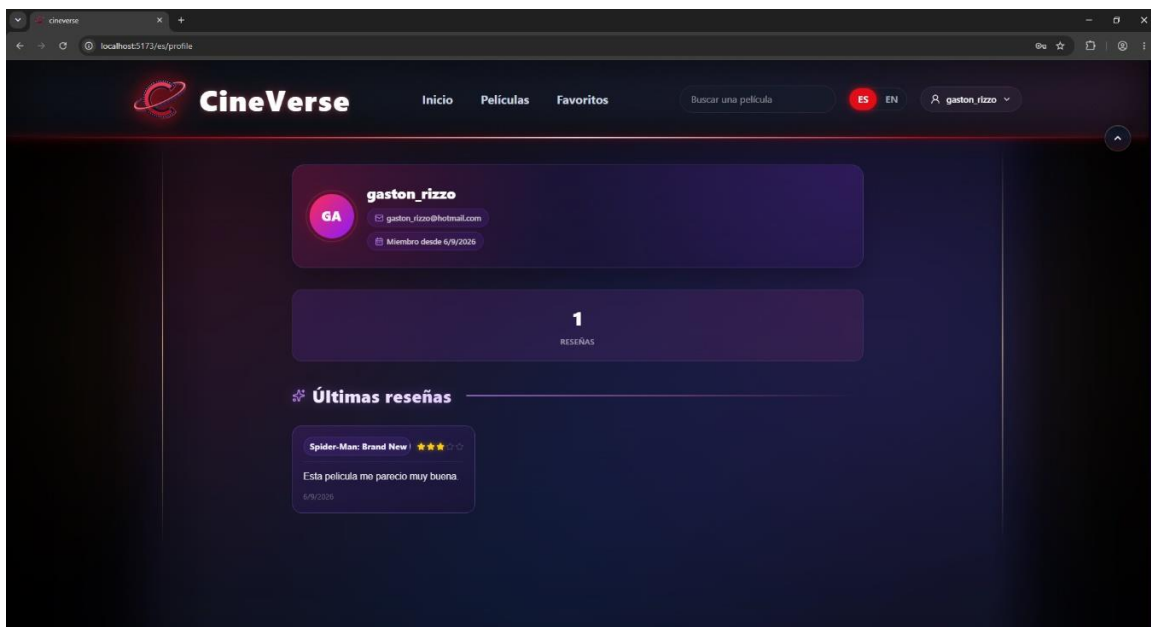


Figura. Perfil de usuario con una reseña.

27.5.2 Ejemplo guiado: Carlos abre “Mis reseñas”

Ubicación en el frontend: resources/js/cineverse/features/profile/views/MyReviewsView.vue · View · loadReviews(), fetchReviews(page) y loadMore().

Ubicación del service: resources/js/cineverse/features/movies/services/reviews.service.ts · getUserReviews(page) · GET /api/reviews?page=N.

Ubicación en Laravel: app/Http/Controllers/Api/ReviewController.php · clase ReviewController · método index(Request \$request). La ruta usa middleware web + auth.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Carlos abre Mis reseñas
-> MyReviewsView.vue :: onMounted(loadReviews)
-> fetchReviews(1)
-> reviews.service.ts :: getUserReviews(1)

```

```

-> GET /api/reviews?page=1
-> ReviewController::index()
-> page = max(1, query page)
-> perPage = 12
-> contar total de reseñas propias
-> forPage(1, 12)
-> JSON pagination + reviews
-> MyReviewsView reemplaza reviews[] con la página 1

```

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Carlos pulsa "Cargar más"
-> MyReviewsView.loadMore()
-> fetchReviews(currentPage + 1)
-> GET /api/reviews?page=2
-> Laravel devuelve hasta otras 12
-> reviews.value = [...reviews.value, ...data.reviews]
-> las tarjetas nuevas aparecen debajo de las anteriores

```

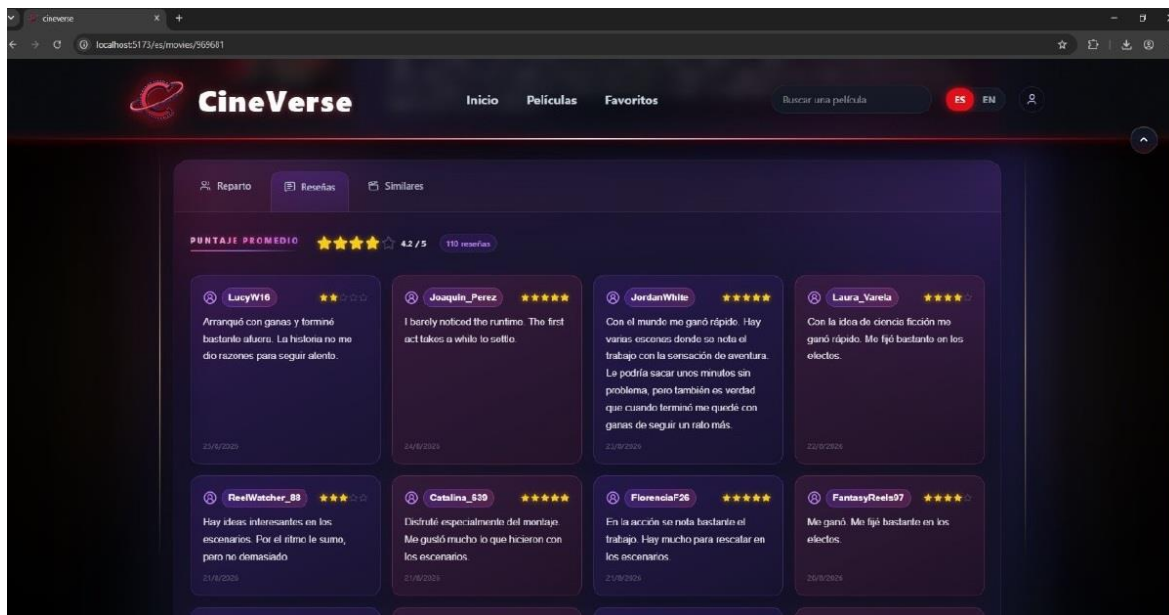


Figura. Pestaña de reseñas de una película.

27.6 Después de crear, editar o borrar

MovieReviews no intenta adivinar localmente cómo quedaron el promedio, el total o la lista después de una mutación. `submitForm()` y `confirmDelete()` llaman nuevamente `fetchReviews(1)`. Así la interfaz vuelve a leer el estado confirmado por Laravel/MySQL y actualiza stats, `userReview`, `reviews`, `currentPage` y `hasNext` con una única respuesta coherente.

27.7 Qué ocurre en MovieReviews antes de enviar una reseña

MovieReviews.vue mantiene estado local para `formMode`, `formRating`, `formComment`, `isSubmitting` e `isFormOpen`. El botón de creación sólo aparece cuando `authStore.isAuthenticated` es true, no existe `userReview` y el formulario está cerrado. Esa comprobación mejora la experiencia, pero no reemplaza `auth` ni la validación del servidor.

Ubicación en el backend: `app/Http/Controllers/Api/ReviewController.php` · método `normalizeReviewComment()`. Antes de validar, Laravel convierte CRLF/CR a LF, elimina espacios alrededor de los saltos, reduce tres o más saltos consecutivos a dos y aplica `trim`. Recién después se evalúan `min:20` y `max:1000`.

27.8 Editar una reseña propia

Editar no vuelve a enviar `movie_id` ni `movie_title`. `ReviewController::update` valida `rating` y `comment`, busca la reseña por `id` y comprueba que `review.user_id` coincida con `request()->user()->id`. Sólo entonces ejecuta `update`.

Ubicación en el proyecto: resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue · **Component Vue** · openEditForm() + submitForm()

Lo utiliza: MovieTabs.vue. Responsabilidad: precargar rating/comment, enviar PUT y recargar la primera página al finalizar.

```
Carlos pulsa Editar
-> formulario carga rating/comment actuales
-> PUT /api/reviews/{id}
-> auth + CSRF
-> validate rating 1..5 / comment 20..1000
-> Review::find(id)
-> si no existe: REVIEW_NOT_FOUND 404
-> si pertenece a otro usuario: NOT_REVIEW_OWNER 403
-> $review->update(...)
-> REVIEW_UPDATED
```

27.9 Eliminar una reseña propia

La eliminación repite la comprobación de existencia y propiedad. No alcanza con ocultar el botón de eliminar en Vue: el backend compara el user_id antes de llamar delete().

Ubicación en el proyecto: app/Http/Controllers/Api/ReviewController.php · **Controller API Laravel** · ReviewController::destroy()

Lo utiliza: DELETE /api/reviews/{id} desde routes/api.php. Responsabilidad: comprobar existencia/propiedad y eliminar la reseña.

```
DELETE /api/reviews/{id}
-> Review::find(id)
-> comprobar propietario
-> $review->delete()
-> REVIEW_DELETED
-> frontend vuelve a consultar la primera página
```

27.10 Por dentro de ReviewController::movieReviews()

La subsección 27.4 siguió el recorrido visible desde MovieTabs y MovieReviews. Aquí se observa únicamente la implementación del servidor que sostiene ese recorrido: la ruta pública de routes/api.php y el método ReviewController::movieReviews(), responsable de separar reseña propia, comunidad, paginación y estadísticas.

El listado comunitario y las estadísticas no son la misma consulta. communityQuery hace JOIN con users para añadir username, excluye al usuario autenticado y pagina de a 10. Una consulta separada sobre Review calcula COUNT(*) y ROUND(AVG(rating), 1) incluyendo todas las reseñas de la película, también la propia.

Ubicación en el proyecto: app/Http/Controllers/Api/ReviewController.php · **Controller API Laravel** · ReviewController::movieReviews()

Lo utiliza: GET /api/movies/{movieId}/reviews. Responsabilidad: construir y paginar el listado comunitario con username, sin duplicar la reseña propia.

```
$communityQuery = Review::query()
->join('users', 'users.id', '=', 'reviews.user_id')
->where('reviews.movie_id', $movieId)
->orderByDesc('reviews.created_at');

if ($userId !== null) {
    $communityQuery->where('reviews.user_id', '<>', $userId);
}

$reviews = $communityQuery
->forPage($page, $perPage)
->get();
```

Ubicación en el proyecto: app/Http/Controllers/Api/ReviewController.php · **Controller API Laravel** · ReviewController::movieReviews()

Lo utiliza: MovieReviews.vue mediante getMovieReviews(). *Responsabilidad:* calcular total y promedio globales de la película.

```
$stats = Review::query()
    ->where('movie_id', $movieId)
    ->selectRaw('COUNT(*) as total_reviews, ROUND(AVG(rating), 1) as average_rating')
    ->first();
```

27.11 Comparación técnica: Perfil versus Mis reseñas

La diferencia ya quedó demostrada en los ejemplos 27.5.1 y 27.5.2: Perfil necesita un resumen fijo y Mis reseñas necesita un historial paginado. En términos de backend, ProfileController::show() aplica limit(6), mientras ReviewController::index() fija perPage = 12 y calcula has_next. Esta subsección conserva sólo la comparación técnica para no repetir los recorridos de Carlos.

Ubicación en el proyecto: app/Http/Controllers/Api/ReviewController.php · **Controller API Laravel** · ReviewController::index()

Lo utiliza: GET /api/reviews?page=N desde MyReviewsView.vue. *Responsabilidad:* paginar el historial propio de a 12 y calcular has_next.

```
$page = max(1, (int) $request->query('page', 1));
$perPage = 12;

$totalReviews = (clone $query)->count();

$reviews = Review::query()
    ->where('user_id', $request->user()->id)
    ->orderByDesc('created_at')
    ->forPage($page, $perPage)
    ->get();
```

Resultado: ProfileController devuelve como máximo seis elementos recientes para una vista de resumen; ReviewController::index() devuelve hasta 12 por página y permite seguir avanzando mientras has_next sea true. Son dos consultas distintas sobre el mismo usuario de sesión.

Figura. Las seis reseñas más recientes.

28. Validación, seguridad, CSRF y RateLimiter

La seguridad de CineVerse no depende de una única función. Validación, hashing, sesiones, CSRF, rate limiting, middleware, restricciones de base y autorización sobre reseñas cubren riesgos diferentes. El frontend también valida por experiencia de usuario, pero Laravel vuelve a comprobar lo que llega por HTTP.

28.1 Validación del frontend versus servidor

TypeScript y las funciones de validation ayudan a detectar errores antes de enviar un formulario, pero pueden ser evitadas con herramientas externas. Laravel valida registro, login, reseñas y perfil. MySQL añade integridad estructural mediante foreign keys y la restricción única de reviews. Esta defensa por capas permite que la UI sea cómoda sin convertirla en la única garantía.

28.2 Contraseñas

User define el cast password => hashed. RegisteredUserController además usa Hash::make al crear la cuenta. LoginRequest nunca compara texto plano con texto plano: Hash::check verifica la contraseña ingresada contra el hash almacenado. Las respuestas de credenciales inválidas no revelan si falló el email o el password, reduciendo información útil para enumeración de cuentas.

28.3 RateLimiter del login

LoginRequest crea throttleKey combinando email normalizado e IP. ensureIsNotRateLimited() permite como máximo tres intentos antes de devolver 429. RateLimiter::availableIn informa el tiempo restante y se dispara el evento Lockout. Después de un login correcto RateLimiter::clear elimina los fallos acumulados.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
throttleKey = lower(email) + "|" + ip

if RateLimiter::tooManyAttempts(key, 3)
  -> Lockout event
  -> ValidationException status 429
else
  -> comprobar credenciales
  -> fallo: RateLimiter::hit(key)
  -> éxito: RateLimiter::clear(key)
```

28.4 CSRF

Una cookie de sesión puede viajar automáticamente con una petición del navegador. CSRF protege contra el escenario donde otro sitio intenta aprovechar esa cookie para ejecutar una acción no deseada. En las rutas web normales, Laravel y Axios utilizan el mecanismo XSRF-TOKEN / X-XSRF-TOKEN. Una vez autenticado el usuario, GET /user devuelve además csrf_token y useAuthStore lo mantiene en memoria; createReview, updateReview, deleteReview y logoutUser envían ese valor explícitamente en X-CSRF-Token.

CSRF no reemplaza a auth. La cookie permite que Laravel sepa quién es el usuario; el token demuestra que la solicitud que modifica estado conoce un valor generado dentro de la sesión. Por eso una operación sensible puede requerir ambas cosas.

28.5 Middleware auth y guest

Las rutas de login/registro/reset están agrupadas bajo guest. Las rutas de /user, reenvío de verificación, confirmación de contraseña, cambio de password y logout están bajo auth. En routes/api.php, perfil y CRUD de reseñas usan web + auth. El frontend añade guards requiresAuth/requiresGuest para evitar pantallas incoherentes, pero el middleware del servidor sigue siendo la barrera real.

28.6 URLs firmadas y throttle de verificación

La verificación de correo combina una URL temporal firmada con throttle:6,1. signed impide modificar parámetros protegidos sin invalidar la firma. El controlador además compara el hash del email con hash_equals antes de marcar la cuenta. Una URL manipulada no debería convertirse en una verificación válida.

28.7 Qué aporta realmente el grupo web

routes/api.php agrega explícitamente middleware web a los endpoints que deben compartir la sesión del navegador. El grupo web de Laravel reúne middleware relacionados con cookies, sesión y protección CSRF. Esto permite que /api/profile y el CRUD de reseñas funcionen con la misma sesión que /login y las rutas web.

La ruta pública de reseñas de una película también usa web. No exige auth, pero si existe una cookie válida Laravel puede resolver request()->user() y devolver user_review separada.

28.8 Guard de Vue Router versus middleware de Laravel

requiresAuth y requiresGuest de Vue Router sirven para que la SPA no muestre pantallas incoherentes. No son una frontera de seguridad porque sólo viven en el navegador. auth y guest de Laravel sí se ejecutan en el servidor para cada request protegida.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Vue guard requiresAuth
  -> experiencia de navegación

Laravel middleware auth
```

```
-> protección real del endpoint
-> una request manual sin sesión tampoco pasa
```

28.9 Hashing de contraseñas

Hash::make crea el hash utilizado al registrar o restablecer una contraseña; Hash::check verifica el login. Las reglas Password::defaults definidas en AppServiceProvider exigen entre 8 y 32 caracteres, mayúsculas y minúsculas, número y símbolo. Validar composición y almacenar un hash son tareas diferentes.

28.10 RateLimiter y cache

RateLimiter no utiliza la tabla login_attempts. Su estado se apoya en el sistema de cache configurado por Laravel. Con CACHE_STORE=database, las claves de conteo y vencimiento se almacenan mediante las tablas de cache. LoginRequest construye una clave específica con email + IP para que el límite corresponda a ese usuario/origen.

28.11 Ejemplo guiado: por qué una reseña necesita sesión y CSRF

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos ya inició sesión
-> GET /user devuelve csrf_token
-> Pinia guarda csrfToken

Carlos pulsa Guardar reseña
-> browser envía cookie de sesión
-> reviews.service.ts agrega X-CSRF-Token
-> middleware web valida sesión/CSRF
-> auth resuelve usuario
-> ReviewController valida payload
-> Eloquent guarda

Si falta sesión: auth corta el recorrido
Si falta/vence CSRF: middleware web rechaza la escritura
```

La cookie responde “qué sesión es”; el CSRF token demuestra que la request de escritura conoce un valor generado dentro de esa sesión. Una cosa no reemplaza a la otra.

28.12 Defensa por capas en POST /api/reviews

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
1. Vue valida experiencia y longitud
2. web/auth protegen la request
3. CSRF protege la mutación
4. $request->validate comprueba tipos/rangos
5. Controller comprueba duplicado/propiedad
6. Eloquent usa parámetros SQL
7. MySQL aplica FOREIGN KEY + UNIQUE
```

Cada capa cubre un problema diferente. El hecho de que MovieReviews no permita seleccionar seis estrellas no elimina la necesidad de max:5 en Laravel; una request puede construirse sin utilizar la interfaz Vue.

29. Estados de la interfaz y manejo de errores

CineVerse representa más estados que “éxito” y “error”. Una pantalla puede estar cargando, puede haber terminado correctamente sin resultados, puede recibir un error esperado de validación o puede sufrir un fallo técnico de una dependencia. Distinguir esos casos permite seguir el código correcto en vez de atribuir cualquier mensaje negativo al backend.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carga
-> la operación todavía no terminó

Estado vacío
-> terminó bien, pero no hay elementos útiles
```

```

Error esperado
-> Laravel o la interfaz conocen la causa y pueden expresarla de forma controlada

Fallo técnico
-> excepción inesperada o dependencia no disponible

Navegación inexistente
-> Vue Router no encontró una View válida

```

Método de lectura usado en esta sección: causa -> archivo que la detecta -> estado/request -> respuesta -> componente que consume -> condición del template -> mensaje visible.

CineVerse diferencia errores esperados —credenciales inválidas, reseña duplicada, validación, 404— de fallos de infraestructura o errores internos. La interfaz intenta mostrar un mensaje útil al usuario mientras el detalle técnico se conserva en logs cuando aporta valor de diagnóstico.

29.1 Errores de API

Los controladores de reseñas devuelven códigos semánticos como `REVIEW_NOT_FOUND`, `NOT_REVIEW_OWNER`, `REVIEW_ALREADY_EXISTS` o `REVIEW_CREATED` junto con estados HTTP apropiados. El frontend puede mapear esos códigos a mensajes de vue-i18n sin exponer excepciones o SQL.

29.2 Base de datos no disponible

`bootstrap/app.php` detecta `QueryException` cuyo mensaje contiene `SQLSTATE[HY000] [2002]`. Para requests JSON devuelve 503 con code `DATABASE_UNAVAILABLE`. Para navegación web devuelve `resources/views/errors/database.blade.php`. Así una caída de MySQL no termina necesariamente en una página de stack trace incomprensible.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

API
{
  "success": false,
  "code": "DATABASE_UNAVAILABLE",
  "message": "La base de datos no esta disponible."
}
HTTP 503

Web
-> errors.database Blade
-> HTTP 503

```

29.3 404 del frontend

Vue Router contiene un `catch-all` localizado y otro global para renderizar `NotFoundView`. Laravel, a su vez, entrega CineVerse para rutas profundas localizadas. Esto permite que una URL como `/es/loquesea` llegue a la SPA y muestre la 404 visual de CineVerse en lugar de fallar antes de montar Vue.

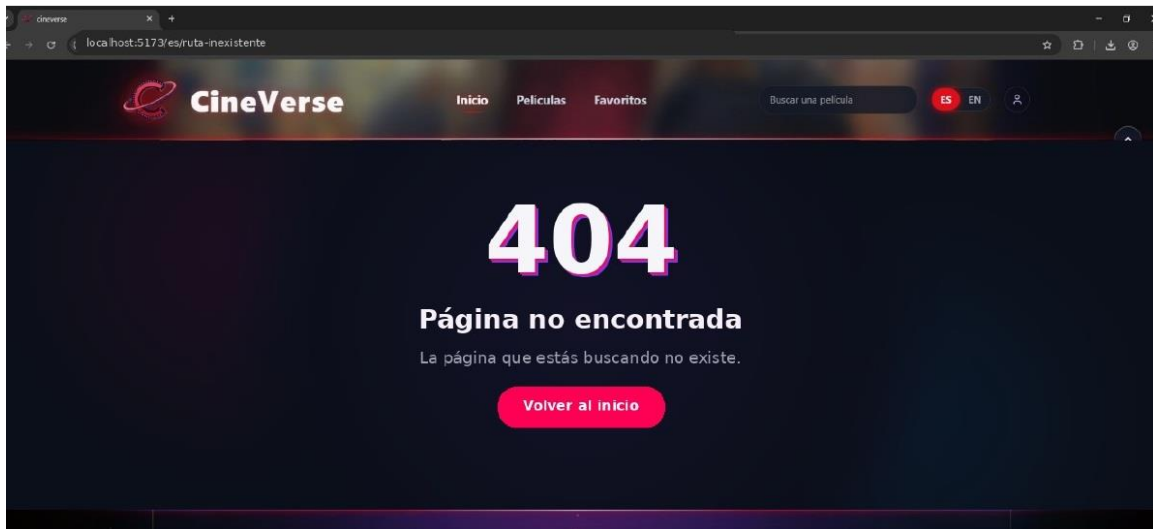


Figura. Vista 404 mostrada por Vue Router.

29.4 Errores esperados no saturan el log

El report handler de bootstrap/app.php evita registrar ValidationException y errores HTTP esperados como 403, 404, 419 o 422. El canal propio queda reservado principalmente para fallos 500 y base no disponible. Esa decisión mejora la relación señal/ruido del archivo de log.

29.5 Cómo leer un estado desde la causa hasta la pantalla

En Vue no existe necesariamente una página distinta para cada estado. Una ref, un resultado de Vue Query, un store o una respuesta HTTP cambia; Vue vuelve a evaluar el template y v-if/v-else decide qué bloque queda visible. Si el texto está traducido, vue-i18n resuelve la clave final.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
CAUSA
-> archivo que produce/detecta
-> ref / store / Vue Query / response HTTP
-> View o Component que consume
-> v-if / catch / computed
-> t("...")
-> MENSAJE QUE VE CARLOS
```

29.6 Ejemplo: por qué aparece “Cargando película”

MovieDetailView separa dos condiciones: la query de useMovieDetail puede seguir cargando y MovieHero puede no haber terminado de cargar el póster. Por eso la llegada del JSON no retira por sí sola el overlay.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
MovieDetailView
-> useMovieDetail(id, locale)
-> isLoading = true mientras Vue Query espera TMDB
-> heroReady = false

TMDB responde
-> isLoading = false
-> MovieHero carga poster
-> onPosterLoad()
-> espera 150 ms
-> emit("ready")
-> onHeroReady() -> heroReady = true
-> isLoading || !heroReady = false
-> desaparece “Cargando película”
```

Si `poster_path` no existe, MovieHero considera esa parte lista para que una imagen ausente no deje el loader bloqueado indefinidamente.

29.7 Ejemplo: “No encontramos películas” no significa que TMDb esté caído

MovieResults recibe `data.pages` de `useInfiniteQuery`, une `results` con `flatMap` y descarta películas sin `poster_path` utilizable. `hasProcessedData` se activa cuando termina ese procesamiento. El mensaje vacío sólo aparece cuando no existe error, el procesamiento terminó y `validMovies` quedó vacío.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
TMDb responde correctamente
-> data.pages
-> flatMap(results)
-> filtrar poster_path inválido
-> hasProcessedData = true
-> validMovies.length === 0
-> estado vacío: “No encontramos películas”
```

Un fallo real de la query sigue el camino `error/ isError`. Visualmente ambos caminos pueden ser negativos, pero técnicamente no significan lo mismo.

29.8 Ejemplo: Favoritos vacío se resuelve sin HTTP

Favoritos utiliza Pinia persistido en `localStorage`. Si `favorites.length` es 0, `MoviesFavoritesView` muestra el estado vacío. No intervienen Axios, Laravel, MySQL ni TMDb.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
localStorage
-> pinia-plugin-persistedstate
-> useFavoritesStore
-> favorites = []
-> MoviesFavoritesView
-> estado vacío
```

29.9 Ejemplo: perfil válido con cero reseñas

Este caso sí atraviesa Laravel. `GET /api/profile` puede responder correctamente con `reviews_count = 0` y `latest_reviews = []`. `ProfileView` interpreta el array vacío y muestra el mensaje correspondiente. No es un 404 ni un 500.

```
ProfileView
-> profile.service.ts
-> GET /api/profile
-> middleware web + auth
-> Api\ProfileController::show()
-> Review::count() = 0
-> latestReviews = []
-> JSON success
-> Vue: latestReviews.length === 0
-> mensaje de perfil sin reseñas
```

29.10 Ejemplo guiado: Carlos escribe una contraseña incorrecta

LoginView primero ejecuta validación de formulario. Si email y password tienen forma válida, envía `POST /login`. `LoginRequest` es quien puede decidir si la contraseña coincide con el hash del usuario en MySQL.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Ingresar
-> LoginView.validateLoginForm()
-> loginUser()
-> POST /login
-> routes/auth.php
-> LoginRequest::rules()
-> LoginRequest::authenticate()
-> User::where(email)->first()
```

```

-> Hash::check() = false
-> RateLimiter::hit()
-> ValidationException errors.email
-> Laravel responde 422
-> Axios rechaza Promise
-> LoginView catch detecta errors.email
-> formError = "invalidCredentials"
-> vue-i18n
-> Carlos ve "Usuario o contraseña incorrectos."

```

El mismo formulario trata 429 como bloqueo temporal, 403 como email no verificado y status ≥ 500 como error interno. Es decir, la vista no depende de una única cadena de texto del servidor; clasifica según status y estructura de validación.

29.11 Ejemplo: enlace de recuperación inválido

En la versión Laravel actual ResetPasswordView considera invalid la URL cuando falta token o email. Cuando ambos valores existen muestra el formulario sin hacer una validación remota en onMounted. La verificación efectiva ocurre al enviar POST /reset-password: Password::reset valida el token y Laravel responde con error de validación si no es utilizable.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

CASO A
falta token o email
-> ResetPasswordView
-> tokenStatus = "invalid"
-> no hay HTTP

CASO B
hay token/email, Carlos envía nueva contraseña
-> POST /reset-password
-> NewPasswordController
-> Password::reset()
-> token inválido/expirado
-> ValidationException
-> Axios catch
-> interfaz muestra error controlado

```

29.12 Estados HTTP y estructura de errores del backend Laravel

Laravel combina status HTTP con JSON o con la estructura estándar de errores de validación. ReviewController devuelve códigos propios para reglas de negocio; ValidationException devuelve errors por campo; el manejo global de base caída devuelve un contrato específico DATABASE_UNAVAILABLE.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

Ejemplos reales
401 / auth middleware      -> usuario no autenticado
403 NOT_REVIEW_OWNER      -> intenta modificar reseña ajena
409 REVIEW_ALREADY_EXISTS -> duplicado controlado
422 ValidationException    -> datos inválidos
429 LoginRequest          -> RateLimiter bloqueado
500                       -> fallo interno inesperado
503 DATABASE_UNAVAILABLE  -> MySQL no disponible

```

29.13 Fallos técnicos y dependencias externas

TMDB puede fallar aunque Laravel y MySQL funcionen; SMTP puede fallar durante registro o recuperación; MySQL puede estar caído antes de que un Controller alcance a ejecutar su consulta. Cada dependencia entra por una capa distinta y no todos los fallos terminan en la misma pantalla.

bootstrap/app.php captura específicamente QueryException con SQLSTATE[HY000] [2002]. Para requests JSON devuelve 503 con DATABASE_UNAVAILABLE; para navegación web devuelve errors.database Blade. En ambos casos el detalle técnico se registra por separado.

29.14 Mapa final de estados

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

404 SPA	-> Vue Router catch-all -> NotFoundView
Cargando película	-> Vue Query + MovieHero -> overlay
Sin películas	-> MovieResults -> estado vacío
Sin favoritos	-> Pinia/localStorage -> estado vacío local
Perfil sin reseñas	-> Laravel/MySQL responde datos vacíos válidos
Credenciales inválidas	-> LoginRequest -> ValidationException -> LoginView
Base caída	-> bootstrap/app.php -> 503 + log técnico
SMTP falla	-> Controller -> CineVerseLog + ValidationException

30. Logs: registro y diagnóstico de errores

El proyecto conserva una capa de log propia para que los errores importantes queden registrados con un formato reconocible. CineVerseLog gestiona el log y CineVerseLogFormatter controla la representación. La ruta base se configura mediante CINEVERSE_LOG_PATH, cuyo ejemplo apunta a storage/logs.

30.1 Qué contexto se registra

El manejador global de excepciones —el handler configurado en bootstrap/app.php— intenta identificar controller y method a partir de la acción de la ruta. También puede incluir status, user_id, email, username o movie_id cuando están disponibles. Esto permite investigar un error sin publicar esos detalles en la respuesta enviada al navegador.

30.2 Errores de correo

RegisteredUserController captura Throwable alrededor del envío de verificación. Si el SMTP falla, registra un mensaje de fallo de correo con contexto y elimina el usuario recién creado para no dejar una cuenta que el flujo espera verificar pero cuyo correo nunca salió. Otros controladores de email aplican la misma idea de registrar y devolver un mensaje controlado.

30.3 No confundir log con mensaje al usuario

La interfaz necesita textos breves y accionables; el log necesita contexto técnico. Una respuesta puede decir que el servicio no está disponible mientras el archivo registra QueryException, controlador y status. Separar ambas responsabilidades evita filtrar detalles internos y, al mismo tiempo, conserva información de diagnóstico.

30.4 Laravel Logging, Monolog y CineVerseLog no son la misma pieza

CineVerseLog.php pertenece al código de CineVerse y ofrece una entrada simple para registrar fallos técnicos. No abre archivos directamente. Internamente utiliza la facade Log de Laravel y selecciona el canal cineverse_unexpected. Laravel configura ese canal y utiliza Monolog por debajo para escribir el archivo.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
código de CineVerse / bootstrap
-> CineVerseLog::error(...)
-> Illuminate\Support\Facades\Log
-> channel("cineverse_unexpected")
-> config/logging.php
-> driver daily
-> Monolog handler
-> CineVerseLogFormatter
-> storage/logs/cineverse-backend-AAAA-MM-DD.log
```

Esta diferencia es importante respecto del backend PHP anterior: ya no existe una clase que construya manualmente Logger y RotatingFileHandler. Laravel crea el canal a partir de config/logging.php y sigue utilizando Monolog internamente.

30.5 El canal `cineverse_unexpected` en `config/logging.php`

Ubicación en el proyecto: `config/logging.php` · **Configuración Laravel** · canal `cineverse_unexpected`

Lo utiliza: `facade Log / CineVerseLog`. *Responsabilidad:* definir archivo diario, nivel y formatter del canal propio.

```
'cineverse_unexpected' => [
    'driver' => 'daily',
    'path' => $cineverseLogPath,
    'level' => 'error',
    'days' => 30,
    'tap' => [App\Logging\CineVerseLogFormatter::class],
    'replace_placeholders' => true,
],
```

`driver=daily` indica rotación diaria. `level=error` hace que este canal no funcione como historial general de acciones INFO. `days=30` conserva hasta 30 archivos rotados. `tap` permite modificar el Logger ya creado por Laravel y aplicar el formatter propio.

30.6 Cómo se calcula la ruta del archivo

`config/logging.php` lee `CINEVERSE_LOG_PATH`. Si contiene una ruta absoluta de Windows o Linux se utiliza directamente. Si es relativa, Laravel la combina con `base_path()`. Si la variable está vacía se usa `storage_path("logs")`. Después se agrega el nombre base `cineverse-backend.log`.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
CINEVERSE_LOG_PATH=storage/logs
-> base_path("storage/logs")
-> ../storage/logs/cineverse-backend.log
-> driver daily
-> cineverse-backend-2026-09-19.log
```

El nombre final con fecha lo produce el driver `daily`. No hay que construir la fecha manualmente dentro de `CineVerseLog`.

30.7 `CineVerseLogFormatter`: formato de cada entrada

`app/Logging/CineVerseLogFormatter.php` recibe el Logger de Laravel y recorre sus handlers. A cada handler le aplica un `LineFormatter` de Monolog con una estructura corta.

Ubicación en el proyecto: `app/Logging/CineVerseLogFormatter.php` · **Clase de logging** ·

`App\Logging\CineVerseLogFormatter`

Lo utiliza: canal `cineverse_unexpected`. *Responsabilidad:* dar formato corto y estable a cada línea del log.

```
new LineFormatter(
    "[%datetime%] %level_name%: %message%\n",
    "Y-m-d H:i:s",
    true,
    true
)
```

`%datetime%` se convierte en fecha y hora, `%level_name%` en ERROR y `%message%` en el texto que `CineVerseLog` entregó. `allowInlineLineBreaks=true` permite que el bloque JSON de contexto conserve saltos de línea.

30.8 `CineVerseLog::error()`: mensaje principal + contexto JSON

`CineVerseLog::error(message, context)` llama `Log::channel("cineverse_unexpected")->error()`. Antes de hacerlo, `formatContext` elimina claves cuyo valor sea null y transforma el resto en JSON legible con `JSON_PRETTY_PRINT`, `JSON_UNESCAPED_UNICODE` y `JSON_UNESCAPED_SLASHES`.

Ejemplo didáctico de invocación; no es un bloque literal del código fuente. El método real está en `app/Support/CineVerseLog.php` · clase `App\Support\CineVerseLog` · `error(string $message, array $context)`.

```
CineVerseLog::error($exception->getMessage(), [
    "controller" => "ReviewController",
    "method" => "store",
```

```

    "user_id" => 42,
    "movie_id" => 550,
  ]);

```

El array no se entrega a Monolog como contexto estructurado separado: CineVerseLog lo convierte a texto y lo concatena al mensaje. Por eso el archivo final muestra primero la causa y debajo un bloque JSON.

30.9 Qué termina escrito físicamente

```

[2026-09-19 12:30:15] ERROR: mensaje técnico
{
  "controller": "ReviewController",
  "method": "store",
  "user_id": 42,
  "movie_id": 550
}

```

La primera parte proviene del LineFormatter. El bloque JSON proviene de CineVerseLog::formatContext(). El navegador no recibe automáticamente este texto; el log está destinado al diagnóstico interno.

30.10 bootstrap/app.php: reportar y renderizar son dos responsabilidades

Laravel separa el reporte de una excepción de la respuesta que se presenta al cliente. CineVerse aprovecha withExceptions() para personalizar ambos recorridos.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```

EXCEPCIÓN
-> report() decide si se registra y con qué contexto
-> render() decide si existe una respuesta especial
-> response final al navegador / Axios

```

La función de reporte registrada como callback de report() identifica dos grupos que entran en el canal propio: base de datos no disponible y errores internos inesperados. ValidationException y errores HTTP esperados como 403, 404, 419 o 422 no se escriben como ERROR por ese camino.

30.11 Cómo report() obtiene el contexto

Ese callback —una función que Laravel ejecuta cuando reporta una excepción— toma el Request actual y su route. Cuando la acción tiene forma Controller@method separa ambos nombres; en consola usa Console/artisan; para una closure identifica la ruta. Después agrega status, user_id, email, username y movie_id cuando existen.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

Ubicación en el proyecto: bootstrap/app.php · callback report() configurado en withExceptions(). Utiliza App\Support\CineVerseLog para registrar contexto técnico antes de decidir la respuesta al cliente.

```

CineVerseLog::error($exception->getMessage(), [
  "controller" => $controller,
  "method" => $method,
  "status" => $isDatabaseUnavailable ? 503 : 500,
  "user_id" => optional($request->user())->id,
  "email" => $request->input("email"),
  "username" => $request->input("username"),
  "movie_id" => $request->route("movieId") ?? $request->input("movie_id"),
]);

```

El callback devuelve false después de registrar. En esta configuración eso evita que Laravel vuelva a duplicar el mismo reporte en el canal default.

30.12 render(): respuesta especial cuando MySQL no está disponible

El render específico sólo intercepta QueryException cuyo mensaje contiene SQLSTATE[HY000] [2002]. Si la request espera JSON o pertenece a api/* devuelve una respuesta 503; si es navegación web devuelve la vista errors.database con el mismo status.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Request JSON /api/...
-> HTTP 503
-> {
  "success": false,
  "code": "DATABASE_UNAVAILABLE",
  "message": "La base de datos no esta disponible."
}

Navegación web
-> resources/views/errors/database.blade.php
-> HTTP 503
```

30.13 Ejemplo completo: MySQL caído

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos abre una pantalla que necesita MySQL
-> Eloquent intenta consultar
-> QueryException SQLSTATE[HY000] [2002]
-> withExceptions report()
  -> CineVerseLog::error(...)
  -> archivo diario conserva la causa
-> withExceptions render()
  -> JSON 503 o Blade 503 según request
-> Carlos ve una respuesta controlada, no el stack trace técnico
```

Así se separan dos públicos: el desarrollador conserva datos de diagnóstico; la interfaz recibe una respuesta segura y estable.

30.14 Ejemplo completo: falla SMTP durante el registro

RegisteredUserController crea el User y llama sendEmailVerificationNotification() dentro de un try/catch. Si el envío falla, CineVerseLog::mailFailureMessage() resume determinados errores de autenticación o conexión SMTP para evitar textos enormes, registra controller/method/username/email, elimina el usuario recién creado y lanza una ValidationException.

Recorrido paso a paso (esquema explicativo; no corresponde a un bloque literal de un archivo):

```
Carlos pulsa Crear cuenta
-> User::create()
-> sendEmailVerificationNotification()
-> SMTP falla
-> catch(Throwable)
-> CineVerseLog::mailFailureMessage()
-> CineVerseLog::error(... contexto ...)
-> $user->delete()
-> ValidationException auth.email_delivery_failed
-> formulario muestra error controlado
```

PasswordResetLinkController aplica la misma idea al solicitar un correo de recuperación: registra el fallo técnico y devuelve un error de validación en lugar de exponer la excepción SMTP completa.

30.15 Qué se registra y qué no

Un resultado negativo no implica automáticamente un log ERROR. Credenciales incorrectas, rating fuera de rango, reseña duplicada, 404 o 422 son resultados esperados y el sistema sabe clasificarlos. El canal cineverse_unexpected se reserva para errores internos, base caída y fallos de correo registrados explícitamente.

- Sí: excepción interna inesperada que termina como 500.
- Sí: MySQL no disponible detectado como 503.
- Sí: fallos de correo capturados en registro o recuperación.
- No por el reporte global: ValidationException, 403, 404, 419, 422.

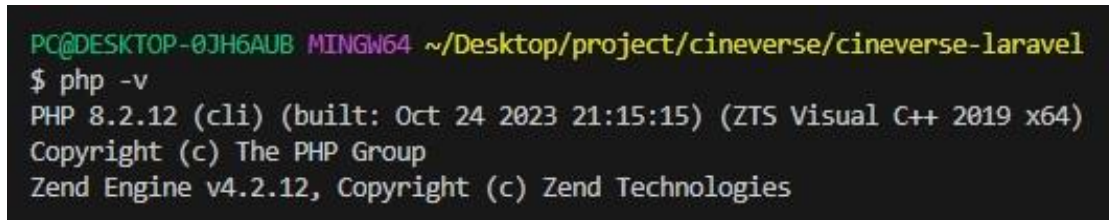
El contexto no debe incluir contraseñas, tokens CSRF ni tokens de verificación/reset. Los archivos de storage/logs tampoco forman parte del contenido público servido desde public/.

31. Instalación local de CineVerse Laravel

Esta sección parte del escenario de una persona que clona o descarga CineVerse Laravel desde GitHub en una computadora nueva. El repositorio contiene el código y los archivos de definición, pero no incluye vendor, node_modules, .env ni public/build porque .gitignore los excluye. La instalación reconstruye esas piezas y prepara MySQL.

31.1 Requisitos del equipo

- PHP compatible con composer.json (^8.2).
- Composer disponible desde la terminal.
- Node.js y npm disponibles para instalar y ejecutar Vite.
- MySQL accesible y un usuario con permisos sobre la base local.
- Un API Read Access Token de TMDB para VITE_TMDB_TOKEN.
- Configuración SMTP si se quieren probar correos reales de verificación y recuperación.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
```

Figura. Comprobación de PHP 8.2.12 en el entorno local.

31.2 Abrir el proyecto recién obtenido

En un repositorio recién clonado deberían verse app, bootstrap, config, database, public, resources, routes, storage, tests, composer.json, package.json y vite.config.js, entre otros. vendor, node_modules y .env aparecen sólo después de preparar el entorno local.

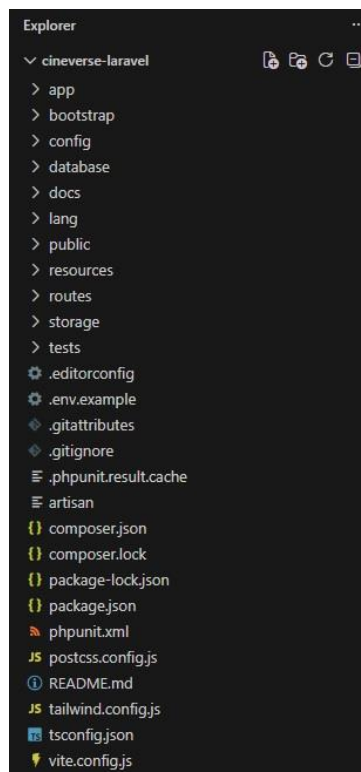


Figura. Estructura local del proyecto CineVerse Laravel.

31.2.1 Qué contiene el repositorio y qué se reconstruye en una instalación limpia

Durante el desarrollo pueden existir también archivos generados o locales, entre ellos `.env`, `public/build`, `public/hot`, `logs` y `datos` generados durante la ejecución bajo `storage`. `.gitignore` excluye esas rutas del versionado normal. Por eso, al trabajar desde un repositorio recién clonado, `.env`, `vendor`, `node_modules` y los `assets` compilados se reconstruyen en el equipo donde se instala.

31.3 Instalar dependencias PHP

```
composer install
```

Composer lee `composer.lock` e instala las versiones bloqueadas dentro de `vendor`. También ejecuta los scripts `post-autoload-dump` de Laravel y `package:discover`. No hace falta instalar Laravel o Inertia manualmente paquete por paquete.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ composer install
```

Figura. Ejecución de `composer install` desde la raíz de `cineverse-laravel`

31.4 Instalar dependencias JavaScript

```
npm install
```

`npm` reconstruye `node_modules` y deja disponibles `Vite`, `Vue`, `TypeScript` y el resto de dependencias. Ejecutar `npm run dev` antes de `npm install` puede producir el error “`vite` no se reconoce” porque el ejecutable local todavía no existe.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ npm install
```

Figura. Instalación de dependencias JavaScript desde la raíz del proyecto

31.5 Crear `.env` desde la plantilla

```
cp .env.example .env
```

El archivo `.env` es local y no debe subirse al repositorio. La copia inicial contiene valores de ejemplo; después se actualizan `APP_URL`, `MySQL`, `SMTP` y `TMDB` según el equipo.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ cp .env.example .env
```

Figura. Creación de `.env` a partir de `.env.example`

31.6 Generar `APP_KEY`

```
php artisan key:generate
```

El comando escribe `APP_KEY` dentro de `.env`. No es un paso opcional de documentación: una instalación Laravel necesita una clave válida para las operaciones criptográficas del framework.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ php artisan key:generate

INFO Application key set successfully.
```

Figura. Laravel confirma que la clave de aplicación fue generada correctamente

31.7 Crear la base vacía

```
CREATE DATABASE cineverse_laravel
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

La base se crea vacía. Las tablas no se dibujan manualmente en Workbench: las migraciones son la fuente reproducible del esquema.

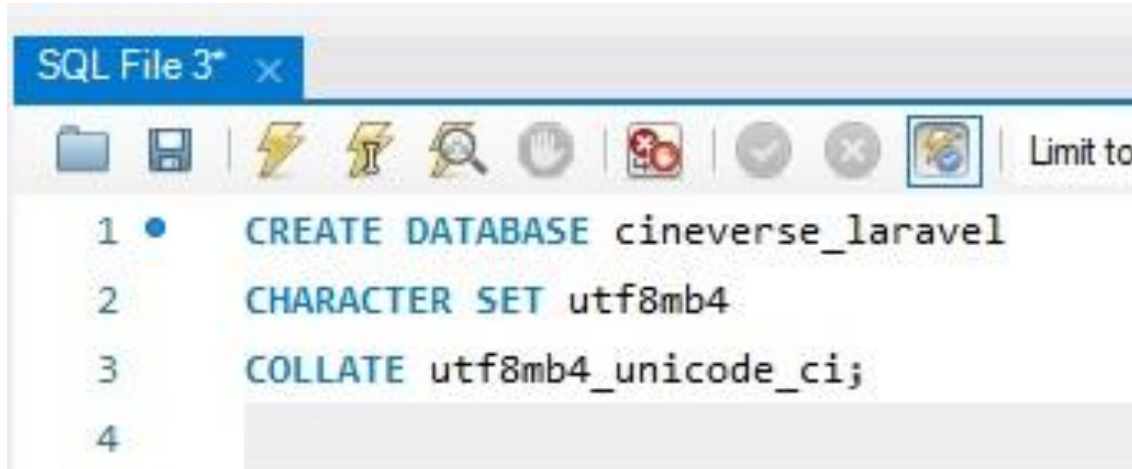


Figura. Creación manual de la base cineverse_laravel en MySQL Workbench antes de las migraciones

31.8 Configurar MySQL y TMDb en .env

Ubicación de referencia: .env.example en la raíz del proyecto. Este bloque muestra qué valores debe completar Carlos en su .env local para MySQL y TMDb; usuario, contraseña y token son marcadores.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
DB_USERNAME=<usuario>
DB_PASSWORD=<contraseña>

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN=<token>
```

Los valores deben corresponder al equipo real. La documentación usa marcadores para usuario, contraseña y token; no se publican credenciales. Si se cambia una variable VITE_* con Vite en ejecución, se reinicia npm run dev.

31.9 Ejecutar migraciones

```
php artisan migrate
```

Laravel crea users, password_reset_tokens, sessions, cache, jobs, personal_access_tokens, reviews y el registro migrations. Si este paso falla por conexión, se revisan DB_HOST, DB_PORT, DB_DATABASE, usuario, contraseña y que MySQL esté levantado.

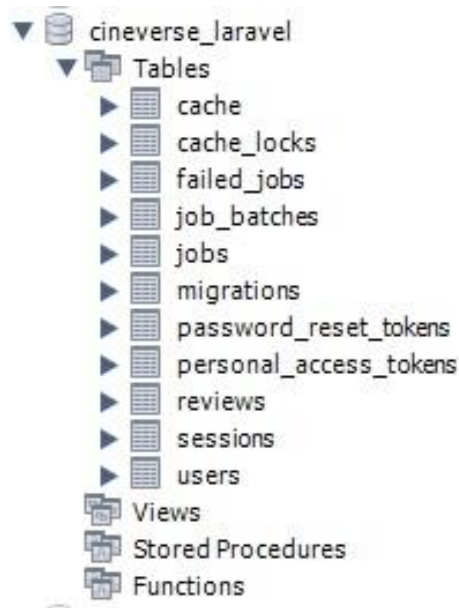


Figura. Tablas de cineverse_laravel después de ejecutar las migraciones.

31.10 Cargar datos de prueba

```
php artisan db:seed
```

DatabaseSeeder llama UserSeeder y ReviewSeeder. Estos seeders exigen tablas vacías: si ya contienen datos lanzan un RuntimeException y recomiendan migrate:fresh --seed. En un entorno de prueba limpio se cargan exactamente 5.000 usuarios y 24.322 reseñas.

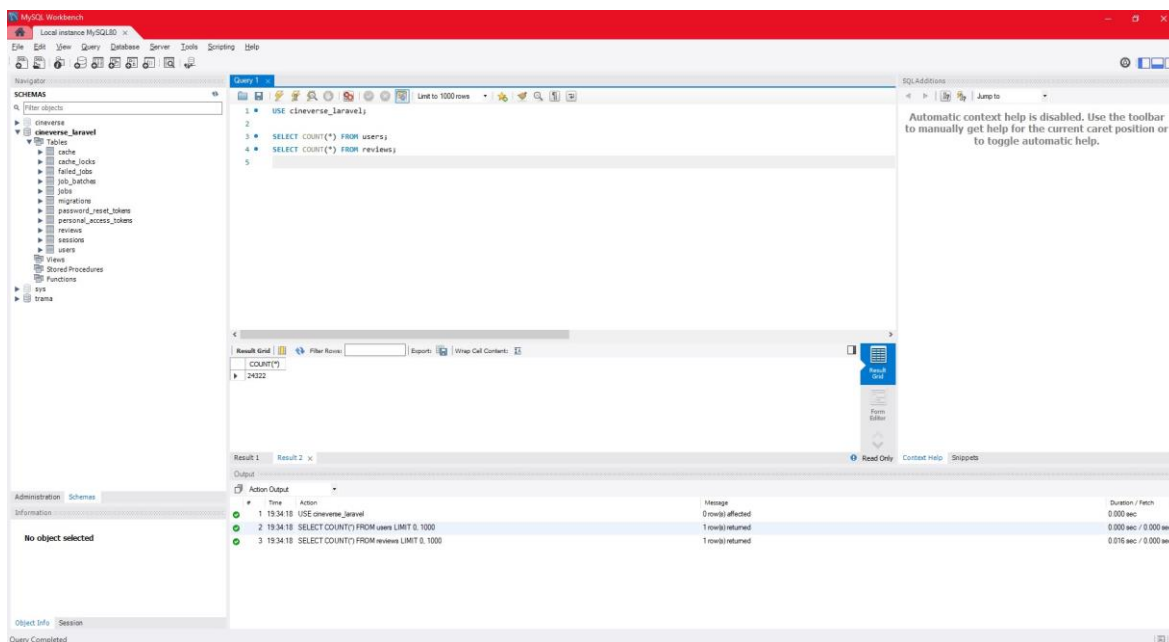


Figura. Comprobación de datos cargados en la base de CineVerse Laravel

31.11 Script composer setup

composer.json incluye un script setup que agrupa composer install, creación condicional de .env, key:generate, migrate --force, npm install y npm run build. Es útil como atajo, pero no ejecuta php artisan db:seed: los 5.000 usuarios y las 24.322 reseñas de prueba sólo se cargan cuando se ejecutan los seeders. El recorrido anterior se documenta paso a paso porque muestra qué responsabilidad tiene cada comando y permite detectar con claridad dónde falla una instalación.

```
composer run setup
```

32. Cómo ejecutar CineVerse Laravel en desarrollo

Una vez completada la instalación, no se repiten composer install, npm install, migraciones ni seeders cada vez que se abre el proyecto. El trabajo diario consiste principalmente en mantener Laravel y Vite en ejecución. MySQL también debe estar disponible porque los datos propios, el caché configurado y otros servicios del proyecto utilizan la base. El lugar donde se guardan las sesiones depende de SESSION_DRIVER: .env.example propone database; si se configura file, Laravel guarda las sesiones en storage/framework/sessions.

32.1 Terminal 1: servidor Laravel

```
php artisan serve
```

Artisan inicia el servidor de desarrollo de Laravel. En este proyecto, al ejecutar php artisan serve, la aplicación queda disponible normalmente en `http://127.0.0.1:8000`. Esa es la URL principal de CineVerse durante el desarrollo: Laravel recibe las solicitudes del navegador, entrega la aplicación mediante Inertia y responde las rutas de autenticación y de la API.

32.2 Terminal 2: Vite

```
npm run dev
```

Vite compila y sirve los assets del frontend con recarga durante el desarrollo. El puerto que muestra Vite —por ejemplo 5173— no reemplaza la URL principal de Laravel en esta integración. `app.blade.php` utiliza `@vite` y el plugin de Laravel conoce cómo enlazar ambos procesos.

32.3 Los dos procesos simultáneos

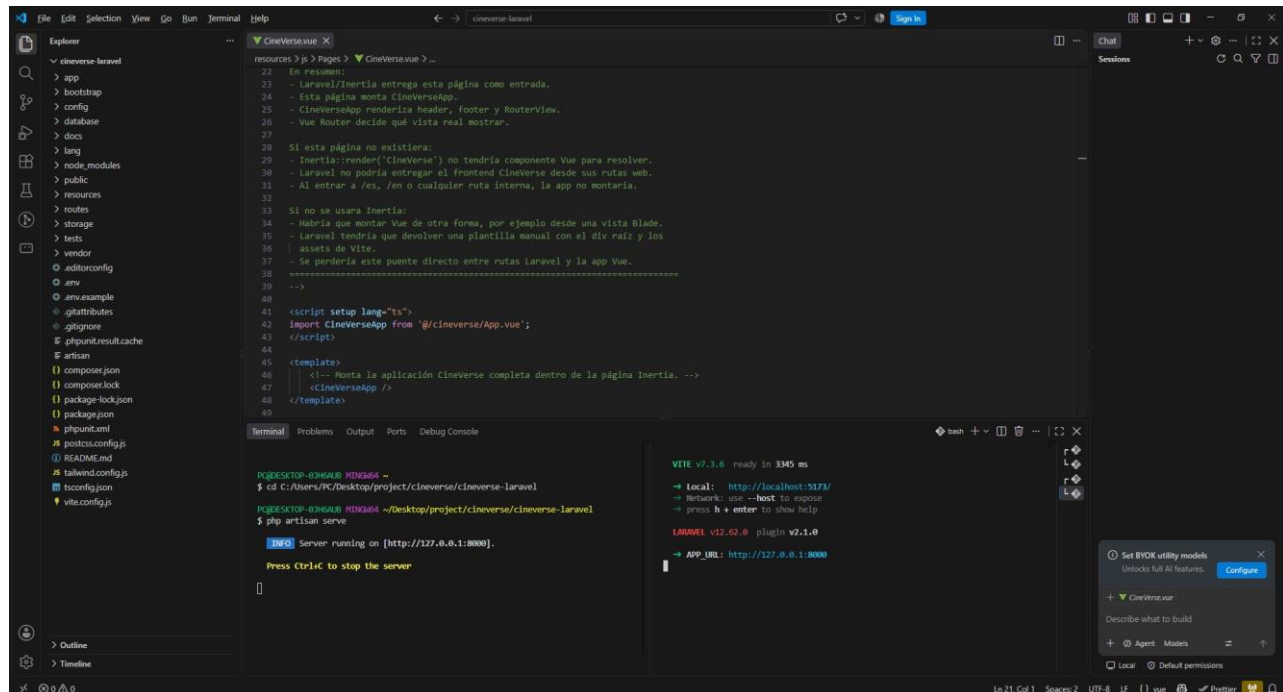


Figura. Laravely Vite ejecutándose durante el desarrollo.

La captura muestra una característica útil de la arquitectura: mientras Laravel atiende `http://127.0.0.1:8000`, Vite mantiene los assets. `resources/js/Pages/CineVerse.vue` no sustituye al frontend; monta `CineVerseApp` y `Vue Router` decide la vista interna.

32.4 Abrir la aplicación

Con ambos procesos activos, se abre `http://127.0.0.1:8000`. `routes/web.php` redirige `/a/es` y el frontend carga la Home. También es válido entrar directamente a una ruta interna localizada porque el catch-all de Laravel entregará la SPA.

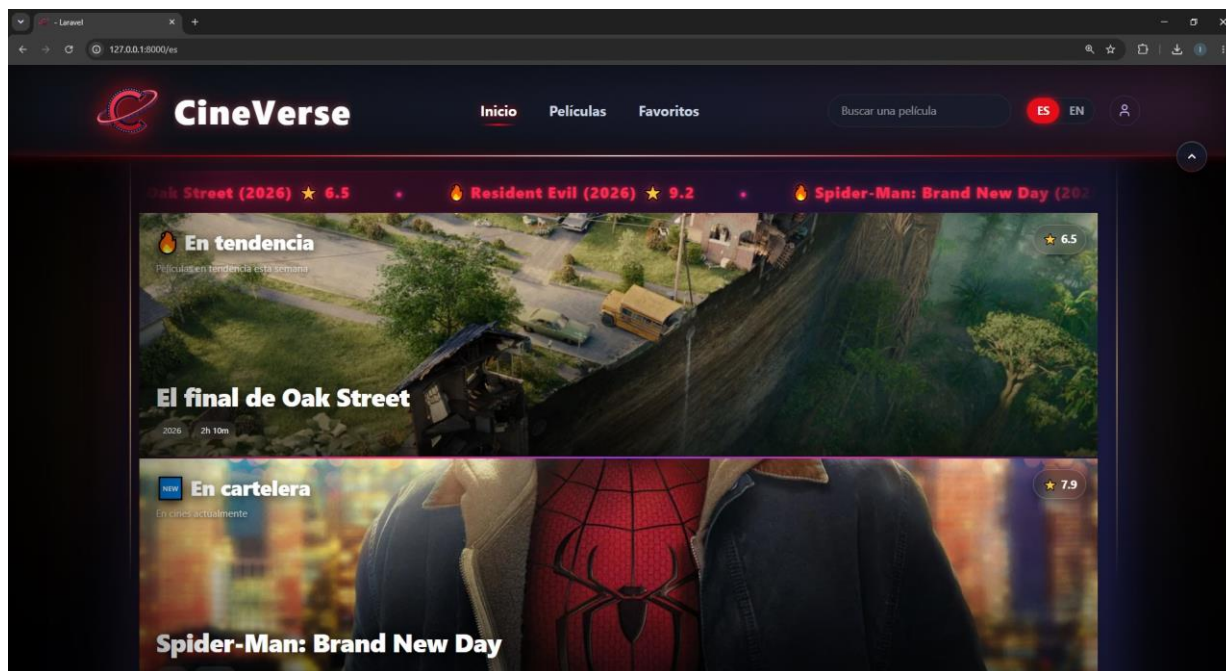


Figura. CineVerse funcionando desde la URL servida por Laravel.

32.5 Error ViteManifestNotFoundException

Si Laravel no detecta el servidor Vite y tampoco existe `public/build/manifest.json`, puede aparecer `ViteManifestNotFoundException`. En desarrollo la solución normal es ejecutar `npm run dev` y mantener la terminal abierta. En producción se ejecuta `npm run build` para generar `public/build` y su manifest.

32.6 Alternativa `composer run dev`

`composer.json` también ofrece un script dev basado en concurrently. Levanta `php artisan serve`, `queue:listen`, `pail` y `npm run dev` en una sola ejecución. Para aprender y diagnosticar el proyecto es más claro comenzar con dos terminales separadas; después el script conjunto puede resultar cómodo.

```
composer run dev
```

33. Cómo publicar CineVerse Laravel en un servidor

La publicación de Laravel no es simplemente copiar la carpeta de desarrollo. El servidor debe disponer de PHP compatible, extensiones requeridas, Composer o un vendor preparado, MySQL, permisos de escritura sobre `storage/` y `bootstrap/cache/`, y un web root que apunte a `public/`. Los assets Vue deben compilarse antes de servir el sitio sin el servidor Vite de desarrollo.

33.1 Configuración de producción

Ejemplo recomendado de configuración de producción; no es un bloque literal de un archivo del proyecto. Se basa en las variables admitidas por `.env.example` y `config/*.php` y debe adaptarse al hosting real sin publicar secretos. Las variables `VITE_TMDB_*` deben estar definidas antes de ejecutar `npm run build` porque Vite las incorpora al frontend compilado.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://dominio.example
```

```
DB_CONNECTION=mysql
DB_HOST=<host-produccion>
DB_PORT=3306
DB_DATABASE=<base-produccion>
DB_USERNAME=<usuario-produccion>
DB_PASSWORD=<secreto>

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

SESSION_SECURE_COOKIE=true
```

Los valores concretos dependen del hosting. APP_DEBUG debe quedar false para evitar exponer stack traces. Las credenciales no se guardan en Git. Si la aplicación se sirve por HTTPS, la cookie de sesión debe configurarse de forma coherente con el dominio y HTTPS. También debe configurarse SMTP si se utilizarán la verificación de correo y la recuperación de contraseña.

33.2 Dependencias y optimización

```
composer install --no-dev --optimize-autoloader
npm install
npm run build
```

npm run build ejecuta primero vue-tsc y después vite build. Si TypeScript detecta un error, el build se detiene antes de generar assets. Antes de este comando deben existir VITE_TMDB_BASE_URL y VITE_TMDB_TOKEN con los valores que utilizará el frontend publicado. Vite produce public/build, que incluye manifest.json. En producción no se deja npm run dev corriendo como requisito para servir la aplicación.

33.3 Migraciones

```
php artisan migrate --force
```

migrate --force permite ejecutar migraciones en production sin la confirmación interactiva que Laravel utiliza como protección. No se recomienda migrate:fresh en un servidor con datos reales porque elimina tablas. Los seeders de prueba tampoco deben ejecutarse automáticamente sobre una base que deba conservar información real.

33.4 Web root y rewrite

El documento raíz del servidor web debe apuntar a public/, donde se encuentra index.php. Laravel recibe las rutas que no corresponden a archivos estáticos y las procesa a través del front controller. Exponer la raíz completa del repositorio como web root aumentaría el riesgo de servir archivos que no deben ser públicos.

33.5 No hay SSR de Inertia en esta versión

El proyecto no define un endpoint SSR ni un proceso inertia:start-ssr. La aplicación se monta en el navegador. Por lo tanto, esta guía no incluye un procedimiento SSR. El build requerido aquí es el build Vite normal.

33.6 Comprobaciones posteriores

- Abrir / y confirmar redirección a /es.
- Abrir directamente una ruta profunda como /es/movies/550.
- Comprobar que TMDB responde y carga imágenes.
- Registrar una cuenta de prueba y verificar el correo.
- Iniciar sesión y confirmar que /user reconstruye la sesión.
- Crear/editar/eliminar una reseña de prueba.
- Revisar storage/logs ante cualquier 500 o error de correo.
- Comprobar /up como verificación básica del estado de Laravel (health check).

34. Pruebas y comprobaciones técnicas

El repositorio contiene pruebas PHPUnit heredadas en gran parte del scaffolding de Breeze y pruebas base de Laravel. Es importante distinguir “hay archivos de test” de “la suite está totalmente alineada con la adaptación actual”. En la suite actual se observan varios tests que todavía esperan el comportamiento Breeze original y, por lo tanto, deben actualizarse antes de tratarlos como evidencia de que todo el flujo actual pasa automáticamente.

34.1 Qué es una prueba automatizada

Una prueba automatizada ejecuta código del proyecto y comprueba que el resultado observado coincida con una expectativa escrita de antemano. No reemplaza todas las pruebas manuales, pero permite detectar regresiones de forma repetible.

34.2 PHPUnit dentro de Laravel

Laravel integra PHPUnit mediante `php artisan test`. `composer.json` declara `phpunit/phpunit ^11.5.50` en `require-dev`. `phpunit.xml` prepara variables del entorno de prueba y Laravel carga la aplicación igual que para otros Feature tests, pero utilizando la configuración de testing.

```
composer test
-> php artisan config:clear
-> php artisan test
-> PHPUnit descubre tests
-> Laravel prepara Application
-> cada test ejecuta acciones/asserts
```

34.3 Unit Test y Feature Test

Un Unit Test intenta comprobar una pieza pequeña y aislada. Un Feature Test atraviesa más infraestructura de Laravel: rutas, middleware, Controllers, sesión o base de datos. Los tests heredados de Breeze bajo `tests/Feature/Auth` son Feature tests porque simulan requests y verifican autenticación, redirects o estado del usuario.

34.4 RefreshDatabase y asserts

`RefreshDatabase` prepara la base de datos para que un test no dependa de filas dejadas por otro. Los métodos `assertStatus`, `assertRedirect`, `assertAuthenticated`, `assertGuest` y comprobaciones sobre la base expresan qué espera el test.

Ejemplo abreviado de un Feature test; no corresponde a un bloque literal completo de los archivos de prueba del proyecto. Las pruebas reales están bajo `tests/Feature/Auth` y utilizan helpers de testing de Laravel como `post()`, `assertStatus()` y `assertAuthenticated()`.

```
$response = $this->post("/login", [...]);
$response->assertStatus(204);
$this->assertAuthenticated();

// La expectativa sólo sirve si refleja el comportamiento actual del proyecto.
```

En CineVerse este último punto es esencial: varios tests fueron heredados de Breeze y todavía esperan campos o redirects que la adaptación cambió. La existencia del test no convierte su expectativa en verdad sobre la versión actual.

34.5 Comandos

```
composer test
# equivale a:
php artisan config:clear
php artisan test
```

`phpunit.xml` prepara el entorno de pruebas. Los tests que usan `RefreshDatabase` reconstruyen la base del entorno de test para cada caso según las herramientas de Laravel.

34.6 Qué pruebas existen

Archivo/grupo	Intención
AuthenticationTest	Login, contraseña inválida y logout.
EmailVerificationTest	Pantalla y enlace de verificación.
PasswordResetTest	Solicitud de enlace y reset.
PasswordConfirmationTest	Confirmación de password.
PasswordUpdateTest	Cambio de password autenticado.
RegistrationTest	Registro.
ProfileTest	Perfil, actualización y eliminación de cuenta.
Feature/ExampleTest	Respuesta básica de la aplicación.
Unit/ExampleTest	Prueba unitaria mínima de ejemplo.

34.7 Desalineaciones visibles en la suite de pruebas actual

RegistrationTest todavía envía name y espera que el usuario quede autenticado y sea redirigido a dashboard. RegisteredUserController actual exige username y deliberadamente no inicia sesión: crea la cuenta y envía verificación. Por lo tanto, ese test describe el Breeze original, no el comportamiento actual de CineVerse.

AuthenticationTest crea un User de factory y espera login sin reflejar de forma explícita la nueva exigencia de email verificado. EmailVerificationTest espera un redirect al dashboard?verified=1, mientras VerifyEmailController actual redirige a /{locale}/verify-email?verified=1. ProfileTest todavía utiliza name en varias operaciones, pero ProfileUpdateRequest actual valida username. Estas diferencias muestran que la suite heredada no representa de forma completa el comportamiento actual del proyecto.

Conclusión sobre tests

La documentación no afirma que la suite actual pase completa. El código de producción y las pruebas heredadas muestran una desalineación real. Por ese motivo, el estado actual de PHPUnit no se presenta como prueba definitiva de los flujos implementados.

34.8 Comprobaciones manuales recomendadas

- Home carga tendencias, cartelera y mejor valoradas.
- Cambiar ES/EN mantiene la ruta y actualiza textos/datos localizados.
- Abrir directamente /es/movies/{id} funciona después de F5.
- Registro rechaza username/email duplicados y envía verificación.
- Cuenta no verificada no puede iniciar sesión.
- Login válido reconstruye el store mediante GET /user.
- RateLimiter bloquea después de los intentos configurados.
- Recuperación de contraseña genera enlace y permite establecer una nueva contraseña.
- Usuario autenticado crea una reseña de hasta 1000 caracteres.
- No se puede crear una segunda reseña para la misma película.
- No se puede editar o borrar una reseña ajena mediante petición manual.
- Perfil muestra cantidad y últimas reseñas.
- Favoritos persisten en el navegador y respetan MAX_FAVORITES=20.
- Con MySQL detenido, la aplicación devuelve la contingencia 503 esperada.

34.9 Cierre técnico

CineVerse Laravel combina una interfaz Vue con un backend Laravel sin mezclar las responsabilidades de cada capa. TMDb continúa siendo la fuente del catálogo cinematográfico, mientras que Laravel y MySQL se ocupan únicamente de la información que pertenece a CineVerse, como usuarios, sesiones, perfiles y reseñas.

La arquitectura resultante mantiene separados los datos externos, la lógica propia de la aplicación y la presentación, lo que facilita el mantenimiento del proyecto y permite incorporar nuevas funcionalidades sin modificar innecesariamente las demás capas.